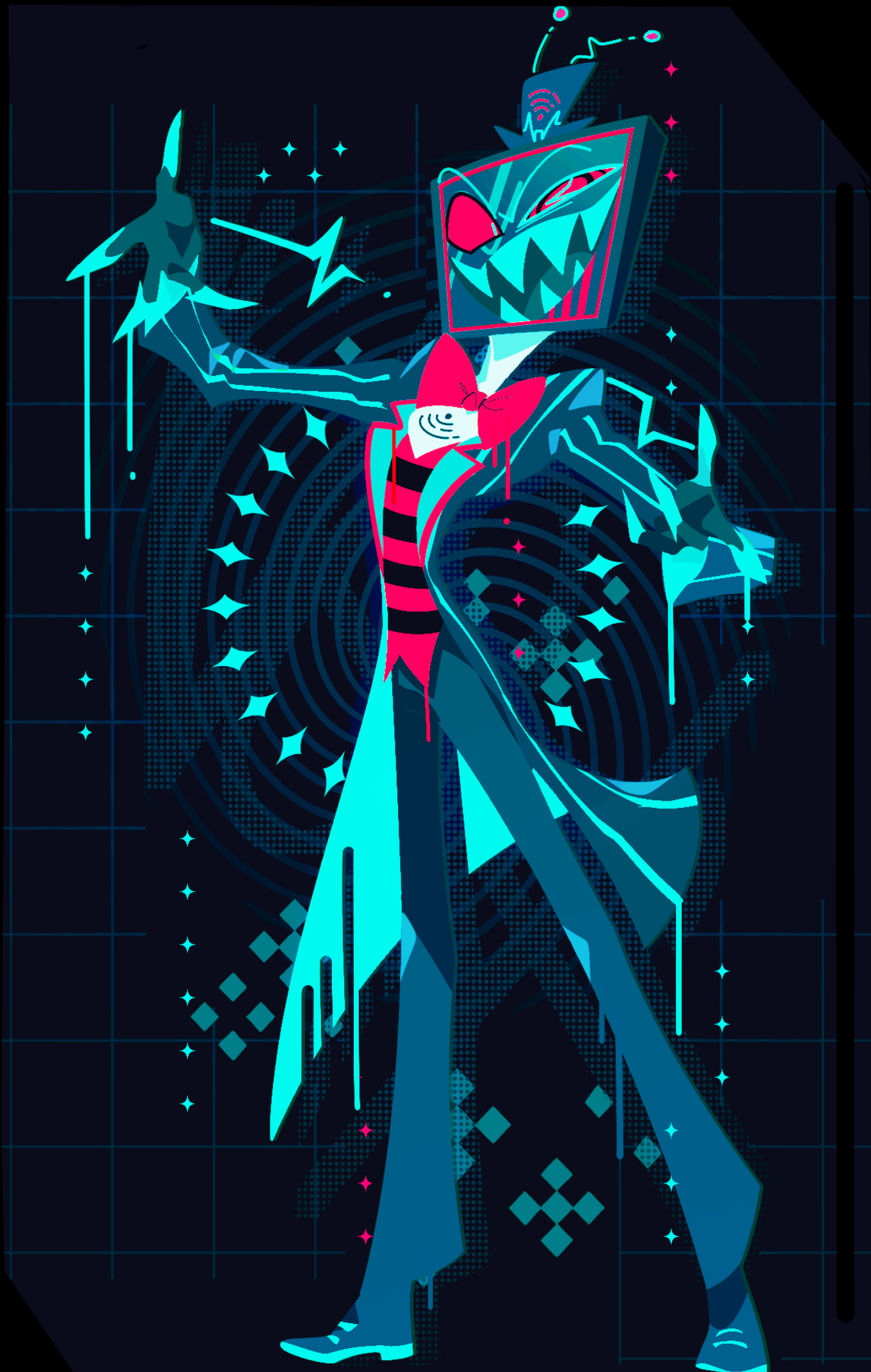




Московский физико-технический институт
Физтех-школа прикладной математики и информатики

Технологии программирования и язык C++

Подробный конспект курса
C++17 с отдельными элементами C++20

Авторы: Хомченко Илья
Шелегеда Наталья

Москва, 2026

Содержание

О конспекте	4
1 Лекция 1. Введение в C++ и первая программа	4
1.1 От задачи к машинному коду	4
1.2 Почему C++	4
1.3 Практический pipeline сборки	5
1.3.1 Preprocessing	5
1.3.2 Object file и linker	5
1.4 Hello, World! по строкам	6
1.5 Первая команда компиляции	6
1.6 Из чего состоит строка C++	7
1.7 Типы как информация о значениях и операциях	7
1.8 Инициализация и арифметика	7
1.9 Сравнения и простые преобразования	8
1.10 Ввод и вывод	9
1.11 Первая стандартная абстракция: <code>std::string</code>	9
1.12 <code>std::vector</code> : последовательность однотипных значений	9
1.13 <code>std::map</code> : ключ и значение	10
1.14 Не смешиваем язык, библиотеку и инструменты	10
1.15 CMake как первая абстракция над сборкой	10
1.16 Что пока остаётся за кадром	11
1.17 Проверка понимания	11
1.18 Источники для сверки	11
2 Лекция 2. Объявления, поток управления и ошибки	11
2.1 Имя, объект, значение и состояние	12
2.2 Declaration, definition, initialization, assignment	12
2.3 Литералы, выражения и преобразования	13
2.3.1 Short-circuit — не только оптимизация	13
2.4 Ветвление <code>if</code> : порядок проверок проектирует алгоритм	13
2.5 <code>switch</code> : выбор по дискретному значению	14
2.6 Циклы: повторение с условием остановки	15
2.6.1 <code>while</code> и <code>validation loop</code>	15
2.6.2 <code>do while</code>	15
2.6.3 <code>for</code> и границы	15
2.6.4 <code>break</code> , <code>continue</code> , <code>sentinel</code> и поиск	16
2.6.5 Loop invariant — человеческое доказательство	16
2.7 Block scope, shadowing и lifetime	17
2.8 Карта ошибок по pipeline	17
2.9 Diagnostics: compiler и sanitizer решают разные задачи	18
2.10 Эксперимент 1: <code>course_status</code>	18
2.11 Эксперимент 2: control flow и реальные diagnostics	19
2.12 Сильные стороны и цена свободы	19
2.13 Проверка понимания	19
2.14 Источники для сверки	19
3 Лекция 3. Указатели, память и массивы	19
3.1 Объект, storage, lifetime и адрес	20

3.2	Pointer basics: адрес и значение по адресу	20
3.2.1	nullptr и действительность	21
3.3	Raw array: фиксированная последовательность	21
3.4	Array-to-pointer conversion: массив не pointer	22
3.5	Pointer arithmetic существует из-за массивов	22
3.6	Коротко о C-style strings	23
3.7	std::array: удобный fixed-size container	23
3.8	std::vector: dynamic size и владение storage	23
3.9	Raw array, std::array или std::vector?	24
3.10	Карта memory errors	24
3.11	ASan и UBSan: наблюдаем часть UB	25
3.12	Производительность без лозунгов	25
3.13	Эксперимент 1: course_inventory	25
3.14	Эксперимент 2: array_vs_vector_lab	26
3.15	Сильная сторона C++ и её цена	26
3.16	Проверка понимания	26
3.17	Источники для сверки	26
4	Лекция 4. Функции, ссылки и const	26
4.1	Функция как единица проектирования	27
4.2	Определение, параметры и вызов	27
4.3	Declaration, prototype, definition и call	28
4.4	Return value как часть контракта	28
4.5	Pass-by-value: отдельный parameter object	29
4.6	Reference: alias существующего объекта	29
4.7	Const reference: read-only interface	30
4.8	Четыре роли const в этом блоке	30
4.9	Return value или output reference?	31
4.10	Overloading и default arguments	31
4.11	Первый function pointer	32
4.12	Scope, lifetime и повторные вызовы	32
4.13	Модель стоимости без магии	33
4.14	Эксперимент 1: рефакторинг course_inventory	33
4.15	Эксперимент 2: function_contract_lab	33
4.16	Сильные стороны и цена	34
4.17	Проверка понимания	34
4.18	Источники для сверки	34
5	Лекция 5. Классы и инкапсуляция	35
5.1	Зачем программе собственный тип	35
5.2	Struct и class: языковое различие	35
5.3	Member function и текущий объект	36
5.4	Definition вне class и operator ::	36
5.5	Public, private и граница доступа	37
5.6	Encapsulation как технический приём	38
5.7	Invariant: что должно оставаться истинным	38
5.8	Const member functions	38
5.9	Getter и setter: не цель инкапсуляции	39
5.10	Responsibility, cohesion и free functions	39
5.11	Composition: отношение has-a	39

5.12	Memory, lifetime и layout	40
5.13	Cost model без объектной магии	40
5.14	Эксперимент 1: реальный class-based refactoring	40
5.15	Эксперимент 2: encapsulation_break_lab	41
5.16	Сильные стороны и цена	41
5.17	Проверка понимания	41
5.18	Источники для сверки	42
6	Лекция 6. Конструкторы, деструкторы и копирование	42
6.1	Создать объект — значит начать его lifetime	42
6.1.1	Default constructor и слово explicit	43
6.2	Member initializer list: сразу создать parts	43
6.3	Constructor как граница invariant	44
6.4	Destructor и конец lifetime	45
6.5	Какие special members способен дать compiler	45
6.6	Copy construction: появляется новый объект	46
6.6.1	Copy elision без подсчёта магических вызовов	46
6.7	Copy assignment: объект уже существует	46
6.8	Shallow copy ломает уникальное владение	47
6.9	Deep copy и Rule of Three	47
6.10	Rule of Zero: предпочтительный современный итог	48
6.11	Стоимость value semantics	49
6.12	Эксперимент 1: copy_semantics_lab	49
6.13	Эксперимент 2: независимый inventory snapshot	49
6.14	Сильные стороны и цена контроля	50
6.15	Проверка понимания	50
6.16	Источники для сверки	50
7	Лекция 7. Наследование и полиморфизм	51
7.1	Inheritance — отношение между типами	51
7.2	Is-a и has-a до синтаксиса	51
7.3	Base subobject и доступ к members	52
7.4	Public, protected и private inheritance	52
7.5	Static type и dynamic type — центр модели	52
7.6	Non-virtual и virtual dispatch	53
7.7	Object slicing: новое base value	53
7.8	Pure virtual function и abstract class	54
7.9	Virtual destructor и polymorphic deletion	54
7.10	Construction и destruction иерархии	55
7.11	Composition и inheritance на одной задаче	55
7.12	Memory model и стоимость без мифов	56
7.13	Эксперимент 1: polymorphism_failure_lab	56
7.14	Эксперимент 2: project design lab	57
7.15	Сильные стороны и цена механизма	58
7.16	Проверка понимания	58
7.17	Источники для сверки	58
8	Лекция 8. Design, SOLID и паттерны	58
8.1	Design после синтаксиса	58
8.2	SOLID — карта проблем, не догма	59
8.3	Strategy: независимо заменяемая policy	59

8.4	Factory Method: выбор product у creator hierarchy	60
8.5	Observer: publisher не знает всех получателей	61
8.6	Pattern, feature, library и композиция	61
8.7	Эксперимент 1: patterns_lab	61
8.8	Эксперимент 2: course_audit_patterns	62
8.9	SOLID-аудит практики	62
8.10	Overengineering и cost model	62
8.11	Проверка понимания	62
8.12	Источники для сверки	63
9	Лекция 9. Templates: generic code как библиотека	63
9.1	Design pressure: алгоритм тот же, тип другой	63
9.2	Function template по частям	63
9.3	Template argument deduction	64
9.4	Instantiation: что требуется compiler	64
9.5	Requirements: generic не означает «любой тип»	65
9.6	Как читать template diagnostics	65
9.7	Value, const reference и mutable reference	65
9.8	Class template: семейство concrete types	66
9.9	Non-type template parameter	66
9.10	Primary template, specialization и overloading	67
9.11	Почему definitions templates часто находятся в headers	67
9.12	Compile-time и runtime polymorphism	68
9.13	Cost model без лозунга zero-cost	68
9.14	Эксперимент 1: mini_generic_library	69
9.15	Эксперимент 2: generic_course_audit	69
9.16	STL начинается с этой идеи	70
9.17	Проверка понимания	70
9.18	Источники для сверки	70
10	Лекция 10. Ошибки API: status, exceptions и RAII	70
10.1	Ошибка как часть contract API	71
10.2	Baseline: явный status	71
10.3	Exceptions: синтаксис и semantics	72
10.4	Rethrow	72
10.5	Stack unwinding и lifetime	73
10.6	RAII: ресурс принадлежит object lifetime	73
10.7	RAII-owner и transactional thinking	74
10.8	Constructor failure	74
10.9	Destructor и exceptions	74
10.10	Exception-safety guarantees	75
10.11	Boundary design	75
10.12	Exceptions или status: честное сравнение	75
10.13	Standard library использует разные модели	76
10.14	Эксперимент 1: error_model_lab	76
10.15	Эксперимент 2: course_audit_error_boundary	76
10.16	Проверка понимания	76
10.17	Источники для сверки	77
11	Лекция 11. Умные указатели и владение	77
11.1	Сначала словарь: object, access, ownership	77

11.2	Проблема naked new/delete	78
11.3	std::unique_ptr: один владелец	78
11.4	Доступ через unique_ptr	79
11.5	Передача unique ownership	79
11.6	Возврат ownership из factory	79
11.7	Rule of Zero возвращается	80
11.8	Массивы и unique ownership	80
11.9	std::shared_ptr: совместное владение	80
11.10	use_count() не проектирует ownership	81
11.11	Цена shared ownership	81
11.12	Цикл shared_ptr	81
11.13	std::weak_ptr: наблюдение без владения	82
11.14	Правильное направление ownership graph	82
11.15	Raw pointer после smart pointer остаётся нормальным	82
11.16	Как читать parameter types	83
11.17	Return types и ownership	83
11.18	Exception safety и smart pointers	84
11.19	Ошибочные способы создать несколько owners	84
11.20	Нужно ли всегда хранить shared_ptr в members?	84
11.21	Custom deleter: зачем существует	84
11.22	Потокобезопасность: аккуратная граница обещаний	85
11.23	Стоимость: что именно мы покупаем	85
11.24	Эксперимент 1: ownership_factory_lab	85
11.25	Эксперимент 2: shared_weak_graph_lab	86
11.26	Сводка выбора	86
12	Лекция 12. Move-семантика и явные приведения типов	86
12.1	Почему одного сору недостаточно	87
12.2	Expression и object — не одно и то же	87
12.3	Карта value categories в C++17	87
12.4	Lvalue	88
12.5	Prvalue	88
12.6	Xvalue	88
12.7	Rvalue references	89
12.8	Named rvalue reference — lvalue-expression	89
12.9	Что на самом деле делает std::move	89
12.10	Move constructor	89
12.11	Move assignment	90
12.12	Moved-from object	90
12.13	Rule of Five	91
12.14	Rule of Zero по-прежнему лучше	91
12.15	Почему noexcept важен для move	91
12.16	Copy elision и RVO	91
12.17	Move-only types как API contract	92
12.18	Forwarding reference	92
12.19	Reference collapsing	92
12.20	std::forward	93
12.21	Move как cast: мост ко второй половине лекции	93
12.22	Implicit conversion или явный cast?	93
12.23	static_cast	94

12.24	Static downcast и его предел	94
12.25	dynamic_cast	94
12.26	Когда base должен быть polymorphic	95
12.27	Нужен ли cast для обычного upcast?	95
12.28	const_cast	95
12.29	reinterpret_cast	96
12.30	C-style cast скрывает намерение	96
12.31	Четыре casts рядом	97
12.32	Cast не должен маскировать design problem	97
12.33	Эксперимент 1: move_ownership_pipeline	97
12.34	Эксперимент 2: casts_lab	97
12.35	Стоимость	98
12.36	Итоговая схема API	98
13	Лекция 13. Вывод типов, лямбды и вызываемые объекты	98
13.1	auto: compiler выводит тип из initializer	99
13.2	Что auto делает с references и const	99
13.3	auto&, const auto&, auto&&	100
13.4	decltype: type expression без вычисления	100
13.5	decltype(auto)	100
13.6	Trailing return type	100
13.7	Structured bindings	101
13.8	CTAD: class template argument deduction	101
13.9	Callable object: общий термин	102
13.10	Function object	102
13.11	Минимальная lambda	102
13.12	Что создаёт compiler	102
13.13	Capture by value	103
13.14	Capture by reference	103
13.15	Default capture	103
13.16	mutable lambda	104
13.17	Init-capture	104
13.18	Generic lambda	104
13.19	Lambda return type	104
13.20	Lambdas и algorithms	105
13.21	Когда template callable лучше	105
13.22	std::function: единый runtime callable type	105
13.23	Зачем нужна type erasure	106
13.24	Стоимость std::function	106
13.25	Пустой std::function	106
13.26	Copyability target в C++17	107
13.27	std::optional<T>: отсутствие как часть типа	107
13.28	Создание optional	107
13.29	Проверка optional	108
13.30	Optional не означает error details	108
13.31	Optional и pointers	108
13.32	Optional reference отсутствует	108
13.33	Прикладной query API	108
13.34	Structured binding в прикладном API	109
13.35	Callback registry как runtime boundary	109

13.36	Capture lifetime в registry	110
13.37	Mutable state внутри callback	110
13.38	Lambda и this	111
13.39	Когда function pointer всё ещё достаточен	111
13.40	Generic lambda или std::function?	111
13.41	Эксперимент 1: course_query_pipeline	111
13.42	Эксперимент 2: callback_registry_lab	112
13.43	Стоимость и границы abstraction	112
13.44	Итоговый checklist	112
14	Лекция 14. STL: контейнеры, итераторы и алгоритмы	113
14.1	Что такое STL в этом блоке	113
14.2	Почему контейнеров несколько	113
14.3	std::array	113
14.4	std::vector	114
14.5	Size и capacity	114
14.6	Почему reallocation важен	114
14.7	std::deque	114
14.8	std::list	115
14.9	std::forward_list	115
14.10	Ordered associative containers	115
14.11	Unordered associative containers	115
14.12	Когда map, когда unordered_map	115
14.13	Адаптеры контейнеров	116
14.14	Iterator: абстракция позиции	116
14.15	Half-open range [first, last)	116
14.16	Iterator categories	116
14.17	Почему list имеет member sort	116
14.18	Const iterator	117
14.19	Range-based for	117
14.20	std::find и std::find_if	117
14.21	std::count_if	117
14.22	std::copy и output iterators	118
14.23	std::transform	118
14.24	std::accumulate	118
14.25	std::sort	118
14.26	Ошибка comparator	119
14.27	Stable sort	119
14.28	lower_bound	119
14.29	all_of, any_of, none_of	120
14.30	Алгоритм не знает container	120
14.31	Complexity — часть API	120
14.32	Cache locality	120
14.33	Iterator invalidation	121
14.34	Vector invalidation при росте	121
14.35	Как безопасно наблюдать reallocation	121
14.36	List stability	121
14.37	Map/set stability	121
14.38	Erase у vector	122
14.39	Erase-remove idiom в C++17	122

14.40	Почему <code>remove_if</code> не вызывает <code>erase</code> сам	122
14.41	Алгоритм или <code>member function</code>	122
14.42	<code>Index</code> рядом с <code>vector</code>	122
14.43	Почему <code>index</code> по <code>position</code> безопаснее <code>pointer</code> только условно	123
14.44	<code>Pipeline</code> : <code>filter</code> → <code>sort</code> → <code>transform</code> → <code>aggregate</code>	123
14.45	<code>Algorithms</code> и <code>mutation</code>	123
14.46	<code>Iterator lifetime</code> и <code>owner lifetime</code>	124
14.47	<code>End iterator</code> тоже может <code>invalidated</code>	124
14.48	<code>Const-correct algorithms</code>	124
14.49	Не копировать <code>container</code> без причины	124
14.50	Когда конкретный <code>container type</code> в <code>API</code> уместен	125
14.51	<code>Exception safety</code>	125
14.52	<code>Allocators</code> : только первый взгляд	125
14.53	Практический выбор <code>container</code>	125
14.54	Эксперимент 1: <code>course_catalog_pipeline</code>	125
14.55	Эксперимент 2: <code>invalidation_and_erase_lab</code>	126
14.56	Что проверяет практика, а что нет	126
14.57	Итоговый <code>checklist STL</code>	126
15	Лекция 15. Современный C++ после C++17	127
15.1	Стандарт и реальная <code>toolchain</code>	127
15.2	<code>Feature-test macros</code>	127
15.3	<code>constexpr</code> : <code>compile-time</code> возможность, не приказ	127
15.4	Как развивался <code>constexpr</code>	128
15.5	<code>if constexpr</code> : напоминание из C++17	128
15.6	<code>constexpr</code> : <code>immediate function</code> , C++20	128
15.7	<code>constexpr</code> : другой <code>contract</code>	128
15.8	<code>Static initialization order</code>	129
15.9	Почему появились <code>Concepts</code>	129
15.10	<code>Named Concept</code>	129
15.11	<code>Constrained template</code>	129
15.12	<code>Abbreviated function template</code>	130
15.13	<code>Requires-expression</code>	130
15.14	Виды <code>requirements</code>	130
15.15	<code>Concepts</code> не доказывают <code>semantics</code>	131
15.16	<code>Concepts</code> и <code>overload resolution</code>	131
15.17	<code>Concept</code> или <code>static_assert</code>	131
15.18	<code>Concept</code> и <code>runtime polymorphism</code>	131
15.19	Стоимость <code>Concepts</code>	131
15.20	<code>Ranges</code> : следующий слой над <code>STL</code>	131
15.21	<code>Ranges algorithms</code>	132
15.22	<code>Projection</code>	132
15.23	<code>Views</code>	132
15.24	<code>Lazy evaluation</code>	132
15.25	<code>Borrowed ranges</code> и <code>dangling</code>	133
15.26	<code>std::span</code> , C++20	133
15.27	<code>Static extent</code>	133
15.28	<code>Designated initializers</code> , C++20	133
15.29	<code>Defaulted operator<=></code>	133
15.30	<code>std::erase_if</code>	134

15.31	<code>std::string::starts_with</code>	134
15.32	<code>std::source_location</code>	134
15.33	<code>std::jthread</code>	134
15.34	Coroutines: что это	135
15.35	Как compiler видит coroutine	135
15.36	Promise type	135
15.37	<code>co_await</code>	135
15.38	<code>co_yield</code>	135
15.39	Lifetime coroutine	135
15.40	Modules: зачем	135
15.41	Module не равен precompiled header	136
15.42	Почему modules зависят от tooling	136
15.43	Headers продолжают жить	136
15.44	Allocators	136
15.45	<code>std::pmr</code>	136
15.46	Compile-time cost	137
15.47	Runtime cost нельзя угадывать по modern syntax	137
15.48	Эксперимент 1: <code>constexpr_and_concepts_lab</code>	137
15.49	Эксперимент 2: <code>cpp20_feature_tour</code>	137
15.50	Почему coroutines/modules не основные experiments	137
15.51	Что появилось дальше: C++23	137
15.52	<code>std::expected</code> : следующий error contract	138
15.53	Как читать новую feature	138
15.54	Что должно остаться после курса	138
15.55	Дальше: concurrency	138
15.56	Дальше: performance engineering	138
15.57	Дальше: library/API design	138
15.58	Дальше: advanced templates	138
15.59	Дальше: tooling	138
15.60	Что значит знать современный C++	139
15.61	Совместимость проекта	139
15.62	Финальный checklist новой feature	139

О конспекте

Этот конспект посвящён технологиям программирования и языку C++. Основной стандарт курса — C++17; заключительный блок отдельно показывает, что меняют ключевые возможности C++20 и как они продолжают уже изученную модель языка.

Материал организован в 15 последовательных лекционных блоков с практическими экспериментами. Код, сборка, работа компилятора, модель памяти, инструменты разработки и производительность рассматриваются вместе, а не как независимые темы.

1 Лекция 1. Введение в C++ и первая программа

Первая лекция нужна не для того, чтобы выучить много синтаксиса. Наша задача — построить правильную картину: что такое исходный код, кто превращает его в исполняемую программу, зачем языку типы и как уже на первом занятии пользоваться простыми полезными абстракциями стандартной библиотеки.

Что нужно уметь после лекции

Создать файл .cpp, собрать его через g++, запустить программу, объяснить каждую строку простого main, пользоваться базовыми типами, понимать целочисленное деление и простые неявные преобразования, а также использовать std::string, std::vector и std::map на уровне их основного контракта.

1.1 От задачи к машинному коду

Процессор не выполняет строку

```
std::cout << "Hello!\n";
```

непосредственно. В конечном счёте он исполняет машинные инструкции конкретной архитектуры. Язык программирования позволяет описывать вычисление на более высоком уровне: через типы, операции, функции и готовые абстракции.

Определение 1.1. Исходный код — текст программы, написанный программистом. Для C++ исходные файлы часто имеют расширение .cpp.

Определение 1.2. Компилятор — программа, которая анализирует исходный код и переводит его в другую форму. В типичном C++ toolchain результат нескольких стадий становится машинным кодом для выбранной платформы.

Компилятор против интерпретатора

Деление языков на «компилируемые» и «интерпретируемые» работает только как первое приближение. Реальные реализации могут использовать байткод, JIT и смешанные схемы. Для нас важно, что обычная C++-сборка заранее создаёт нативный executable, а компилятор получает информацию о типах до запуска.

1.2 Почему C++

C++ интересен сочетанием высокоуровневых абстракций и достаточно прямого контроля над стоимостью программы. Можно говорить «динамическая последовательность чисел»

через `std::vector`, а позже при необходимости разбираться с размещением памяти, временем жизни и машинным кодом.

Производительность

Нельзя делать вывод «программа быстрая, потому что она написана на C++». Скорость зависит от алгоритма, структуры данных, аллокаций, кэшей, оптимизатора, ввода-вывода и железа. Полезная идея C++ — абстракция не обязана означать большую runtime-стоимость.

1.3 Практический pipeline сборки

Одна команда

```
g++ main.cpp -o program
```

скрывает несколько разных этапов. Для обучения удобно видеть цепочку

`main.cpp` → preprocessing → assembly → object file → linking → executable.

1.3.1 Preprocessing

До основной компиляции обрабатываются директивы препроцессора, например

```
#include <iostream>
```

На первом уровне понимания этого достаточно: после обработки заголовка объявления стандартного потокового ввода-вывода становятся доступны текущей единице трансляции.

Что не надо говорить

Фраза «`#include` подключает библиотеку» смешивает разные стадии. Заголовок участвует в preprocessing, а связывание с уже скомпилированным кодом происходит на стадии linking.

1.3.2 Object file и linker

После анализа C++ компилятор может сгенерировать assembly, а assembler — объектный файл. Объектный файл уже содержит машинный код, но часто ещё имеет неразрешённые ссылки на другие части программы и библиотеки.

Определение 1.3. Линкер связывает объектные файлы и библиотеки, разрешает внешние ссылки и создаёт итоговый executable или библиотеку.

Что гарантирует стандарт

Стандарт C++ определяет translation units и фазы трансляции, но не требует, чтобы реализация буквально сохраняла промежуточные .ii, .s и .o. Мы создаём их специально, чтобы увидеть устройство конкретного toolchain.

Эксперимент 1: разобрать сборку по частям

Файл `examples/01_intro_and_first_program/01_compilation_pipeline/inspect.sh` отдельно запускает preprocessing, генерацию assembly, создание object file и linking. После эксперимента команда `g++` перестает быть магической кнопкой.

1.4 Hello, World! по строкам

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!\n";
5     return 0;
6 }
```

`#include <iostream>` — директива preprocessing. Заголовок `iostream` предоставляет интерфейс стандартных потоков.

`main` — особая функция обычной C++-программы: с неё начинается выполнение пользовательского кода после необходимой работы среды выполнения.

Слово `int` означает, что функция возвращает целое значение. Для `main` это код завершения программы.

`std::cout` — стандартный поток вывода. Префикс `std::` указывает пространство имён стандартной библиотеки.

Почему сразу пишем `std::`:

В старых учебных примерах часто встречается `using namespace std;`. Конструкция допустима, но как основной стиль она скрывает происхождение имён и может создавать конфликты. В этом конспекте стандартные имена по умолчанию пишутся явно: `std::cout`, `std::string`, `std::vector`.

Оператор `<` передаёт значение в поток вывода. Позже мы увидим, что это перегруженный оператор; пока важен его контракт.

Текст в двойных кавычках — строковый литерал. Последовательность `\n` обозначает перевод строки.

`return 0;` завершает функцию и сообщает об успешном завершении. Достижение конца `main` без явного `return` также считается успешным, но на первых примерах оставим возврат видимым.

1.5 Первая команда компиляции

```
g++ -std=c++17 -Wall -Wextra -Wpedantic hello.cpp -o hello
```

Здесь:

g++: C++-драйвер GCC; **-std=c++17:** используем стандарт C++17; **-Wall:** включаем большой набор полезных предупреждений; **-Wextra:** включаем дополнительные предупреждения; **-Wpedantic:** просим предупреждать об отклонениях от стандарта; **-o hello:** задаём имя результата.

Осторожно

Название `-Wall` не означает буквально «все предупреждения GCC». Поэтому полезные группы `warnings` часто комбинируют.

Запуск в Linux:

```
./hello
```

Префикс `./` явно указывает текущий каталог. Обычно shell ищет команды в каталогах из PATH, а текущий каталог туда не включён.

1.6 Из чего состоит строка C++

Рассмотрим

```
int age = 20;
```

`int` — keyword, `age` — identifier, `20` — literal, `=` — operator, `;` — punctuation.

Комментарии нужны для неочевидной причины, а не для пересказа кода.

```
// Плохо: увеличиваем count на один.  
count += 1;
```

Хороший комментарий объясняет, почему увеличение происходит здесь или какой инвариант мы поддерживаем.

1.7 Типы как информация о значениях и операциях

```
1 int age = 20;  
2 double temperature = 36.6;  
3 char grade = 'A';  
4 bool passed = true;
```

Определение 1.4. Тип определяет множество допустимых значений и операций, которые имеют смысл для объекта или выражения.

Тип нужен не только человеку: он сообщает компилятору, как трактовать данные, какие операции разрешены и какие преобразования возможны.

Для первой лекции достаточно знать группы:

`bool`; символьный `char`; целые `short`, `int`, `long`, `long long` и `unsigned`-варианты; `float`, `double`, `long double`.

Не запоминаем `int = 4` байта как закон языка

На современных desktop-системах это часто так, но стандарт не определяет `int` фразой «четырёхбайтовое целое». Конкретный размер относится к реализации и платформе.

`double` не равен множеству вещественных чисел

Floating-point хранит конечное множество представимых значений. Многие десятичные дроби нельзя представить точно, поэтому появляются ошибки округления. Подробно вернёмся к этому позже.

1.8 Инициализация и арифметика


```
1 int count = 5;
2 double price{12.5};
```

Фигурная инициализация полезна тем, что запрещает ряд сужающих преобразований, например попытку записать 3.14 непосредственно в `int`.

Базовые арифметические операции:

```
1 int a = 10;
2 int b = 3;
3
4 int sum = a + b;      // 13
5 int product = a * b; // 30
6 int quotient = a / b; // 3
7 int rest = a % b;    // 1
```

Самый важный ранний пример:

```
1 int x = 7 / 2;      // 3
2 double y = 7.0 / 2.0; // 3.5
```

Почему 7 / 2 даёт 3

Оба операнда целые, поэтому выполняется целочисленное деление. В `7.0 / 2.0` операнды `floating-point`, поэтому операция другая.

Ещё важнее:

```
1 int a = 1;
2 int b = 4;
3
4 double wrong = a / b;      // 0.0
5 double right = 1.0 * a / b; // 0.25
```

В первой строке сначала вычисляется целочисленное `1 / 4`, получается 0, и только затем ноль превращается в `double`. Тип переменной слева не меняет задним числом семантику выражения справа.

1.9 Сравнения и простые преобразования

Операторы `==`, `!=`, `<`, `<=`, `>`, `>=` дают значение типа `bool`.

```
1 int age = 20;
2 bool adult = age >= 18;
```

C++ также выполняет неявные преобразования:

```
1 int whole = 5;
2 double real = whole; // 5 -> 5.0
```

Обратное направление может терять информацию:

```
1 double real = 3.9;
2 int whole = real; // дробная часть теряется
```

Неявное преобразование — конкретная операция с правилами, а не «магическая совместимость типов».

1.10 Ввод и вывод

```
1 #include <iostream>
2
3 int main() {
4     int age{};
5
6     std::cout << "Enter age: ";
7     std::cin >> age;
8
9     std::cout << "Age: " << age << '\n';
10    return 0;
11 }
```

`std::cin > age` просит поток ввода разобрать следующее значение как `int`. Обработку ошибочного ввода пока откладываем: для неё нужны управляющие конструкции.

1.11 Первая стандартная абстракция: `std::string`

```
1 #include <string>
2
3 std::string language{"C++"};
4 std::cout << language << '\n';
5 std::cout << language.size() << '\n';
```

Пока читаем `language.size()` как «спросить у объекта длину». Полный механизм методов появится вместе с классами.

1.12 `std::vector`: последовательность однотипных значений

```
1 #include <vector>
2
3 std::vector<int> numbers{10, 20, 30};
4
5 std::cout << numbers[0] << '\n';
6 std::cout << numbers.size() << '\n';
7
8 numbers.push_back(40);
```

`std::vector<int>` пока читаем буквально как «vector, элементы которого имеют тип `int`». Почему тип записан в угловых скобках, объяснит лекция о шаблонах.

Важно

`vector` даёт абстракцию изменяемой последовательности однотипных элементов и сам управляет необходимыми ресурсами. Поэтому мы можем решать задачи с последовательностями до изучения ручного управления памятью.

Индексация начинается с нуля. `operator[]` не проверяет границы автоматически; к последствиям вернёмся в лекции о памяти.

1.13 `std::map`: ключ и значение

```
1 #include <map>
2 #include <string>
3
4 std::map<std::string, int> frequency;
5
6 frequency["cat"] += 1;
7 frequency["dog"] += 1;
8 frequency["cat"] += 1;
9
10 std::cout << frequency["cat"] << '\n'; // 2
```

`std::map<std::string, int>` читаем как соответствие «ключ типа `string` → значение типа `int`».

Почему новый счётчик начинается с нуля

`map::operator[]` при отсутствии ключа создаёт для него значение. Для `int` значение по умолчанию равно нулю, после чего `+= 1` увеличивает счётчик.

1.14 Не смешиваем язык, библиотеку и инструменты

Уже сейчас существуют три разных уровня:

Язык C++: типы, выражения, функции, синтаксис и правила преобразований; **Стандартная библиотека:** `iostream`, `string`, `vector`, `map`; **Toolchain:** `g++`, `assembler`, `linker`, `CMake`, `shell`.

`vector` не ключевое слово языка. Флаг `-Wall` не входит в стандарт C++. `CMake` — не компилятор.

1.15 `CMake` как первая абстракция над сборкой

Для одного файла удобно написать команду `g++`. Когда исходников и настроек становится больше, выгоднее описать цель сборки.

```
1 cmake_minimum_required(VERSION 3.16)
2
3 project(basic_abstractions LANGUAGES CXX)
4
5 add_executable(basic_abstractions main.cpp)
6
7 target_compile_features(basic_abstractions PRIVATE cxx_std_17)
```

`add_executable` создаёт build target, а `target_compile_features` задаёт требование конкретно для него.

Сборка:

```
cmake -S . -B build
cmake --build build
./build/basic_abstractions
```

`-S .` задаёт каталог исходников, `-B build` — отдельный каталог генерируемых файлов.

Эксперимент 2: простые абстракции

Программа в `examples/01_intro_and_first_program/02_basic_abstractions` использует `string`, `vector` и `map`, но специально обходится без циклов, собственных функций, указателей и классов. Так мы начинаем с небольшого числа уже понятных абстракций.

1.16 Что пока остаётся за кадром

Мы ещё не разобрали `declaration` и `definition`, `scope`, условия и циклы, категории ошибок, память и `lifetime`, устройство `vector`, шаблоны, классы и стоимость операций контейнеров.

Это не пробел, а карта следующих лекций.

1.17 Проверка понимания

После лекции нужно суметь без подсказки объяснить:

1) чем исходный файл отличается от `executable`; 2) зачем нужны `compiler` и `linker`; 3) что делает `#include`; 4) почему пишется `std::cout`; 5) почему `7 / 2` не равно `3.5`; 6) что означает `std::vector<int>`; 7) что означает `std::map<std::string, int>`; 8) почему новый маршчётчик можно увеличить через `++`; 9) чем `CMake` отличается от компилятора.

1.18 Источники для сверки

Для терминов языка и фаз трансляции используем рабочий draft C++17 N4659. Практический GCC pipeline сверяем с *An Introduction to GCC*. Первое педагогическое объяснение compilation pipeline сопоставляем с MIT 6.096 Lecture 1. Target-based сборку сверяем с *Modern CMake*.

2 Лекция 2. Объявления, поток управления и ошибки

Первая программа выполняла почти прямую последовательность команд. Реальная программа должна хранить состояние, принимать решения, повторять действия и останавливаться при нужном условии. Поэтому поток управления — не набор отдельных ключевых слов, а способ записать алгоритм.

Что нужно уметь после лекции

Объявлять и инициализировать простые локальные объекты, строить выражения, писать ветвления и циклы, проверять ввод, объяснять область видимости и различать `compile-time`, `linker`, `runtime`, `logic errors` и `undefined behavior`. Итоговая практическая цель — небольшой корректный консольный инструмент.

2.1 Имя, объект, значение и состояние

```
1 int attempts{0};
2 bool running{true};
3 double threshold{2.5};
4 char command{'s'};
```

Определение 2.1. Identifier — имя, которым исходный код обозначает сущность. В примере это `attempts`, `running`, `threshold` и `command`. Имя не может начинаться с цифры и не должно совпадать с ключевым словом.

Определение 2.2. Объект в терминологии C++ — область хранения с типом и временем жизни. **Значение** — информация, которую выражение вычисляет или объект хранит. Переменной будем называть именованный объект, значение которого можно менять.

Это точнее бытовой фразы «переменная — коробка». Коробка полезна как первая модель, но стандарт не обещает, что локальный объект обязательно лежит в CPU stack: оптимизатор может держать значение в регистре или вообще устранить отдельное хранение, сохранив наблюдаемое поведение программы.

Тип `int` хранит целые из диапазона, зависящего от реализации; стандарт не обещает ему ровно четыре байта. `double` — один из floating-point типов, а не точные вещественные числа. `char` хранит символьные/малые целочисленные значения, `bool` — `false` или `true`. Этих фундаментальных типов достаточно для текущих алгоритмов.

2.2 Declaration, definition, initialization, assignment

Определение 2.3. Declaration сообщает компилятору имя и свойства сущности. **Definition** — объявление, которое вводит саму сущность; для обычного локального объекта **definition** предоставляет хранение.

```
1 int score{10};           // definition и initialization
2 score = 12;              // assignment уже существующему объекту
3 extern int total;        // declaration; definition может быть в другом .cpp
```

Инициализация создаёт начальное значение в момент начала lifetime объекта. Присваивание заменяет значение уже живого объекта. Поэтому слова «инициализируем второй раз» для `score = 12` неверны.

Основные формы, достаточные сейчас:

```
1 int a = 3;               // copy-initialization
2 int b(3);                // direct-initialization
3 int c{3};                // direct-list-initialization
4 int count{};             // value-initialization: для int получаем 0
```

Фигурная форма хорошо видит некоторые narrowing conversions:

```
1 int truncated = 3.9; // допустимо с потерей дробной части; warning зависит от флагов
2 // int rejected{3.9}; // ошибка: list-initialization запрещает narrowing
```

Не нужно превращать выбор скобок в догму. В учебных локальных объектах `{}` явно пока-

зывает инициализацию и защищает от части сужений.

```
1 const int block_count{15};  
2 // block_count = 16; // compile-time error
```

`const` означает, что через это имя нельзя менять объект после инициализации. Это полезный контракт намерения, а не обещание разместить число в особой физической памяти.

2.3 Литералы, выражения и преобразования

Литерал записывает значение прямо в исходнике: 42, 3.5, 'q', true. Выражение сочетает литералы, имена и операторы и вычисляет значение:

```
1 int completed{7};  
2 int total{15};  
3 int remaining = total - completed;  
4 bool finished = remaining == 0;
```

Арифметика использует +, -, *, / и для целых %. При 7 / 2 оба операнда целые, результат равен 3. В 7.0 / 2 целое 2 преобразуется к floating-point, результат 3.5.

```
1 int part{1};  
2 int whole{4};  
3 double wrong = part / whole;      // сначала integer 0, затем double 0.0  
4 double right = 1.0 * part / whole; // 0.25
```

Неявное `int` → `double` удобно, но некоторые большие целые представляются неточно. Обратное преобразование отбрасывает дробную часть; слишком большое floating-point значение для целевого целого типа может привести к undefined behavior. Сужение лучше делать заметным и предварительно проверять диапазон. Подробные casts появятся позже.

Операторы `==`, `!=`, `<`, `<=`, `>`, `>=` возвращают `bool`. Логические `&&`, `||`, `!` означают «и», «или», «не»:

```
1 const bool valid_number = number >= 1 && number <= 15;  
2 const bool should_print = show_all || !pair_complete;
```

2.3.1 Short-circuit — не только оптимизация

Для `left && right` правый операнд вычисляется только если левый оказался true. Для `left || right` — только если левый оказался false. Это гарантия языка и способ безопасно поставить проверку перед опасной операцией:

```
1 const bool ratio_is_large = denominator != 0 && numerator / denominator > 2;
```

Если знаменатель ноль, деление не выполняется. Переставить операнды нельзя: integer division by zero имеет undefined behavior.

2.4 Ветвление if: порядок проверок проектирует алгоритм

```
1 if (score < 0) {
2     std::cout << "invalid\n";
3 } else if (score < 50) {
4     std::cout << "needs work\n";
5 } else {
6     std::cout << "passed\n";
7 }
```

Условия проверяются сверху вниз. Выполняется первое подходящее тело, остальные ветви цепочки пропускаются. Значит, порядок влияет на корректность: сначала часто ставят ошибки и более узкие случаи, затем общий случай.

Вложенный if нужен, когда второе решение имеет смысл только внутри первого:

```
1 if (lecture_exists) {
2     if (examples_exist) {
3         std::cout << "complete pair\n";
4     } else {
5         std::cout << "examples missing\n";
6     }
7 }
```

Фигурные скобки синтаксически необязательны для одного statement, но в нашем стиле ставятся всегда: добавление второй строки тогда не изменит границы ветви.

Присваивание вместо сравнения

if (answer = 0) присваивает ноль и затем преобразует результат к bool; это корректный синтаксис, но почти наверняка logic error. GCC с -Wall -Wextra -Wpedantic реально пишет warning вида suggest parentheses around assignment used as truth value. Нужно сравнение: if (answer == 0).

Warning не обязан останавливать сборку и не доказывает ошибки. Но его нужно прочитать и устранить или осознанно обосновать.

2.5 switch: выбор по дискретному значению

Меню естественно выражается через switch:

```
1 switch (choice) {
2     case 1:
3         std::cout << "show all\n";
4         break;
5     case 2:
6         std::cout << "check one\n";
7         break;
8     case 3:
9     case 4:
10        std::cout << "report\n";
11        break;
12    default:
13        std::cout << "unknown command\n";
14        break;
```

```
15 }
```

Выражение `switch` вычисляется один раз, затем выбирается совпавший `case`; `default` обрабатывает отсутствие совпадения. `break` выходит из ближайшего `switch`. Без него выполнение продолжается в следующий `case` — это `fallthrough`. Иногда оно намеренно, как для 3 и 4 выше. Если между непустыми `case` переход нужен специально, C++17 позволяет отметить его `[[fallthrough]]`;

Типичная реализация

Компилятор может реализовать `switch` цепочкой сравнений, `jump table` или иначе. Стандарт задаёт поведение, но не требует `jump table`. Выбираем `switch` ради ясности дискретного выбора, не по легенде о скорости.

Для диапазонов и сложных независимых условий удобнее `if`; `case labels` должны быть подходящими `constant expressions` и не могут означать диапазон.

2.6 Циклы: повторение с условием остановки

Перед циклом ответьте на четыре вопроса: какое состояние меняется, когда продолжаем, какой шаг приближает остановку и что должно оставаться истинным.

2.6.1 while и validation loop

```
1 int number{};
2 while (!(std::cin >> number) || number < 1 || number > 15) {
3     std::cout << "Enter an integer from 1 to 15: ";
4     std::cin.clear();
5     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
6 }
```

`while` сначала проверяет условие, поэтому тело может не выполниться ни разу. Здесь готовый stream API сообщает об ошибке ввода; `clear` сбрасывает состояние ошибки, `ignore` удаляет неподходящий остаток строки. Это `validation loop`: выход возможен только после допустимого значения.

2.6.2 do while

```
1 int choice{};
2 do {
3     std::cout << "1 - continue, 0 - exit: ";
4     std::cin >> choice;
5 } while (choice != 0);
```

`do while` проверяет условие после тела, поэтому тело выполняется хотя бы один раз. Это бывает естественно для показа меню, но если ввод может сломаться, всё равно нужна явная обработка состояния потока.

2.6.3 for и границы


```
1 int sum{}; // accumulator
2 for (int value = 1; value <= 5; ++value) {
3     sum += value;
4 }
```

Порядок таков: initialization выполняется один раз; condition проверяется перед каждой итерацией; затем тело; iteration expression; новая проверка. Переменная `value` видна в заголовке и теле, но после цикла её scope заканчивается. `sum` — accumulator: хранит результат уже обработанной части. Переменная, считающая события, называется `counter`.

Off-by-one

Для значений 1, 2, 3, 4, 5 нужно `value <= 5`. Условие `value < 5` выполнит четыре итерации. Для индексов последовательности, напротив, обычно естествен полуинтервал `0 <= index < size`. Проверяйте первую, последнюю и пустую итерации отдельно.

Если шаг не меняет состояние к остановке, например забыто `++value`, может возникнуть бесконечный цикл. Запись `while (true)` сама по себе не ошибка: `menu loop` может завершаться явным `break`, но путь выхода должен быть понятен.

2.6.4 break, continue, sentinel и поиск

`break` завершает ближайший цикл. `continue` пропускает остаток текущей итерации и переходит к следующей проверке/шагу. Чрезмерное количество таких переходов усложняет чтение, но для раннего отсева они полезны:

```
1 for (int value = 1; value <= 20; ++value) {
2     if (value % 2 != 0) {
3         continue;
4     }
5     if (value > 10) {
6         std::cout << "first suitable: " << value << '\n';
7         break;
8     }
9 }
```

Sentinel-controlled loop заканчивает последовательность по специальному значению, которое не хранит данные:

```
1 int sum{};
2 int value{};
3 while (std::cin >> value && value != -1) {
4     sum += value;
5 }
```

Здесь `-1` — sentinel. Short-circuit гарантирует: сравнение выполняется только после успешного чтения. Так обрабатывается последовательность неизвестной длины.

2.6.5 Loop invariant — человеческое доказательство

Инвариант — утверждение, истинное перед каждой итерацией. Для суммирования: «`sum` равен сумме уже обработанных значений». Оно истинно до первой итерации (обработанных

значений нет, `sum == 0`); тело сохраняет его, добавляя ровно текущее значение; вместе с условием выхода оно объясняет итог. Это не формальная магия, а компактный способ проверить алгоритм.

2.7 Block scope, shadowing и lifetime

Фигурные скобки создают block scope:

```
1 int count{1};
2 if (count > 0) {
3     int message_count{2};
4     std::cout << message_count << '\n';
5 }
6 // message_count здесь не виден
```

Scope отвечает, где имя можно использовать в тексте программы. Lifetime отвечает, когда объект существует при выполнении. Для простого локального объекта в блоке lifetime начинается после корректной инициализации и заканчивается при выходе из блока; эти понятия связаны, но не одинаковы.

```
1 int status{1};
2 if (ready) {
3     int status{2}; // shadowing: скрывает внешнее имя внутри блока
4     std::cout << status << '\n';
5 }
```

Shadowing допустим, но легко заставляет читать не тот status. В обычном коде лучше выбрать другое имя. Компиляторы могут иметь отдельные warnings для shadowing; базовые три флага не обязаны включать их все.

2.8 Карта ошибок по pipeline

Compile-time error: исходник не удовлетворяет правилам языка, например присваивание объекту `const`; `executable` не создаётся; **Linker error:** translation unit скомпилирован, но linker не нашёл definition объявленной сущности: обычное `undefined reference`. Это продолжает pipeline лекции 1; **Warning:** compiler сообщает о подозрительном, но обычно допустимом коде; сборка может продолжиться; **Runtime failure:** программа запустилась, но не смогла выполнить задачу: например, не найден корень курса или вход неожиданно закончился. Ненулевой `exit status` позволяет сообщить об этом окружению; **Logic error:** программа корректно компилируется и выполняется, но даёт неверный ответ, например сумма 1–5 вычисляется циклом `value < 5`; **Undefined behavior (UB):** стандарт не накладывает требований на поведение, например при `signed integer overflow` или `integer division by zero`. UB не обязано приводить к crash.

Implementation-defined behavior означает, что реализация выбирает один из допустимых вариантов и документирует выбор (например, размер `int`). Unspecified behavior также допускает несколько вариантов, но реализация не обязана документировать выбор. Это не синонимы UB.

Почему «у меня работает» недостаточно

Неправильная программа с UB может напечатать ожидаемое число, упасть или изменить поведение после оптимизации. Проверять нужно правила операции, а не только один запуск на одной машине.

2.9 Diagnostics: compiler и sanitizer решают разные задачи

Для GCC и Clang используем минимум:

```
g++ -std=c++17 -Wall -Wextra -Wpedantic program.cpp -o program
```

Компилятор обязан диагностировать некоторые нарушения, а warnings помогают найти дополнительные подозрительные места. Но корректная сборка не доказывает корректность алгоритма.

UndefinedBehaviorSanitizer добавляет runtime-проверки некоторых классов UB:

```
g++ -std=c++17 -fsanitize=undefined \
    -fno-sanitize-recover=undefined program.cpp -o program
./program
```

Это режим конкретного toolchain, а не часть стандарта C++. Он видит только проверяемые события реально пройденного пути. Отсутствие diagnostics не доказывает отсутствие UB во всей программе.

2.10 Эксперимент 1: course_status

Инструмент `examples/02_declarations_flow_and_errors/01_course_status` read-only проверяет все 15 известных пар `lectures/NN_slug.tex` ↔ `examples/NN_slug/`. Он ищет корень курса от текущего каталога вверх, но ничего не создаёт, не удаляет и не переписывает.

```
cmake -S examples/02_declarations_flow_and_errors/01_course_status \
    -B examples/02_declarations_flow_and_errors/01_course_status/build
cmake --build examples/02_declarations_flow_and_errors/01_course_status/build
./examples/02_declarations_flow_and_errors/01_course_status/build/course_status
```

Программа хранит running и выбранный пункт меню. Внешний while повторяет меню, switch выбирает действие, а for проверяет 15 пар. Результаты двух проверок имеют тип bool; локальные блоки case ограничивают score временных имён. continue возвращает к меню после плохого ввода, break завершает case. Штатный выход имеет status 0, невозможность найти корень — 2.

Язык и библиотека не смешиваем

if, switch, while, for и scope — тема языка этой лекции. `std::filesystem` — готовый library API C++17, которым мы спрашиваем о путях; внутреннее устройство API сейчас не изучаем.

Правильный результат: пункты 1 и 2 сообщают реальное состояние файлов, пункт 3 показывает только неполные пары (для утверждённой структуры их нет), буква вместо номера не ломает menu loop, 0 завершает программу.

2.11 Эксперимент 2: control flow и реальные diagnostics

Второй каталог блока, 02_control_flow_lab, содержит несколько маленьких cases вместо одного искусственного файла.

```
printf 'y\n' | \
examples/02_declarations_flow_and_errors/02_control_flow_lab/run_lab.sh
```

Лаборатория реально проверяет три ветви if, границу цикла и short-circuit. Затем она ожидает: ненулевой exit status у демонстрации logic error; warning для assignment в условии; отказ компиляции при изменении const; runtime-диагностику UBSan для signed overflow. Expected failure — PASS только когда log содержит именно ожидаемую причину. Полные сообщения конкретного установленного компилятора сохраняются в локальном generated/ и не выдаются за универсальный текст стандарта.

2.12 Сильные стороны и цена свободы

C++ прямо выражает control flow, а информация о типах позволяет найти много ошибок до запуска. Состояние и шаги алгоритма видны в коде; стандартная библиотека даёт готовые абстракции вроде streams, vector и filesystem. Один исходник можно собирать в обычном режиме, с усиленными warnings или sanitizer.

Цена этой свободы технически существенна: implicit conversions способны терять данные; logic errors compiler находить не обязан; некоторые неверные программы успешно компилируются; язык допускает UB. Поэтому программист отвечает за условия операций, границы циклов, progress к остановке и проверку диагностик.

2.13 Проверка понимания

После лекции нужно уметь без подсказки:

1) отличить initialization от assignment и declaration от definition; 2) объяснить тип результата сравнения и short-circuit; 3) выбрать if или switch и обосновать порядок проверок; 4) записать validation, sentinel и menu loops; 5) назвать initialization, condition, body и iteration expression цикла; 6) найти off-by-one и сформулировать простой loop invariant; 7) отличить scope от lifetime и заметить shadowing; 8) классифицировать compile, link, runtime, logic errors и UB; 9) объяснить, почему warnings и sanitizer полезны, но не доказывают корректность; 10) написать небольшую консольную программу с корректным exit status.

2.14 Источники для сверки

Правила declarations, statements, conversions и undefined behavior сверены с рабочим draft C++17 N4659. Педагогический порядок условий и циклов сопоставлен с MIT 6.096 Lecture 2, но старый стиль примеров заменён современным явным std:: и фигурными скобками. Практика warnings и стадий сборки сверена с *An Introduction to GCC*; target-based сборка продолжает подход первой лекции и *Modern CMake*.

3 Лекция 3. Указатели, память и массивы

В предыдущей лекции объект хранил одно значение, а цикл повторял действие. Теперь нужно хранить несколько однотипных объектов подряд и понимать их адреса. Сначала постро-

им честную модель raw array и указателя, затем увидим, какую ручную работу снимают `std::array` и `std::vector`.

Главный вопрос

Что означает «несколько объектов подряд в памяти», почему возникает pointer arithmetic и почему для обычного изменяемого набора данных современный C++ чаще выбирает `std::vector`?

3.1 Объект, storage, lifetime и адрес

Уточним три разных понятия: *storage* — память, достаточно большая и правильно выровненная для объекта; *object lifetime* — интервал выполнения, в котором объект существует; *value* — информация, которую он хранит. Storage само по себе ещё не всегда означает живой объект. Для обычного локального объекта lifetime начинается после инициализации и заканчивается при выходе из блока.

```
1 int score{42};
2 std::cout << score << '\n'; // значение
3 std::cout << &score << '\n'; // адрес объекта
```

Определение 3.1. Адрес — значение, обозначающее положение объекта или функции в абстрактной модели C++. Указатель может хранить такое значение.

Выведенный адрес помогает наблюдать взаимное размещение, но его нельзя без оговорки называть физическим адресом RAM. Automatic storage duration также не обязывает реализацию держать объект на CPU stack: это обычная реализация, а не требование стандарта.

`sizeof(expression)` даёт размер представления в байтах типа `std::size_t`. Байт C++ равен размеру `char` и содержит не менее восьми бит; универсального обещания ровно восьми бит нет. Обычно операнд `sizeof` не вычисляется. Результат «int равен 4 байтам» можно наблюдать на машине, но нельзя превращать в закон языка. Alignment означает, что адрес некоторых типов должен быть подходяще выровнен; компилятор учитывает это при размещении, подробности сейчас не нужны.

3.2 Pointer basics: адрес и значение по адресу

```
1 int score{42};
2 int* pointer{&score};
3
4 std::cout << pointer << '\n'; // pointer value: адрес score
5 std::cout << *pointer << '\n'; // pointed-to value: 42
6 *pointer = 50; // меняет score
```

`int* pointer` объявляет объект типа «pointer to int». Звёздочка в declaration участвует в записи типа; `*pointer` в expression выполняет *dereference*. Оператор `&score` берёт адрес. Pointer — не «просто integer»: у него отдельный тип, допустимые операции и правила действительности.

Зачем различать `pointer` и `pointed-to value`

Копирование `pointer` копирует адресное значение, не `score`. Присваивание `*pointer = 50` работает с объектом по адресу.

Минимальный `const` для чтения без изменения:

```
1 int score{42};
2 const int* view{&score};
3 // *view = 50; // compile-time error
```

Сам `view` можно направить на другой `int`. Полная система `const` будет в следующем блоке.

3.2.1 `nullptr` и действительность

`nullptr` явно означает «сейчас не указывает на объект» и имеет специальный тип. Старый литерал `0` не используем как основной стиль.

```
1 int* selected{nullptr};
2 if (selected != nullptr) {
3     std::cout << *selected << '\n';
4 }
```

Dereference требует подходящий живой объект. Проверка на `nullptr` устраняет только null case, но не распознаёт dangling pointer.

Определение 3.2. Dangling pointer сохраняет адресное значение после окончания lifetime объекта, на который раньше указывал. Доступ к прежнему объекту через него недопустим.

Неинициализированный локальный `pointer` фундаментального типа имеет неопределённое значение; его использование может иметь undefined behavior. Безопасная привычка — сразу дать адрес живого объекта или `nullptr`.

3.3 Raw array: фиксированная последовательность

```
1 int temperatures[4]{18, 20, 21, 19};
```

Это built-in, или raw, array из четырёх `int`. Extent 4 входит в array type и известен при компиляции. Все элементы имеют один тип, индексы идут от нуля, объекты расположены непрерывно без промежутков между соседними элементами.

```
1 int values[4]{};           // все элементы равны нулю
2 int inferred[]{2, 4, 6};   // extent 3 выведен из initializer
3 int partial[4]{7, 8};      // 7, 8, 0, 0
4 values[0] = 10;
```

Во время выполнения `values[i]` выбирает элемент, но built-in access не вставляет обязательную проверку границ.

```
1 int values[]{10, 20, 30, 40};
2 const std::size_t count = sizeof(values) / sizeof(values[0]);
```

`sizeof(values)` здесь означает весь array object, а `sizeof(values[0])` — элемент. Деление — лишь локальный учебным приёмом там, где expression всё ещё имеет array type.

Out-of-bounds

Для extent N действительны индексы от 0 до $N - 1$. `values[N]` уже за границей. Чтение или запись имеет UB. UB не обещает crash: возможны правдоподобный вывод, повреждение состояния или иное поведение после оптимизации.

3.4 Array-to-pointer conversion: массив не pointer

```
1 int values[]{10, 20, 30, 40}; // type: int[4]
2 int* first{values};           // conversion; type: int*
```

Array object и pointer object различны. У массива четыре элемента и fixed extent; у `first` одно pointer value. Во многих expressions array неявно преобразуется в pointer на первый элемент: при инициализации `first`, в `values + 1` и при передаче обычному pointer parameter. Функции подробно появятся в блоке 4.

Преобразование происходит не везде:

```
1 std::cout << sizeof(values) << '\n'; // весь int[4]
2 std::cout << sizeof(first) << '\n';  // один int*
3 std::cout << &values << '\n';       // pointer на whole array, другой тип
```

После conversion pointer не несёт extent. Поэтому деление размера pointer на размер элемента не находит длину исходного массива. Pointer можно направить заново, имя массива — нельзя; raw arrays также не присваиваются целиком. Фраза «имя массива — pointer» скрывает эти различия. Точно: array expression во многих контекстах *преобразуется* в pointer.

3.5 Pointer arithmetic существует из-за массивов

```
1 int values[]{10, 20, 30, 40};
2 int* p{values};
3 std::cout << *(p + 2) << '\n';
4 ++p;
5 std::cout << p - values << '\n';
```

Для `int*` операция `p + 1` означает следующий `int`, не следующий машинный байт. `a[i]` имеет семантику `*(a + i)`. Вычитание pointers даёт число элементов типа `std::ptrdiff_t`, но оба должны относиться к одному array object.

```
1 int values[]{10, 20, 30, 40};
2 int* begin{values};
3 int* end{values + 4};
4 for (int* current = begin; current != end; ++current) {
5     std::cout << *current << '\n';
6 }
```

`end` — one-past pointer: его разрешено сформировать и сравнить как границу полуинтервала

[*begin*, *end*), но **end* имеет UB. Нельзя уходить дальше one-past или выполнять арифметику между несвязанными arrays.

Индекс или pointer increment?

`a[i]` и `*(a + i)` выражают ту же адресацию. Нельзя обещать, что одна запись быстрее: нужен benchmark или assembly для конкретных compiler, flags и workload. В обычном коде выбираем ясность.

3.6 Коротко о C-style strings

```
1 char label[]{'C', '+', '+', '\\0'};
2 char same_label[]{"C++"};
```

C-style string — последовательность `char` с null terminator `'\\0'`. Extent второго массива равен 4, хотя видимых символов три. Отсутствие terminator позволяет C API читать за границу. Для совместимости такая форма важна, но основной текстовый тип курса — `std::string`.

3.7 std::array: удобный fixed-size container

```
1 #include <array>
2 std::array<int, 4> values{10, 20, 30, 40};
3 std::cout << values.size() << '\\n';
4 std::cout << values[1] << ' ' << values.at(2) << '\\n';
5 std::cout << values.data() << '\\n';
```

Размер входит в тип и не меняется runtime, storage contiguous. Объект знает `size()`, копируется и присваивается целиком; `at()` проверяет границу и при ошибке сообщает exception. Исключения будут отдельной темой. `data()` даёт pointer на первый элемент, а `begin()/end()` интуитивно обозначают начало и one-past; iterators системно изучим позже.

3.8 std::vector: dynamic size и владение storage

Raw array не растёт. Ручной dynamic array потребовал бы получить новый участок, перенести элементы и освободить старый на всех путях. Naked `new[]` / `delete[]` сейчас не используем: управление динамической памятью будет позже.

```
1 std::vector<int> values{10, 20, 30};
2 values.push_back(40);
3 std::cout << values.size() << ' ' << values.capacity() << '\\n';
4 std::cout << values[0] << ' ' << values.at(3) << '\\n';
```

Vector владеет внутренним storage и управляет lifetime элементов. Элементы contiguous, `data()` возвращает pointer на первый. `empty()` проверяет нулевой size. `operator[]` требует правильной границы от программиста; `at()` проверяет её.

Определение 3.3. `size()` — число живых элементов. `capacity()` — сколько поместится в текущем storage без нового выделения. Capacity не меньше size, но позиции между ними ещё не являются элементами.

Когда `capacity` исчерпана, `push_back` может сделать *reallocation*: получить больший contiguous участок, перенести элементы и отказаться от старого. Точный коэффициент роста и layout не гарантированы.

```
1 std::vector<int> values{10};
2 int* old_data{values.data()};
3 const std::size_t old_capacity{values.capacity()};
4 while (values.capacity() == old_capacity) {
5     values.push_back(20);
6 }
7 // old_data больше не dereference: reallocation его инвалидировал
```

Элементы теперь живут в новом storage, поэтому старый адрес не обозначает прежний живой объект. Reallocation инвалидирует pointers, references и iterators на элементы. Последние два термина здесь лишь анонс будущих тем.

```
1 std::vector<int> values;
2 values.reserve(100);
3 std::cout << values.size();      // 0
4 std::cout << values.capacity(); // не меньше 100
```

`reserve` подготавливает `capacity`, но не создаёт элементы и не меняет `logical size`; это не `resize`. Он уменьшает число `reallocations` при хорошем прогнозе, но сам способен инвалидировать старые pointers.

Один рост с `reallocation` стоит $O(n)$. `Capacity` растёт с запасом, поэтому дорогая операция нужна не при каждом добавлении. `push_back` имеет *amortized* $O(1)$: постоянную среднюю стоимость на длинной серии, не гарантию $O(1)$ каждого вызова.

3.9 Raw array, `std::array` или `std::vector`?

Свойство	raw array	<code>std::array</code>	<code>std::vector</code>
Размер	fixed	fixed, часть типа	dynamic
Знает размер	extent в array type	<code>size()</code>	<code>size()</code>
Checked access	нет	<code>at()</code>	<code>at()</code>
Copy/assignment	не целиком	целиком	целиком
Storage	contiguous	contiguous	contiguous
Pointer access	conversion	<code>data()</code>	<code>data()</code>
Обычная роль	ABI/C API, low-level	fixed container	растущая sequence

Raw arrays не запрещены: они раскрывают язык и нужны некоторым ABI, embedded- и C interfaces. Для обычного владения fixed sequence удобнее `std::array`, runtime-размером — `std::vector`.

3.10 Карта memory errors

Uninitialized pointer: неопределённое значение адреса; **Null dereference:** `*nullptr` не обозначает объект и имеет UB; **Dangling/use-after-lifetime:** адрес сохранился после конца lifetime; **Out-of-bounds:** индекс или `dereference` вышел за элементы; **Invalid arithmetic:** pointer вышел за one-past одного массива или операция смешала несвязанные arrays; **Vector invalidation:** `reallocation` закончила lifetime элементов в старом storage, поэтому

старый pointer нельзя dereference.

Scope отвечает, где имя доступно в тексте; lifetime — когда объект существует runtime. Вынос адреса за scope имени не продлевает lifetime.

3.11 ASan и UBSan: наблюдаем часть UB

AddressSanitizer добавляет runtime-проверки для части out-of-bounds, use-after-free и use-after-scope. UndefinedBehaviorSanitizer проверяет часть других UB. Это instrumentation toolchain, не изменение стандарта.

```
cmake -S . -B build-sanitize -DENABLE_SANITIZER_CASES=ON \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build-sanitize
./build-sanitize/raw_oob
```

PASS требует реальных причин: stack-buffer-overflow, stack-use-after-scope и heap-use-after-free для pointer после vector reallocation. Эти программы намеренно имеют UB и изолированы от safe lab.

Sanitizer не доказывает корректность

Instrumentation видит не все нарушения и только на выполненных путях. Отсутствие сообщения не доказывает отсутствие UB.

3.12 Производительность без лозунгов

Все три варианта дают contiguous элементы. Последовательный проход часто помогает cache locality, но конкретный cache — свойство реализации и железа. Vector хранит metadata; обычно она мала, но стандарт не обещает layout или число слов. Reallocation стоит $O(n)$, reserve может уменьшить allocations, а проход по готовому vector может быть столь же cache-friendly, как по array.

Поэтому «vector медленнее массива» не универсально. Нужно назвать operation, size, access pattern, compiler flags и платформу, затем измерить. Benchmark здесь N/A: лаборатория наблюдает семантику storage, не выдаёт нестабильные наносекунды за закон.

3.13 Эксперимент 1: course_inventory

Read-only утилита в examples/03_pointers_memory_and_arrays/01_course_inventory проходит по фиксированным 15 slugs в std::array, реально проверяет lecture file/examples directory, считывает байты, строки и обычные файлы. Измеренные строки собираются в std::vector<BlockInfo>.

Адреса blocks.data() и rows.data() наблюдают начало двух contiguous областей, но не физический RAM address. Fixed input и dynamic results показывают выбор контейнера; reserve меняет capacity, не size.

```
cd examples/03_pointers_memory_and_arrays/01_course_inventory
cmake -S . -B build
cmake --build build
./build/course_inventory --root ../../..
./build/course_inventory --root ../../.. --block 3
```

```
ctest --test-dir build --output-on-failure
```

PASS: вывод основан на текущих файлах, totals согласованы, один блок и все блоки завершаются с exit code 0. Утилита ничего не изменяет.

3.14 Эксперимент 2: array_vs_vector_lab

Safe executable показывает адреса raw array, оба sizeof, conversion, pointer arithmetic и one-past без dereference; затем fixed std::array; затем size/capacity, смену data() при vector reallocation и стабильность в пределах reserve.

```
cd examples/03_pointers_memory_and_arrays/02_array_vs_vector_lab
cmake -S . -B build
cmake --build build
./build/array_vs_vector_lab
ctest --test-dir build --output-on-failure
```

Safe run не dereference старый pointer. Три отдельные программы намеренно показывают raw out-of-bounds, dangling access и vector invalidation. PASS — успешный safe test и три ожидаемые sanitizer diagnostics с ненулевыми exit codes, а не простое наличие исходников.

3.15 Сильная сторона C++ и её цена

C++ позволяет видеть адреса и стоимость роста, затем выбрать abstraction без отказа от contiguous storage. Vector снимает ручное ownership, но не отменяет lifetime/invalidation. Raw pointers не несут ownership guarantee; свобода означает ответственность за границы. Производительность зависит от access pattern, allocations и locality, а не лозунга «низкоуровневое быстрее».

3.16 Проверка понимания

Нужно уметь объяснить: 1) различие объекта, адреса и значения по адресу; 2) почему pointer не просто integer, а array не pointer; 3) где conversion теряет extent и почему различаются два sizeof; 4) границы pointer arithmetic и роль one-past; 5) отличие scope от lifetime; 6) отличие size/capacity и reserve/resize; 7) причинную связь reallocation и invalidation; 8) выбор raw array, std::array, std::vector; 9) что показывает и чего не доказывает sanitizer.

3.17 Источники для сверки

Правила lifetime, arrays, conversions, additive operators и sizeof сверены с C++17 working draft N4659. Основы сопоставлены с MIT 6.096 Lectures 4, 5 и lifetime-фрагментами Lecture 8, но устаревшие мифы «array is pointer» и «pointer is just integer» исправлены. Контейнеры сверены с *Modern C++ Tutorial*, performance context — с ISO Technical Report on C++ Performance без превращения implementation facts в гарантии.

4 Лекция 4. Функции, ссылки и const

В первых программах вся последовательность действий помещалась в main. В утилите инвентаризации уже появились разбор аргументов, проверка файлов, подсчёты и печать таб-

лицы. Они работают, но спрятаны в одном длинном блоке. Следующее изменение заставляет заново искать границы каждой задачи и повышает риск случайно сломать соседнюю.

Главный вопрос

Как превратить последовательность команд в набор именованных операций с ясными контрактами: что операция получает, что изменяет и что возвращает?

4.1 Функция как единица проектирования

Функция — именованная операция. Она позволяет заменить повторённый алгоритм одним вызовом, дать намерению имя и отделить интерфейс от реализации.

```
1 int square(int value) {  
2     return value * value;  
3 }  
4  
5 const int area = square(6);
```

Читающий `square(6)` сначала видит задачу, а детали умножения может изучить отдельно. Исправление алгоритма делается в одном месте. Так функции поддерживают читаемость, повторное использование, тестирование и декомпозицию.

Но механически выносить каждую строку не цель. Хорошая функция выполняет одну понятную задачу, имеет ясные входы и выходы, разумное имя и уровень абстракции. `doStuff()` скрывает смысл; цепочка функций по одной очевидной строке заставляет читателя прыгать по файлу. Граница полезна, когда часть программы можно назвать и понять как самостоятельную операцию.

Определение 4.1. Интерфейс функции на этом уровне — имя, тип возвращаемого значения и типы параметров. **Реализация** — тело, которое выполняет обещанную операцию.

4.2 Определение, параметры и вызов

```
1 int power(int base, int exponent) {  
2     int result{1};  
3     for (int i = 0; i < exponent; ++i) {  
4         result *= base;  
5     }  
6     return result;  
7 }  
8  
9 const int answer = power(3, 4);
```

Разберём запись: **int**: return type: вызов вычисляет значение этого типа; **power**: function name; **(int base, int exponent)**: parameter list; **фигурные скобки**: body — выполняемые действия; **return result**;: завершает вызов и передаёт значение caller; **power(3, 4)**: call; 3 и 4 — arguments.

Параметр — локальное имя внутри definition, argument — выражение в месте call. Порядок важен: `power(3, 4)` и `power(4, 3)` связывают разные значения с `base` и `exponent`. Компилятор проверяет число аргументов, допустимые conversions и тип результата.

Runtime управление переходит в тело функции, затем возвращается в точку после call. Каждый вызов получает собственные параметры и локальные объекты. Это семантика языка. Отдельный stack frame и call stack — полезная обычная модель реализации, но стандарт не требует конкретного CPU stack или ABI.

Функция без значения результата имеет return type void:

```
1 void print_score(int score) {
2     std::cout << "score=" << score << '\n';
3 }
```

В void-функции допустим return; без expression. В функции с результатом значение должно быть возвращено на каждом достигнутом пути (кроме специальных правил для main). Код после безусловного return недостижим.

4.3 Declaration, prototype, definition и call

Чтобы проверить call, compiler должен раньше увидеть declaration:

```
1 int power(int base, int exponent); // declaration, или prototype
2
3 int main() {
4     return power(2, 5);           // call
5 }
6
7 int power(int base, int exponent) { // definition
8     int result{1};
9     for (int i = 0; i < exponent; ++i) {
10         result *= base;
11     }
12     return result;
13 }
```

Declaration сообщает имя и function type; завершающий semicolon означает, что тела здесь нет. Definition — тоже declaration, но дополнительно задаёт body. Call не объявляет функцию. Имена параметров в prototype можно опустить, но содержательные имена документируют роли.

В блоке 1 translation units компилировались независимо. Обычно общие declarations помещают в header, а definition — в .cpp, чтобы каждая единица трансляции могла проверить вызов. Полные правила headers, ODR и linkage будут развиваться позже; фундамент таков: compiler проверяет видимый declaration, linker затем должен найти подходящее definition.

В строгой терминологии signature не всегда включает всё, что новичок называет «заголовком функции». Здесь безопаснее говорить явно: function name, parameter types и return type; перегрузка различается списком параметров, но не одним return type.

4.4 Return value как часть контракта

Функция-вычисление естественно возвращает результат:

```
1 int absolute(int value) {
2     if (value >= 0) {
```

```
3     return value; // ранний выход упрощает основной путь
4 }
5 return -value;
6 }
```

Ранний return полезен для простого guard condition; слишком много выходов может, наоборот, затруднить обзор. Контракт должен объяснять смысл результата, включая special values. return возвращает вычисленное значение, поэтому обычный return-by-value безопасен после уничтожения local.

Dangling return

Нельзя вернуть pointer или reference на обычный local object и затем получить через него доступ: при выходе lifetime local заканчивается. Адресное значение может сохраниться, но живого объекта по нему уже нет.

```
1 int* bad_address() {
2     int local{42};
3     return &local; // после return pointer dangling
4 }
```

Это мост к модели lifetime блока 3. const не продлевает жизнь такого объекта. Sanitizer способен обнаружить некоторые выполненные случаи, но не обязан диагностировать каждую ошибку lifetime.

4.5 Pass-by-value: отдельный parameter object

```
1 void change_copy(int value) {
2     value = 99;
3 }
4
5 int original{10};
6 change_copy(original);
7 // original всё ещё 10
```

При pass-by-value параметр — отдельный локальный объект, инициализированный из argument. Изменение value не меняет original. Копирование здесь не случайная цена, а полезная семантика: функция получает собственное значение.

Для маленьких scalar types вроде int, bool и pointer это обычно самый ясный выбор. Передача большого std::string или std::vector по value может копировать элементы и выделять storage. Точная стоимость зависит от типа и реализации. Иногда копия нужна функции по смыслу, поэтому правило «reference всегда быстрее» неверно.

4.6 Reference: alias существующего объекта

```
1 int score{10};
2 int& alias{score};
3 alias = 11; // меняется score
```

T& — reference to T; она — alias уже существующего объекта. Reference должна быть привязана при инициализации и обычно не моделирует optional/null состояние. После привязки assignment через неё изменяет объект, а не «перенаправляет» reference.

```

1 void add_point(int& score) {
2     ++score;
3 }
4
5 int result{10};
6 add_point(result); // result становится 11

```

Mutable reference parameter выражает контракт «функция может изменить этот argument». В месте call нет знака &, поэтому значимую мутацию важно подчеркнуть именем: add_point, read_arguments.

Reference связана с моделью адреса из блока 3, и реализация часто передаёт её подобно pointer. Но стандарт задаёт aliasing semantics, а не требует «скрытый pointer» определённого размера или layout. Pointer уместен, когда нужно выразить nullptr или менять выбранный адрес; reference — когда объект обязателен и alias соответствует контракту. Обе конструкции требуют, чтобы referent оставался жив.

4.7 Const reference: read-only interface

```

1 std::size_t total_characters(
2     const std::vector<std::string>& words) {
3     std::size_t result{};
4     for (const std::string& word : words) {
5         result += word.size();
6     }
7     return result;
8 }

```

const T& связывается с существующим объектом без копирования всего T и запрещает функции изменять его *через эту reference*. Поэтому это естественный read-only interface для больших std::string/std::vector. Компилятор отвергнет изменение элемента или push_back через words.

Обещание локально: исходный объект не становится вечным или глобально immutable; другой non-const alias может его менять. Reference не владеет объектом и не решает lifetime. Const reference может связываться с temporary и в некоторых строго определённых контекстах продлевать его lifetime, но это не универсальный механизм ownership; value categories появятся позже.

4.8 Четыре роли const в этом блоке

```

1 const int limit{15};                // local нельзя изменить
2 void inspect(int value);            // call interface по value
3 void inspect_const(const int value); // тот же тип для caller
4 void read(const std::string& text);  // read-only alias
5 const int* view(&limit);            // read-only pointed-to int

```

Top-level const у value parameter ограничивает только local copy в definition и не создаёт другую overload. Поэтому его часто опускают в declaration; в definition можно добавить для защиты реализации. У const T& const относится к referent и существенен для call interface. У const int* нельзя менять pointed-to value через pointer, но сам pointer можно перенаправить.

const выражает намерение и даёт compiler возможность запретить часть мутаций. Он не гарантирует thread safety, владение, lifetime или вечность.

4.9 Return value или output reference?

```

1 int square(int value) {
2     return value * value;
3 }
4
5 void square_out(int value, int& output) {
6     output = value * value;
7 }
```

Первая форма читается как expression, и результат нельзя забыть инициализировать внутри функции. Для одного вычисленного результата обычно предпочтителен return value. Mutable output reference разумна, если главная цель операции действительно изменить существующий объект. Большое число output references скрывает направление данных, требует заранее создать variables и усложняет проверку всех изменений. Более сложные способы возврата нескольких значений пока не нужны.

4.10 Overloading и default arguments

```

1 int twice(int value)      { return value * 2; }
2 double twice(double value) { return value * 2.0; }
3
4 const int a = twice(4);
5 const double b = twice(2.5);
```

Overload set содержит функции с одним именем и разными parameter lists. Compiler выбирает viable candidate и сравнивает требуемые conversions. Если единственного лучшего кандидата нет, call ambiguous и программа ill-formed.

```

1 void show(int);
2 void show(double);
3 // show(1L); // ambiguous: две conversions одного ранга
```

Return type сам по себе не различает overload: compiler должен выбрать функцию до использования результата. Перегрузки полезны для одной операции над близкими типами; одинаковое имя для несвязанных действий скрывает смысл.

Default argument подставляется в месте call:

```

1 void print_line(const std::string& text, int repetitions = 1);
2 print_line("ready"); // как print_line("ready", 1)
```


После первого параметра с default все последующие в этом parameter list тоже должны иметь default. Его обычно указывают в declaration один раз. Это не новая функция и не runtime-решение; defaults не должны скрывать важные inputs.

4.11 Первый function pointer

В блоке 3 pointer мог хранить адрес объекта. Адрес функции тоже можно хранить и затем вызывать:

```
1 int add(int left, int right) { return left + right; }
2 int subtract(int left, int right) { return left - right; }
3
4 int (*operation)(int, int) = add;
5 const int first = operation(9, 4); // 13
6 operation = subtract;
7 const int second = operation(9, 4); // 5
```

Тип читается из имени наружу: operation — pointer на функцию с двумя int parameters, возвращающую int. Скобки вокруг *operation существенны. Это простейший callback-like interface:

```
1 int apply(int left, int right, int (*operation)(int, int)) {
2     return operation(left, right);
3 }
```

Pointer может быть null, поэтому общий interface должен определить это контрактом или проверить. Синтаксис быстро становится трудным; более удобные abstractions будут позже вместе с lambdas и std::function. Сейчас важна идея: поведение тоже можно выбрать через типизированный адрес.

4.12 Scope, lifetime и повторные вызовы

Parameter names и locals доступны только в своих scopes. Их automatic storage duration обычно означает: lifetime начинается при выполнении declaration во время конкретного call и заканчивается при выходе из блока. Следующий call создаёт новые parameter/local objects.

```
1 int local_scope_case() {
2     const int local{17};
3     return local;
4 }
5
6 const int copy = local_scope_case(); // copy жив, local уже нет
7 // std::cout << local;               // имя вне scope: error
```

Scope — область текста, где доступно имя; lifetime — интервал выполнения, когда существует объект. Они связаны, но не тождественны. Reference может покинуть scope исходного имени, но не продлевает lifetime автоматически. Recursion использует те же правила: каждый call получает свои locals. Её алгоритмические применения оставим отдельным задачам.

4.13 Модель стоимости без магии

Обычный function call может требовать передачи arguments, сохранения точки возврата и переходов. Оптимизатор может встроить тело (inlining), удалить call или преобразовать код иначе, если observable behavior сохраняется.

Inlining

Машинное решение зависит от compiler, flags, visibility и target. Keyword inline не обещает вставку машинного кода; его языковая роль связана прежде всего с правилами нескольких definitions. И маленькая функция без keyword может быть встроена, а inline-функция — нет.

Копирование большого vector/string по value может копировать элементы и делать allocation. Const reference часто избегает этой работы, но добавляет aliasing и зависимость от lifetime, а для маленького int обычно лишь усложняет interface. Оптимизация способна убрать некоторые копии; утверждение о времени требует конкретных flags, данных и измерения. Поэтому лаборатория проверяет семантику, а BENCHMARKS для блока N/A.

4.14 Эксперимент 1: рефакторинг course_inventory

Исходная утилита блока 3 фиксирует поведение: принимает `--root/--block`, инспектирует те же 15 пар, печатает ту же таблицу и возвращает те же классы exit codes. Новая версия лежит в `examples/04_functions_references_and_const/01_course_inventory_refactored` и не меняет блок 3.

Декомпозиция проведена по задачам: `read_arguments` формирует входное состояние, `count_lines` и `count_example_files` измеряют один ресурс, `inspect_block` возвращает одну строку, `collect_rows` строит vector, `print_report` агрегирует и печатает. `main` теперь выражает верхнеуровневый сценарий.

`argc`, `indexes` и `selected` передаются по value как маленькие значения. `Path`, `string`, массив имён и vector строк отчёта читаются по const reference. Mutable references `root/selected` оправданы в `parser`: его задача — заполнить существующее состояние. `BlockInfo` и vector естественно возвращаются как values.

```
cd examples/04_functions_references_and_const/01_course_inventory_refactored
cmake -S . -B build
cmake --build build
./build/course_inventory_refactored --root ../../.. --block 3
ctest --test-dir build --output-on-failure
```

Функциональная эквивалентность проверяется одинаковыми arguments и diff после нормализации только process-specific addresses. PASS — одинаковый нормализованный stdout, одинаковые ключевые exit codes и два CTest PASS.

4.15 Эксперимент 2: function_contract_lab

Каталог `02_function_contract_lab` содержит один safe executable. Он проверяет неизменность argument после pass-by-value, изменение через mutable reference, чтение vector через const reference, return value против output reference, local lifetime, overloads и выбор add/subtract через function pointer.

```
cd examples/04_functions_references_and_const/02_function_contract_lab
cmake -S . -B build
cmake --build build
./build/function_contract_lab
ctest --test-dir build --output-on-failure
```

CTest отдельно запускает compiler для трёх ill-formed programs. В `const_mutation.cpp` операция `+=` требует изменить `const std::string`; compiler запрещает это через read-only reference. Другие cases доказывают, что return type не образует overload и что call может быть ambiguous. Test считается PASS именно при ненулевом compiler exit code; реальная diagnostic сохраняется в build.

Unsafe `dangling_return` opt-in собирается с ASan/UBSan. PASS требует ненулевого exit и фактической sanitizer diagnostic на этой toolchain, а не заранее придуманного текста. Safe executable должен закончить строкой `FUNCTION_CONTRACT_LAB=PASS` и code 0.

4.16 Сильные стороны и цена

Строго типизированные функции дают прозрачную декомпозицию и compile-time interface. Value/reference/const-reference позволяют явно выбирать копию, мутацию существующего объекта или read-only alias. Programmer контролирует форму и потенциальную стоимость передачи данных.

Цена контроля: неверный выбор вызывает лишнюю копию либо неожиданную мутацию; dangling references/pointers остаются возможны; function pointer имеет тяжёлый синтаксис; const не решает ownership/lifetime. Compiler проверит типы, но не определит, хорошо ли программа разделена архитектурно.

4.17 Проверка понимания

Нужно уметь: 1) объяснить, когда именованная операция улучшает большой main; 2) различить declaration, definition и call, parameter и argument; 3) описать передачу управления и lifetime locals без требования stack frame; 4) выбрать value, mutable reference или const reference по контракту; 5) объяснить, почему const не означает ownership или вечный объект; 6) предпочесть return value простому output parameter; 7) объяснить overload resolution и два negative cases; 8) прочесть базовый тип function pointer; 9) объяснить dangling return и границы sanitizer; 10) отделить возможный call/copy cost от голословного «быстрее».

4.18 Источники для сверки

Function declarations/definitions, calls, return, references, cv-qualification, overloading и function pointers сверены с C++17 working draft N4659. Порядок объяснения и исходная задача reuse сопоставлены с MIT 6.096 Lecture 3; references — с Lecture 5 и релевантными частями Advanced Topics II. Их утверждение «reference is just a pointer internally» сознательно не принято как гарантия стандарта. Современный выбор value/const reference и C++17 style сопоставлены с *Modern C++ Tutorial*; сборка использует target-based CMake.

5 Лекция 5. Классы и инкапсуляция

В блоке 4 большой `main` утилиты инвентаризации был разделён на функции. Но строка отчёта всё ещё представлена `BlockInfo` с открытыми полями. Имя, пути и результаты проверок постоянно путешествуют вместе. `Caller` может записать `lecture_exists=true`, оставить нулевое число строк или связать имя одного блока с путём другого. Типы полей корректны, состояние в целом — нет.

Главный вопрос

Как создать тип предметной области, который хранит связанные данные рядом, предлагает только осмысленные операции и сокращает число мест, где можно нарушить согласованность?

5.1 Зачем программе собственный тип

В языке уже есть `int`, `bool` и `pointer`. Библиотека даёт строки, контейнеры и пути: `std::string`, `std::vector`, `std::filesystem::path`. Но задача говорит не о случайном числе и строке, а о *блоке курса*. Набор переменных плохо выражает сущность:

```
1 int number{5};
2 std::string slug{"classes_and_encapsulation"};
3 bool lecture_exists{true};
4 bool examples_exist{true};
```

Если параметры почти всегда передаются вместе, это сигнал проверить, не являются ли они одним понятием. Не всякая длинная `parameter list` требует класса, но повторяющаяся группа данных и операций часто обнаруживает потерянный тип. Класс позволяет создать новый пользовательский тип, соединяя состояние с операциями над ним.

Определение 5.1. **Class** задаёт пользовательский тип. **Object**, или **instance**, — конкретный объект этого типа со своим состоянием и `lifetime`. **Class** — описание; `first` и `second` ниже — два разных объекта.

```
1 struct BlockLabel {
2     int number;
3     std::string slug;
4 };
5
6 BlockLabel first{4, "functions_references_and_const"};
7 BlockLabel second{5, "classes_and_encapsulation"};
```

`number` и `slug` — `data members`, то есть объекты, являющиеся частями каждого `BlockLabel`. У двух `instances` свои `members`. Точка выбирает `member` конкретного объекта. Поэтому `first.number` не обращается к состоянию `second`.

5.2 Struct и class: языковое различие

Обе формы могут содержать данные и функции:

```
1 struct PublicLabel {
2     int number; // public по умолчанию
```

```
3     std::string slug;
4     bool valid() const { return number >= 1 && number <= 15; }
5 };
6
7 class HiddenLabel {
8     int number_; // private по умолчанию
9     std::string slug_;
10 };
```

Ключевое различие здесь — default access: members struct по умолчанию public, members class — private. Обе формы способны иметь data members, member functions и access sections. Миф «struct только для данных, class только для поведения» не правило C++.

Практическая эвристика: struct удобен для простой агрегатоподобной записи с намеренно открытым представлением; class — когда нужно скрыть representation или защищать invariant. Это style guideline, не ограничение языка.

5.3 Member function и текущий объект

Функция внутри определения класса называется member function (часто method):

```
1 class Progress {
2 public:
3     bool is_complete() const {
4         return lecture_exists_ && examples_exist_;
5     }
6
7 private:
8     bool lecture_exists_{};
9     bool examples_exist_{};
10 };
```

Call `first.is_complete()` выбирает method через `..`. Он работает с состоянием именно `first`. Интуитивная модель: у non-static member function есть неявный object parameter. Внутри неё `this` — pointer на текущий объект; поэтому `lecture_exists_` означает member текущего объекта и при необходимости записывается `this->lecture_exists_`. Обычно явный `this->` здесь только зашумляет код.

Это не означает копирование объекта при каждом call. Способ передачи скрытого адреса зависит от реализации и ABI; языковая семантика задаёт, к какому объекту относится вызов. Method принимает обычные parameters, возвращает values и может менять текущий non-const object.

Compiler проверяет типы arguments, доступность method и допустимость операции для выбранного объекта. Runtime выполняется обычное тело функции и меняется member конкретного instance.

5.4 Definition вне class и operator ::

Короткие methods можно определить внутри class definition. Длинное тело часто выносят после объявления:

```
1 class Counter {
2 public:
3     void add(int amount);
4     int value() const;
5
6 private:
7     int value_{};
8 };
9
10 void Counter::add(int amount) {
11     value_ += amount;
12 }
13
14 int Counter::value() const {
15     return value_;
16 }
```

`Counter::add` означает: имя `add` находится в scope класса `Counter`. Это тот же `scope resolution operator ::`, что в `std::cout`, но слева `class`, а не `namespace`. Declaration и definition должны согласоваться, включая parameters, return type и завершающий `const`.

Definition внутри класса удобно для маленькой операции и по правилам языка — `inline definition`; это не обещание машинного `inlining`. Вынос тела помогает отделить `interface` от `implementation`.

5.5 Public, private и граница доступа

`public members` образуют интерфейс, доступный `caller`. `private members` доступны самому классу (и `friends`, которые сейчас не нужны).

```
1 class CourseProgress {
2 public:
3     bool is_complete() const { return lecture_ && examples_; }
4     void mark_lecture_found() { lecture_ = true; }
5
6 private:
7     bool lecture_{};
8     bool examples_{};
9 };
10
11 CourseProgress progress;
12 progress.mark_lecture_found(); // допустимо
13 // progress.lecture_ = false; // compile-time error: private
```

Access control — `compile-time mechanism`. Корректная программа не получает `runtime-проверку` на каждое обращение к `private field`. Если `caller` называет недоступный `member`, программа `ill-formed`, и `compiler` выдаёт `diagnostic`.

Private не защищает от атак

`private` организует C++ `interface`, но не шифрует память и не создаёт `security boundary`. UB, ошибочный `raw pointer` или `debugger` не останавливаются словом `private`.

Когда всё `public`, любое место программы может менять каждую деталь и зависит от её формы. Смена представления требует искать `callers`, а корректность поддерживается рас-
пределённой дисциплиной.

5.6 Encapsulation как технический приём

Определение 5.2. Encapsulation здесь означает совместное размещение связанного со-
стояния и операций, скрытие деталей `representation` и ограничение набора допустимых опе-
раций публичным `interface`.

Это не лозунг «положить поля под `private`». Полезный результат: меньше раздельных па-
раметров и комбинаций несогласованных данных; проверки и переходы состояния сосре-
доточены в небольшом числе `methods`; `caller` зависит от смысла операций, а не от способа
хранения; `representation` можно менять без переписывания `caller`, пока сохраняется кон-
тракт `interface`.

Например, реализация может заменить два `bool members` на другое внутреннее представ-
ление, оставив `is_complete()` неизменным. Это возможность `design`, а не автоматическое
обещание `binary compatibility`: изменение `layout` может быть существенно для отдельно со-
бранных `components`.

5.7 Invariant: что должно оставаться истинным

Определение 5.3. Invariant объекта — условие, которое должно оставаться истинным
для корректно используемого объекта на границах его публичных операций.

Для `CourseBlock`: `number` находится в диапазоне 1–15, `slug` не пуст, а `lecture/examples paths`
согласованы с `number` и `slug`. Плохой `public record` разрешает `caller` стереть `slug`, записать
99 и оставить старые `paths`. Хороший `design` не предоставляет таких операций: `paths` вычис-
ляются вместе с `identity`, а `public interface` только читает измерения.

`private` сам по себе `invariant` не гарантирует. `Method` может записать 99 или ошибочно вы-
числить `path`. Гарантию создаёт совокупность `interface` и `implementation`, подтверждённая
тестами. `Compiler` знает типы и `access rules`, но не выводит из предметной области диапазон
1–15.

В практике применяется `factory-like operation` `CourseBlock::inspect`. Она строит согласован-
ное состояние и возвращает его как `value`. Для создания внутри используется минимальный
`default constructor`; полная модель `constructors/destructors` и более сильная инициализация
— тема блока 6.

5.8 Const member functions

```
1 class CourseBlock {  
2 public:  
3     int number() const { return number_; }  
4     bool is_complete() const {  
5         return lecture_exists_ && examples_exist_;  
6     }  
7  
8 private:  
9     int number_{};  
10    bool lecture_exists_{};
```

```

11  bool examples_exist_{};
12  };

```

`const` после `parameter list` относится к `object parameter`, а не к `return type`. Такой `method` обещает `read-only access` к обычному `observable member state`: через него нельзя присвоить значение обычному `non-mutable member`. Тему `mutable` сейчас не вводим.

```

1  void print_one(const CourseBlock& block) {
2      std::cout << block.number() << ' ' << block.is_complete() << '\n';
3  }

```

Связь с блоком 4 прямая: `const CourseBlock&` даёт `read-only alias` без копии, а вызываемые через него `methods` должны быть `const`. Попытка вызвать `modifying non-const method` через `const object` — `compile-time error`. Поэтому `const methods` формируют читающую часть `interface`.

`Const member function` не обещает глобальную неизменность объекта: другой `non-const alias` может его изменить. Она также не решает `lifetime` и `ownership`.

5.9 Getter и setter: не цель инкапсуляции

Getter бывает уместен: `number()` сообщает устойчивое свойство, `lecture_path()` предоставляет `read-only view`. Setter тоже нормален, если независимое присваивание действительно допустимо предметной области.

Но `getX()/setX()` для каждого `private field` часто лишь имитирует `public fields`. Caller всё ещё создаёт бессмысленные переходы. Осмысленный `interface` говорит `is_complete()`, `mark_lecture_found()`, `missing_parts()` или `lecture_path()`; названия выражают задачу и дают реализации место поддержать `invariant`.

Нельзя оценивать `encapsulation` числом `getters`. Важнее, какие неправильные действия `interface` сделал невозможными и какие изменения `representation` перестали затрагивать `callers`.

5.10 Responsibility, cohesion и free functions

У маленького класса должна быть одна внятная `responsibility`. `CourseBlock` представляет и инспектирует один блок. Он не разбирает `command line` и не печатает общую таблицу.

Определение 5.4. На начальном уровне высокая **cohesion** означает, что `members` и `methods` класса связаны одной ролью и его состоянием.

`Method` естественен, когда операция требует состояния/`invariant` конкретного объекта: `block.is_complete()`. Операция, не использующая это состояние, не обязана быть `method`. `print_report(blocks)` и `read_arguments(...)` остаются `free functions`.

Заталкивание всех функций в один `class` создаёт `God object`: много причин для изменения, сильную связанность и трудный `interface`. Плохой `class design` может быть хуже ясных данных и нескольких функций.

5.11 Composition: отношение has-a

```

1  class CourseBlock {

```



```
2     std::string slug_;
3     std::filesystem::path lecture_path_;
4     std::filesystem::path examples_path_;
5 };
```

CourseBlock has-a string и два path objects. Их lifetimes являются частью lifetime содержащего объекта. Уже знакомый `std::vector<CourseBlock>` содержит элементы. Composition естественна, когда модель действительно «имеет» часть; inheritance и virtual здесь не нужны и появятся позже.

5.12 Memory, lifetime и layout

Объект class type имеет lifetime по тем же общим правилам. Локальный CourseBlock живёт до выхода из scope; элемент vector — пока существует элемент и операция контейнера его не уничтожила. Pointer/reference не продлевает lifetime.

Non-static data members являются частями complete object. Их создание и уничтожение связано с содержащим объектом, но подробный порядок и управление ресурсами относятся к блоку 6. `sizeof(CourseBlock)` можно измерить: результат включает представления members и возможный padding. Нельзя по одному измерению объявлять точный layout гарантией всех compiler/ABI.

Access specifiers не обязаны добавлять «флаг private» в object. Стандарт задаёт layout ограниченно; alignment, padding и представления библиотечных members зависят от реализации.

5.13 Cost model без объектной магии

Обычная non-virtual member function не требует отдельной объектной VM. Access control проверяется при трансляции; public/private сами по себе не runtime checks. Method может быть оптимизирован/inlined по тем же общим причинам, что free function.

Нельзя утверждать, что method всегда быстрее или медленнее free function. Значимая стоимость здесь приходит от filesystem calls, allocations string и vector, копий и I/O. Encapsulation может не добавлять отдельной runtime платы, но плохой API способен вызвать лишние копии или coupling. Zero-overhead intent C++ — возможность избежать обязательной доплаты, не обещание нулевой цены любого design. Virtual dispatch и vtable относятся только к блоку 7.

5.14 Эксперимент 1: реальный class-based refactoring

Каталог `examples/05_classes_and_encapsulation/01_course_inventory_classes` развивает `course_inventory_refactored` блока 4. CLI, столбцы, totals и классы exit codes сохранены.

В procedural версии BlockInfo открывал каждую деталь. Теперь private members одного CourseBlock держат number, slug, согласованные paths и измерения. Static public operation `inspect` создаёт их вместе. Read-only methods `lecture_lines()`, `example_file_count()` и `is_complete()` являются const. Setter для каждого field отсутствует: caller не должен переписывать измеренное состояние.

`collect_blocks`, `read_arguments` и `print_report` остались free functions: они управляют коллекцией, CLI и выводом, а не invariant одного блока. Поэтому main читает как сценарий, а не как God object.

```
cd examples/05_classes_and_encapsulation/01_course_inventory_classes
cmake -S . -B build
cmake --build build
./build/course_inventory_classes --root ../../.. --block 5
ctest --test-dir build --output-on-failure
```

Эквивалентность проверяется запуском старого и нового executable с одинаковыми arguments, заменой только process-specific printed addresses и сравнением stdout через diff. PASS: normalized output и exit codes совпали, warnings отсутствуют, три CTest прошли.

5.15 Эксперимент 2: encapsulation_break_lab

В 02_encapsulation_break_lab public struct PublicBlock принимает number 99, пустой slug и несогласованные flags. CourseProgress скрывает state и предлагает for_block, mark_lecture_found, mark_examples_found, is_valid и is_complete. Два instances изменяются независимо.

Free function status_line(const CourseProgress&) форматирует отчёт через read-only interface: форматирование не обязано быть method. Const reference вызывает const methods. Positive executable заканчивает ENCAPSULATION_BREAK_LAB=PASS.

```
cd examples/05_classes_and_encapsulation/02_encapsulation_break_lab
cmake -S . -B build
cmake --build build
./build/encapsulation_break_lab
ctest --test-dir build --output-on-failure
```

CTest вызывает настоящий compiler для двух ill-formed files. private_access.cpp пытается записать private number_; const_mutation.cpp вызывает non-const modifying method через const object. PASS требует ненулевой exit status и непустую diagnostic в build. Текст ошибки не подделывается и различается между GCC/Clang.

Sanitizers здесь N/A: проверяются compile-time запреты, а positive cases не выполняют опасной memory operation. Намеренное UB не проверило бы access control лучше.

5.16 Сильные стороны и цена

C++ позволяет создавать доменные типы, compiler проверяет access control и const object contract, а ordinary encapsulation может не иметь отдельной runtime платы. Programmer выбирает method для операции над состоянием и free function для внешней операции.

Цена: бессмысленные getters/setters не создают хороший interface; compiler не знает business invariant; слишком большой class усиливает coupling; lifetime и ownership не исчезают. Constructors/destructors следующего блока нужны для более сильных invariants и resource-owning types.

5.17 Проверка понимания

После блока нужно уметь: 1) различить class, object/instance, data member и member function; 2) назвать default-access различие struct/class; 3) объяснить operator ., текущий

объект и базовую роль `this`; 4) описать `public interface`, `private representation` и границы защиты; 5) определить `invariant` и объяснить, почему `private` недостаточно; 6) прочитать `definition method` через `::`; 7) объяснить `const` после `parameter list`; 8) отличить `domain operation` от механического `setter`; 9) решить, должна ли операция быть `method` или `free function`; 10) описать `composition`, `lifetime members` и осторожный `cost model`; 11) показать, чем новый `CourseBlock` лучше `procedural record`; 12) интерпретировать обе реальные `negative diagnostics`.

5.18 Источники для сверки

Class definitions, members, access control, `this`, cv-qualification member functions, member access и object lifetime сверены с C++17 working draft N4659. Последовательность `user-defined datatype`, `instance`, `fields`, `methods`, `composition` и `::` сопоставлена с MIT 6.096 Lecture 6; из Lecture 7 использована только `encapsulation`, без `inheritance/polymorphism`. Старые публичные учебные `classes` не приняты как современное правило `design`. Style и `target-based` сборка сверены с локальными *Modern C++ Tutorial* и *Modern CMake*.

6 Лекция 6. Конструкторы, деструкторы и копирование

В блоке 5 `CourseBlock::inspect` сначала создавал почти пустой объект, а затем по одному заполнял `private members`. Доступ извне закрыт, но внутри `class` всё ещё существует промежуток, когда объект не представляет настоящий блок курса. Конструктор позволяет связать начало `lifetime` с установлением корректного начального состояния.

Идея

Запись `T b = a`; обычно создаёт новый объект `b`, а не второе имя для `a`. У нового объекта свой `lifetime` и своё состояние. Чтобы такая `value semantics` была корректной, тип должен правильно определять создание, копирование, присваивание и уничтожение.

6.1 Создать объект — значит начать его lifetime

Class задаёт собственный тип, а объект этого типа хранит значение. Для обычного `local object` определение выделяет подходящее `storage`, выполняет инициализацию, и после успешного завершения инициализации начинается `lifetime`. При выходе из `scope` `lifetime` заканчивается. Стандарт не требует описывать это как размещение в конкретном CPU stack frame.

```
1 class CourseBlock {
2 public:
3     CourseBlock(int number, const std::string& slug)
4         : number_{number}, slug_{slug} {}
5
6 private:
7     int number_;
8     std::string slug_;
9 };
10
11 CourseBlock block{6, "constructors_destructors_and_copy"};
```

CourseBlock внутри class — имя constructor. У него нет return type: даже void писать нельзя. Parameters задают данные, необходимые для начала корректного состояния. Compiler проверяет существование подходящего constructor, типы arguments и доступ. Runtime инициализирует subobjects number_ и slug_, затем выполняет body.

Здесь различим три записи:

```
1 Widget first;           // default initialization
2 Widget second{};        // value initialization
3 Widget third{42};        // direct-list-initialization
```

Для class type первые две формы вызывают доступный default constructor; детальные различия zero/value initialization важны прежде всего для типов без пользовательского constructor и будут уточняться по мере необходимости. Форма с argument напрямую выбирает подходящий constructor. Braces также запрещают ряд narrowing conversions.

6.1.1 Default constructor и слово explicit

Constructor, который можно вызвать без arguments, называется default constructor:

```
1 class Counter {
2 public:
3     Counter() : value_{0} {}
4 private:
5     int value_;
6 };
```

Если class не объявляет constructors, compiler может неявно объявить default constructor. Но как только programmer объявил constructor с параметрами, автоматически получить ещё и parameterless constructor нельзя:

```
1 class BlockNumber {
2 public:
3     explicit BlockNumber(int value) : value_{value} {}
4 private:
5     int value_;
6 };
7
8 BlockNumber good{6};
9 // BlockNumber absent; // error: default constructor отсутствует
```

explicit у one-argument constructor запрещает использовать его как неявное преобразование `int -> BlockNumber`. Это естественно, когда число не должно незаметно становиться domain object. Подробная overload resolution здесь не нужна. Если пустое состояние осмысленно, default constructor можно объявить явно или написать `Type() = default`; . Добавлять его только ради удобства, разрушая invariant, не следует.

6.2 Member initializer list: сразу создать parts

После parameter list constructor стоит двоеточие и список mem-initializers:

```
1 CourseBlock::CourseBlock(int number, const std::string& slug)
2     : number_{number}, slug_{slug} {
3     // Здесь оба member object уже инициализированы.
4 }
```

Это initialization, не assignment. Members создаются до входа в body. Вариант «сначала создать пустую string, потом присвоить» выполнял бы другую последовательность операций и вообще невозможен для reference member или обычного const member:

```
1 class Selection {
2 public:
3     Selection(const int& source, int limit)
4         : source_{source}, limit_{limit} {}
5 private:
6     const int& source_;
7     const int limit_;
8 };
```

Reference должна быть привязана, а const object — получить значение при инициализации. При этом reference не владеет referent: design обязан гарантировать, что тот живёт достаточно долго.

Порядок задаёт class, а не список

Non-static data members инициализируются в порядке их declarations в class. Порядок записей после двоеточия этого не меняет. Поэтому mem-initializer list пишут в том же порядке: иначе visual order обманывает reader, а зависимость одного member от ещё не инициализированного может привести к ошибке.

В `initializer_order_warning.cpp` class объявляет `first_` до `second_`, но визуально перечисляет наоборот. GCC/Clang с `-Wreorder` реально предупреждает, что фактический порядок иной. CTest требует это warning; языковое правило действует независимо от warning.

При уничтожении содержащего объекта members уничтожаются в обратном порядке завершения их construction. Так часть, созданная позже, исчезает раньше.

6.3 Constructor как граница invariant

Плохая модель заставляет caller выполнить протокол:

```
1 CourseBlock block;
2 block.set_number(6);
3 block.set_slug("constructors_destructors_and_copy");
4 block.rebuild_paths();
```

Между строками существуют состояния «номер без slug» и «новая identity со старыми paths». Private fields плюс public setters не устраняют проблему. Лучше получить все обязательные данные в constructor и сразу вычислить dependent members. Тогда доступный public interface не создаёт полуинициализированный CourseBlock.

```
1 CourseBlock(const std::filesystem::path& root, int number,
2             const std::string& full_name)
```

```
3     : number_{number},  
4     full_name_{full_name},  
5     slug_{full_name.substr(3)},  
6     lecture_path_{root / "lectures" / (full_name + ".tex")},  
7     examples_path_{root / "examples" / full_name} {}
```

Compiler не знает domain rule «номер от 1 до 15»; его проверяет implementation. Полная стратегия сообщения об ошибочном construction будет связана с exceptions позже. Сейчас experiment передаёт только известную таблицу корректных identity.

6.4 Destructor и конец lifetime

Destructor — special member function с именем ~Type:

```
1 class Trace {  
2 public:  
3     explicit Trace(const std::string& name) : name_{name} {  
4         std::cout << "construct " << name_ << '\n';  
5     }  
6     ~Trace() {  
7         std::cout << "destroy " << name_ << '\n';  
8     }  
9 private:  
10    std::string name_;  
11 };
```

У destructor нет return type и parameters. Для обычного local object он вызывается при выходе из scope; затем заканчивается lifetime. Перед освобождением storage уничтожаются members. Вложенный scope лаборатории делает порядок наблюдаемым, но не доказывает конкретную реализацию stack.

Destructor нужен типу для завершения владения ресурсом. Его не следует вызывать вручную как обычную cleanup() function: явный вызов не превращает object обратно в безопасное «пустое» значение, а последующее автоматическое уничтожение способно нарушить lifetime rules. Ручное управление lifetime относится к advanced low-level code, не к практике этого блока.

Deterministic destruction

Для local objects с automatic storage duration обычный выход из scope определяет момент destruction языковыми правилами. Это важная основа будущей RAII, хотя полную теорию acquisition, exceptions и smart pointers мы изучим позже.

6.5 Какие special members способен дать compiler

В фокусе четыре операции: default constructor; copy constructor; copy assignment operator; destructor.

При выполнении условий языка compiler неявно объявляет/определяет некоторые из них. «Generated» не значит «ничего не делает»: операция вызывает соответствующие операции members. Взаимодействие правил зависит от того, что programmer объявил. Поэтому опасно писать один manual resource operation, не проверив остальные. Move constructor и move assignment существуют, но их семантика и Rule of Five относятся к будущему блоку.

6.6 Copy construction: появляется новый объект

Обычная форма copy constructor:

```
1 class Card {
2 public:
3     Card(const Card& other)
4         : number_{other.number_}, title_{other.title_} {}
5 private:
6     int number_;
7     std::string title_;
8 };
```

Reference нужна, чтобы читать source, не пытаясь сначала скопировать его для parameter. const разрешает копировать const source и обещает не менять его через other. При

```
1 Card a{6, "copy"};
2 Card b = a;
```

b ещё не существовал: это initialization, выбирающая copy constructor, несмотря на знак =. У b отдельный lifetime и его members копируются по copy semantics собственных типов. Copy std::string и std::vector создаёт независимое value: изменение элементов одной копии не меняет другую.

Pass-by-value создаёт parameter object из argument и поэтому может вызвать copy. Return-by-value тоже может предполагать такую операцию на уровне исходного кода, но фактический отдельный вызов не всегда происходит.

6.6.1 Copy elision без подсчёта магических вызовов

Copy elision позволяет не материализовать промежуточную копию. В C++17 есть гарантированные случаи, где result object конструируется непосредственно в назначении. В других случаях compiler может применить разрешённую elision. Поэтому нельзя обещать: «каждый return class object вызывает один copy constructor».

GCC/Clang flag -fno-elide-constructors полезен для отдельных наблюдений, но это experimental toolchain flag, не другая семантика C++ и не способ отменить гарантированные C++17 cases. Лаборатория проверяет независимость values, а не хрупкое универсальное число сообщений.

6.7 Copy assignment: объект уже существует

```
1 class Card {
2 public:
3     Card& operator=(const Card& other) {
4         number_ = other.number_;
5         title_ = other.title_;
6         return *this;
7     }
8 };
```

Здесь compiler видит перегруженный assignment operator. Слева уже живой object, его lifetime не начинается заново: состояние заменяется состоянием other. Возврат `*this` по reference — conventional контрактом assignment и позволяет цепочки `a = b = c`; отдельный объект для результата не создаётся.

```
1 Card b = a; // copy construction: b создаётся
2 b = a;      // copy assignment: b уже существует
```

Self-assignment `b = b`; допустим. Generated assignment стандартных members с ним справляется. Manual resource owner иногда использует `if (this == &other)`, чтобы не удалить resource до чтения из него. Guard не обязателен для каждого корректного operator: можно написать self-safe algorithm без отдельной ветви.

6.8 Shallow copy ломает уникальное владение

Рассмотрим намеренно старомодный учебный type:

```
1 class ShallowBuffer {
2 public:
3     explicit ShallowBuffer(int value) : data_{new int{value}} {}
4     ~ShallowBuffer() { delete data_; }
5     int& value() { return *data_; }
6 private:
7     int* data_; // owning raw pointer
8 };
9
10 ShallowBuffer first{10};
11 ShallowBuffer second = first; // generated copy копирует pointer value
```

Здесь pointer — не просто доступ, а заявление: object владеет allocation и обязан освободить её. Generated copy выполняет memberwise copy и получает два одинаковых адресных значения. Оба objects считают один resource своим. Изменение через `second` видно через `first`; при destruction первый `delete` освобождает allocation, второй пытается освободить его ещё раз. Это double delete и undefined behavior, а не гарантированный crash.

Определение 6.1. **Shallow copy** resource-owning representation копирует handle/address, не создавая независимый resource. **Deep copy** создаёт новый resource и копирует его содержимое, сохраняя отдельное владение.

Raw pointer также может быть non-owning view. Тогда он не освобождает referent, но copy всё равно не продлевает lifetime. Ownership должен быть понятен из design и документации. Naked new/delete в этом блоке изолированы именно для демонстрации ошибки; domain model их не использует.

6.9 Deep copy и Rule of Three

Исправленный DeepBuffer определяет три операции:

```
1 DeepBuffer(const DeepBuffer& other)
2     : size_{other.size_}, data_{new int[other.size_]} {
3     for (std::size_t i = 0; i < size_; ++i) {
4         data_[i] = other.data_[i];
```



```
5     }
6 }
7
8 DeepBuffer& operator=(const DeepBuffer& other) {
9     if (this == &other) {
10         return *this;
11     }
12     int* replacement = new int[other.size_];
13     for (std::size_t i = 0; i < other.size_; ++i) {
14         replacement[i] = other.data_[i];
15     }
16     delete[] data_;
17     data_ = replacement;
18     size_ = other.size_;
19     return *this;
20 }
21
22 ~DeepBuffer() { delete[] data_; }
```

Copy constructor выделяет отдельный array. Assignment сначала готовит замену, потом освобождает старый resource; self-assignment guard не даёт читать уже удалённый собственный array. Destructor освобождает ровно resource этого object. Полный файл проверяет, что изменения копии независимы.

Определение 6.2 (Rule of Three). Если class вручную управляет resource и ему нужен custom destructor, copy constructor или copy assignment, обычно необходимо внимательно рассмотреть все три операции.

Это design guideline, не синтаксическое требование стандарта. Compiler может принять нарушение, как он принял ShallowBuffer; проблема проявится runtime как неправильное ownership и UB.

6.10 Rule of Zero: предпочтительный современный итог

Ту же задачу хранения последовательности умеет решать member `std::vector<int>`:

```
1 class Buffer {
2 public:
3     Buffer(const std::string& label, const std::vector<int>& values)
4         : label_{label}, values_{values} {}
5 private:
6     std::string label_;
7     std::vector<int> values_;
8 };
```

Class не пишет destructor/copy constructor/copy assignment. String и vector сами владеют ресурсами и корректно копируются; generated memberwise operations дают независимое value. Это и есть практическая мотивация Rule of Zero: строить обычный class из resource-managing members так, чтобы ему не требовался ручной код special members.

Полная RAII и smart pointers будут позже. Уже сейчас вывод применим к CourseBlock: members `int`, `bool`, `std::string`, `std::filesystem::path` и контейнеры имеют подходящие операции. Добавлять custom copy/destructor «потому что лекция о копировании» было бы

лишним кодом и новой площадью для ошибок.

6.11 Стоимость value semantics

Generated не означает бесплатный: copy scalar members обычно дешёв и имеет постоянную стоимость; copy string/vector может выделять storage и копировать characters или elements; стоимость class copy складывается из стоимости members; deep copy buffer длины n требует $O(n)$ времени и отдельной памяти; destructor owner должен освободить resource; vector value semantics имеет цену, но даёт ясное независимое владение.

При copy assignment контейнер иногда переиспользует имеющуюся capacity. Точные allocations зависят от library implementation и текущего состояния. Поэтому не утверждаем конкретное число allocations без измерения. В этом блоке важна семантика; BENCHMARKS — N/A. Позже move semantics позволит в подходящих сценариях передавать resource вместо deep copy, но её синтаксис пока не нужен.

6.12 Эксперимент 1: copy_semantics_lab

Каталог 01_copy_semantics_lab объединяет шесть связанных cases. LessonCard проверяет copy construction и copy assignment обычного value type. Копия vector меняет element и добавляет topic, не меняя source. Trace печатает constructor, copy constructor, assignment и destructor во вложенном scope. После строки — nested scope end — destructors идут в обратном порядке construction.

DeepBuffer проверяет Rule of Three, независимые allocations и self-assignment. ZeroBuffer получает ту же value semantics от string и vector без custom copy/destructor. Safe target должен закончить строкой COPY_SEMANTICS_LAB=PASS.

```
cd examples/06_constructors_destructors_and_copy/01_copy_semantics_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/copy_semantics_lab
ctest --test-dir build --output-on-failure
```

Bad shallow copy отделён opt-in option:

```
cmake -S . -B build-sanitize -DENABLE_UNSAFE_CASE=ON \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build-sanitize
./build-sanitize/bad_shallow_copy
```

На GCC 13 в фактическом запуске process завершился с nonzero status, а ASan сообщил attempting double-free. Это подтверждает выполненный bad path на этой toolchain. Отсутствие sanitizer diagnostic в другом запуске не доказывало бы отсутствия UB. Normal target намеренно не падает; его safe cases чисты и под ASan/UBSan.

6.13 Эксперимент 2: независимый inventory snapshot

02_inventory_snapshot развивает class-based inventory блока 5, не меняя его файлы. Constructor CourseBlock сразу устанавливает number, identity, derived paths и измеренное состояние. Затем реальные данные всех 15 блоков проходят две разные операции:

```
1 const std::vector<CourseBlock> inventory = collect_blocks(root, names);  
2 std::vector<CourseBlock> inventory_copy = inventory; // construction  
3 std::vector<CourseBlock> inventory_assigned;  
4 inventory_assigned = inventory; // assignment
```

Custom special members у CourseBlock отсутствуют. Сравнение observable fields подтверждает, что generated copy сохраняет данные. Чтобы не портить read-only domain interface искусственным setter, для анализа введён отдельный MutableBlockSnapshot. Изменение label и vector notes в копии не затрагивает original; assignment второго snapshot также получает собственный vector. Это reusable pattern: анализировать независимый снимок, не воздействуя на исходную inventory.

```
cd examples/06_constructors_destructors_and_copy/02_inventory_snapshot  
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug  
cmake --build build  
./build/inventory_snapshot --root ../../..  
ctest --test-dir build --output-on-failure
```

PASS: 15 entries скопированы construction и assignment, исходный snapshot остался неизменным, программа печатает INVENTORY_SNAPSHOT=PASS, оба CTest имеют code 0 с учётом ожидаемого rejection неверного root.

6.14 Сильные стороны и цена контроля

C++ позволяет user-defined type вести себя как обычное value. Programmer при необходимости точно задаёт creation, copy и destruction; deterministic destruction создаёт основу управления ресурсами. Composition из string/vector часто позволяет compiler сгенерировать правильные operations.

Цена: generated shallow copy raw owner создаёт серьёзное UB; special member rules требуют понимания; deep copy может быть дорогим; ручные destructor и copy code увеличивают вероятность ошибки. Поэтому ownership должен быть явным, а обычный современный class по возможности стремится к Rule of Zero.

6.15 Проверка понимания

После блока нужно уметь: 1) объяснить начало lifetime и роль constructor в invariant; 2) прочитать member initializer list и назвать настоящий порядок members; 3) объяснить вызов destructor при выходе local object из scope; 4) различить $T\ b = a;$ и $b = a;;$ 5) описать generated memberwise copy для string/vector; 6) обнаружить owning raw pointer и объяснить shallow-copy double delete; 7) прочитать deep-copy Rule-of-Three implementation и self-assignment; 8) назвать Rule of Three guideline, а не требованием синтаксиса; 9) обосновать Rule of Zero для реального CourseBlock; 10) интерпретировать trace, warning и настоящую ASan diagnostic; 11) оценить deep copy как $O(n)$, не называя generated copy бесплатной; 12) аккуратно учесть copy elision при наблюдении constructor calls.

6.16 Источники для сверки

Object lifetime, constructors, mem-initializers, initialization order, destructors, implicitly declared special members, copying и copy elision сверены с C++17 working draft N4659,

прежде всего clauses 6, 12 и 15. Педагогическая последовательность constructor/destructor и ownership сопоставлена с MIT 6.096 Lectures 6 и 8; старые raw-pointer examples сохранены только как изолированная диагностика, не как modern style. Composition и standard-library value semantics сверены с локальным *Modern C++ Tutorial*; target-based sanitizer build — с принятым проектом Modern CMake.

7 Лекция 7. Наследование и полиморфизм

В блоках 5–6 один class описывал один самостоятельный тип: состояние, инвариант, создание, копирование и уничтожение. Теперь несколько видов проверок блока курса должны запускаться одинаково, хотя каждая проверяет своё условие. Нужен общий контракт поведения, но не обязательно одна реализация.

Главный вопрос

Когда один тип — разновидность другого, как C++ выбирает реализацию операции runtime и почему копирование derived object в base object меняет смысл программы?

7.1 Inheritance — отношение между типами

```
1 class Report {
2 public:
3     virtual std::string kind() const { return "Report"; }
4 };
5
6 class LectureReport : public Report {
7 public:
8     std::string kind() const override { return "LectureReport"; }
9 };
```

Report — base class, LectureReport — derived class. Запись : public Report сообщает design-намерение: объект LectureReport допустимо использовать там, где контракт требует Report.

Определение 7.1. Inheritance вводит отношение base-derived между типами. **Runtime polymorphism** позволяет обращаться к объектам разных derived types через общий base interface и выбирать virtual implementation по типу фактического объекта.

Повтор кода сам по себе недостаточен. Если два несвязанных типа имеют функцию print, один не становится разновидностью другого. Общий алгоритм можно вынести в функцию или member-object. Inheritance оправдано, когда важны общий контракт и substitutability.

7.2 Is-a и has-a до синтаксиса

Сначала формулируем связь предметной области: Car is-a Vehicle: автомобиль — вид транспортного средства; public inheritance может соответствовать смыслу; Car has-a Engine: автомобиль имеет двигатель; engine — member/collaborator, то есть нужна composition; CourseAudit has checking components: аудит использует компоненты проверок, но сам не совпадает с конкретной проверкой.

Интуитивная проверка подстановки: если функция обещает работать с любым Vehicle, сохраняет ли Car ожидания base contract? Это подводит к Liskov substitution, но полный SOLID относится к блоку 8.

Неверный пример — `class Engine : public Car`: двигатель — не автомобиль. Наследование ради доступа к готовому коду создало бы ложную связь. Аналогично `CourseAudit : BlockCheck` сомнительно, если аудит координирует несколько checks, а не представляет одну взаимозаменяемую check.

Composition не «побеждает всегда»

Для has-a/uses-a и переиспользования реализации без substitutability обычно предпочтительна composition. Для independently extensible операций, которые должны вызываться через единый interface runtime, public inheritance может точнее выразить задачу.

7.3 Base subobject и доступ к members

Derived object содержит base class subobject. Это корректная модель языка: при создании `LectureReport` сначала инициализируется его `Report` subobject, затем собственные members derived. Не следует говорить «base просто лежит как поле»: у base subobject специальные правила, включая conversions и virtual dispatch.

```

1 class Report {
2 public:
3     const std::string& name() const { return name_; }
4 protected:
5     void mark_checked() { checked_ = true; }
6 private:
7     std::string name_;
8     bool checked_{};
9 };

```

Derived может вызвать `mark_checked`, но не обратиться прямо к `name_` или `checked_`. Private state существует в base subobject, но base сохраняет контроль над invariant. Много protected data ослабляет encapsulation: каждый derived сможет создавать новые сочетания state. Обычно private data и узкие protected/public operations проще контролировать.

7.4 Public, protected и private inheritance

Форма	public member base	protected member base	обычный смысл
public Base	public	protected	is-a, substitutability
protected Base	protected	protected	специальная реализация
private Base	private	private	implementation technique

Private members base всё равно не становятся доступны derived напрямую. Protected/private inheritance существуют, но не выражают публичное is-a для caller. Основной production-style лекции — public Base. Access inheritance не надо путать с access sections внутри class.

7.5 Static type и dynamic type — центр модели

```

1 LectureReport lecture;
2 Report& reference{lecture};
3 Report* pointer{&lecture};

```

Определение 7.2. Static type expression известен compiler из исходного кода. Static types имён reference и pointer — Report& и Report*. **Dynamic type** обозначенного ими most-derived object во время вызова — LectureReport.

Reference и pointer здесь не владеют объектом. lecture имеет automatic storage duration и должен оставаться жив дольше их использования. Pointer может быть nullptr; reference обозначает обязательный object.

```
1 Report report = lecture;
```

У самостоятельного report static и dynamic object type — Report. Это новый base object, не скрытый derived object.

7.6 Non-virtual и virtual dispatch

```
1 class Report {
2 public:
3     std::string non_virtual_kind() const { return "Report"; }
4     virtual std::string virtual_kind() const { return "Report"; }
5 };
6
7 Report& view{lecture};
8 view.non_virtual_kind(); // Report: static dispatch
9 view.virtual_kind();     // LectureReport: dynamic dispatch
```

Non-virtual member выбирается по static type expression. Для virtual call через base pointer/reference final overrider выбирается по dynamic type object runtime. Virtual member остаётся virtual в derived без повторного слова virtual. Modern C++ всё равно пишет:

```
1 std::string virtual_kind() const override;
```

override не «включает virtual заново». Оно фиксирует намерение и требует от compiler найти подходящую virtual function base. Если забыть const, изменить parameter type или ошибиться в имени, compilation завершится ошибкой вместо создания другой функции.

Overriding не overloading

Overriding заменяет virtual behavior base для подходящей signature в derived. Overloading создаёт функции одного имени с разными parameter lists. Derived declaration с похожим именем способно скрыть base overloads; совпадения имени недостаточно. override защищает от этой путаницы.

7.7 Object slicing: новое base value

```
1 void print_by_value(Report report);
2 LectureReport lecture{"lecture 07", 500};
3 print_by_value(lecture);
```

Parameter report — новый Report object. Он инициализируется из base subobject argument; derived-specific members вроде lines_ в него не входят. Dynamic type parameter равен Report,

поэтому даже virtual call выбирает base behavior.

Определение 7.3. Object slicing — потеря derived-specific части при создании base object by value из derived object. Копируется base portion/state, но новый object — не alias исходного derived object.

```
1 void print_by_reference(const Report& report);
2 print_by_reference(lecture); // нет нового Report, slicing отсутствует
3 Report* view{&lecture};      // non-owning view того же object
```

Reference/pointer обозначает существующий LectureReport, поэтому dynamic type сохраняется. Polymorphic objects обычно передают по reference или pointer, если нужна identity/dynamic behavior, а не новая base value. Slicing не обязан вызывать warning: программа может быть формально корректна, но иметь неверную semantics.

7.8 Pure virtual function и abstract class

```
1 class BlockCheck {
2 public:
3     virtual ~BlockCheck() = default;
4     virtual bool passes(const CourseBlock& block) const = 0;
5 };
```

= 0 делает функцию pure virtual. Class с pure virtual function — abstract: создать самостоятельный BlockCheck нельзя. Concrete derived предоставляет final override:

```
1 class LectureExistsCheck final : public BlockCheck {
2 public:
3     bool passes(const CourseBlock& block) const override {
4         return block.lecture_exists();
5     }
6 };
```

Abstract class задаёт interface-like contract, но C++ не требует превращать все classes в «интерфейсы». Он способен иметь state, implemented members и constructors. Здесь stateless interface точнее задачи. final запрещает дальнейшее наследование от concrete check; это локальное решение.

7.9 Virtual destructor и polymorphic deletion

Главная lifetime-опасность возникает не от любого base pointer, а от уничтожения объекта через него:

```
1 class SafeBase {
2 public:
3     virtual ~SafeBase() = default;
4 };
5 class SafeDerived : public SafeBase {
6 public:
7     ~SafeDerived() override = default;
8 };
```



```

9
10 SafeBase* base = new SafeDerived;
11 delete base; // destruction начинается с SafeDerived

```

Virtual destructor выбирается по dynamic type. Сначала выполняется destructor derived class, затем уничтожаются derived members и base subobject через destructor base class. Если base используется polymorphically и контракт допускает delete через base pointer, destructor base должен быть virtual.

Non-virtual destructor

Если static type operand delete отличается от dynamic type object, для корректности base должен иметь virtual destructor с учётом точных правил языка. Удаление UnsafeDerived через UnsafeBase* без него имеет undefined behavior. Это не гарантирует ни «только base destructor», ни crash: после UB стандарт не задаёт результат.

Raw new/delete используются только в изолированном negative experiment. Основной design применяет automatic objects и non-owning views с очевидным lifetime. Smart pointers появятся позже.

7.10 Construction и destruction иерархии

```

1 LectureReport::LectureReport(const std::string& name, int lines)
2   : Report{name}, lines_{lines} {}

```

Base subobject создаётся до members derived, а тело derived constructor выполняется после initialization. Уничтожение идёт обратно: сначала derived destructor и members derived, затем base. Это продолжает lifetime model блока 6.

Virtual calls из constructors/destructors требуют осторожности. Во время base construction более-derived часть ещё не построена, а во время base destruction уже уничтожена. Dispatch ограничен текущим этапом иерархии, а не ведёт себя как вызов через полностью готовый most-derived object. Не следует строить initialization/cleanup на ожидании derived override из base constructor/destructor.

7.11 Composition и inheritance на одной задаче

Проект проверяет CourseBlock: есть ли lecture file, examples directory и содержимое. В inheritance-design каждая проверка is-a BlockCheck:

```

1 LectureExistsCheck lecture;
2 ExamplesExistCheck examples;
3 ContentCheck content;
4 const std::array<const BlockCheck*, 3> checks{
5     &lecture, &examples, &content
6 };
7 for (const BlockCheck* check : checks) {
8     results.push_back(check->run(block));
9 }

```

Checks живут до конца функции, поэтому pointers — безопасные non-owning views. Плюс:

caller одинаково запускает `independently extensible types`, не зная `concrete class`. Цена: `base interface`, `virtual functions` и `class` на каждое правило создают `coupling` и `boilerplate`.

Composition-design использует `collaborators`:

```

1 class CourseAudit {
2 public:
3     std::vector<CheckResult> run(const CourseBlock& block) const;
4 private:
5     PathChecks paths_;
6     ContentChecks content_;
7 };

```

`CourseAudit` has-a `PathChecks` и `ContentChecks`. Для фиксированного локального набора правил форма короче, вызовы `static` и нет ложной `is-a relation`. Если проверки должны независимо выбираться `runtime` через стабильный `contract`, `polymorphic BlockCheck` полезнее. Решение зависит от отношения и точки изменения, а не от лозунга.

7.12 Memory model и стоимость без мифов

Стандарт задаёт `observable behavior virtual calls`, но не требует `vtable` или `vptr layout`. Обычная `ABI implementation` хранит в `polymorphic object` служебный `pointer (vptr)` на таблицу `virtual functions (vtable)`, а `call` может стать `indirect call`. Это `mental model`, не переносимая `layout guarantee`.

`sizeof(Report)` и `sizeof(LectureReport)` можно измерить, но числа зависят от `compiler`, `ABI`, `members`, `alignment` и `platform`. `Metadata` в обычной реализации способна увеличить `object size`. `Inheritance` без `virtual functions` само по себе не обязано добавлять `runtime dispatch overhead`.

Virtual не означает «всегда медленно»

`Virtual call` часто реализуется как `indirect call`, что влияет на `inlining` и `branch prediction`. `Compiler` иногда знает `dynamic type` и `devirtualizes` вызов. Итог зависит от `optimization/context`; `allocations`, `algorithm` и `locality` могут быть важнее одного `dispatch`. Нужны `benchmark/assembly` конкретной сборки.

`Dynamic dispatch` — выбранная стоимость `runtime substitutability`. Когда вариативность не нужна, `ordinary function` или `composition` оставляют выбор `compile time`. Цель — платить за механизм там, где `contract` требует `runtime choice`.

7.13 Эксперимент 1: `polymorphism_failure_lab`

`Safe executable` показывает контраст:

```

static dispatch: Report
dynamic dispatch: LectureReport
pointer dispatch: LectureReport
sliced value: kind=Report, name=07_inheritance_and_polymorphism
reference: kind=LectureReport, name=07_inheritance_and_polymorphism
virtual destructor trace:
destroy SafeDerived
destroy SafeBase
POLYMORPHISM_FAILURE_LAB=PASS

```

Полный output также подтверждает protected operation, derived state и concrete реализацию abstract interface. Три CTest cases требуют compilation failure: нет const у override; создаётся abstract base; derived напрямую обращается к private base state.

```
cd examples/07_inheritance_and_polymorphism/01_polymorphism_failure_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/polymorphism_failure_lab
ctest --test-dir build --output-on-failure
```

Unsafe executable не входит в normal run:

```
cmake -S . -B build-sanitize -DENABLE_UNSAFE_CASE=ON \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build-sanitize
ctest --test-dir build-sanitize --output-on-failure
```

На фактической GCC 13/ASan сборке unsafe process завершился nonzero, а test нашёл relevant AddressSanitizer diagnostic. PASS означает выполненный bad path и ожидаемую причину, не «UB стало безопасным». На другой toolchain diagnostic может отличаться; языковое правило остаётся.

7.14 Эксперимент 2: project design lab

02_inheritance_vs_composition_project_lab читает реальные lecture/examples блока 07. Оба design дают одинаковый результат:

```
inheritance:
  lecture exists=PASS
  examples directory exists=PASS
  lecture and examples non-empty=PASS
composition:
  lecture exists=PASS
  examples directory exists=PASS
  lecture and examples non-empty=PASS
same observable results=true
PROJECT_DESIGN_LAB=PASS
```

```
cd examples/07_inheritance_and_polymorphism/02_inheritance_vs_composition_project_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/project_lab --root ../../..
ctest --test-dir build --output-on-failure
```

PASS требует exit code 0, трёх успешных checks и эквивалентного observable result. Для fixed audit composition проще. Inheritance-version показывает полезную точку: новый concrete BlockCheck можно передать тому же caller. Это reusable audit idea, но не преждевременный framework.

7.15 Сильные стороны и цена механизма

C++ включает runtime polymorphism в type system: compiler проверяет conversions и override, programmer выбирает static/dynamic dispatch, а public inheritance выражает настоящую substitutability. Composition остаётся отдельным инструментом, и virtual abstraction не навязывается каждому class.

Цена ошибок: ложная hierarchy усиливает coupling; slicing может тихо изменить semantics; non-virtual destructor опасен при polymorphic deletion; layout становится менее очевидным; indirect dispatch может влиять на optimization. Inheritance ради одного code reuse часто ухудшает design, потому что обещает отношение между типами, которого в задаче нет.

7.16 Проверка понимания

После блока нужно уметь: 1) назвать base/derived и прочитать `class D : public B`; 2) отличить is-a от has-a и предложить composition для второго; 3) объяснить public inheritance и осторожное применение protected; 4) определить static/dynamic type до объяснения virtual call; 5) предсказать non-virtual и virtual output через base view; 6) объяснить override как compiler-checked intent; 7) причинно описать slicing и отличие base value от alias; 8) объяснить pure virtual function и abstract class; 9) обосновать virtual destructor именно polymorphic deletion; 10) назвать порядок base/derived construction/destruction; 11) сравнить inheritance/composition без абсолютного правила; 12) отделить vptr/vtable model от standard guarantee; 13) оценивать virtual cost по контексту, а не словом «всегда».

7.17 Источники для сверки

Base/derived classes, virtual functions, final overrider, abstract classes, initialization/destruction и delete-expression сверены с C++17 working draft N4659, прежде всего clauses 10, 12 и 15. Педагогическая последовательность сопоставлена с MIT 6.096 Lecture 7, а classes/lifetime — с Lectures 6 и 8. Современное применение override сверено с локальным *Modern C++ Tutorial*. Vtable/vptr названы обычной ABI strategy, не требованием стандарта; performance context согласован с локальным C++ Performance TR.

8 Лекция 8. Design, SOLID и паттерны

В блоке 7 мы получили runtime polymorphism. Но корректный синтаксис ещё не даёт удобный design: программа может компилироваться и заставлять менять один центральный switch при каждом новом требовании.

Главная причинная цепочка

Простой работающий design → новое требование → неудобные связи → design pressure → pattern и его цена. Pattern — повторяемая идея взаимодействия объектов, не готовый кусок синтаксиса C++.

8.1 Design после синтаксиса

Responsibility — связанная обязанность и осмысленная причина менять модуль. **Cohesion** высока, когда его данные и операции служат этой обязанности. **Coupling** показывает, сколько одна часть знает о другой. **Dependency** означает, что одной части нужна другая.

Interface задаёт контракт, **implementation detail** можно менять за ним, а **extension point** принимает ожидаемый новый вариант.

Например, checkout с формулами всех скидок зависит от каждой policy. Checkout, вызывающий `PricingStrategy::price`, зависит от узкого interface; формула стала detail. Coupling не исчез, но dependency направлен к более стабильной abstraction. Главный тест design — изменение требований.

8.2 SOLID — карта проблем, не догма

SOLID — guidelines design, не требования стандарта C++ и не законы языка. Compiler не способен полностью проверить их смысл.

SRP. Single Responsibility Principle: class/module имеет одну связную responsibility, один осмысленный reason to change. Это не «один method на class». Несколько измерений CourseBlock связны; чтение filesystem, выбор policy и три формата вывода имеют разные причины изменения. Разделение повышает cohesion, но лишние классы для одноразовой функции только мешают.

OCP. Open/Closed Principle: стабильный central code желательно расширять вариантами без постоянного редактирования. Абсолютной закрытости нет: интерфейс и требования меняются. Extension point создают лишь на ожидаемой оси вариативности; иначе это speculative complexity.

LSP. Liskov Substitution Principle связывает design с блоком 7: derived сохраняет base contract. Если pricing interface обещает неотрицательную цену, override с -1 ломает ожидание client, хотя signature корректна. Compiler проверяет override, но не поведенческий смысл. Client не должен делать исключения по concrete subtype.

ISP. Interface Segregation Principle: client не зависит от операций большого interface, которые ему не нужны. Result observer требует on_result, но не save_html и set_color. Маленькие целевые interfaces полезны при разных clients; дробить interface без причины тоже не нужно.

DIP. Dependency Inversion Principle: high-level policy не привязана жёстко к low-level implementation; обе зависят от abstraction. Простейшая dependency injection — constructor parameter:

```
1 explicit Checkout(const PricingStrategy& strategy) : strategy_{strategy} {}
```

DI container/framework не требуется. Reference non-owning: strategy должна жить дольше checkout.

8.3 Strategy: независимо заменяемая policy

Baseline с двумя стабильными вариантами честен:

```
1 int direct_price(int base, PricePolicy policy) {  
2     switch (policy) {  
3     case PricePolicy::regular: return base;
```

```

4     case PricePolicy::student: return base * 80 / 100;
5     }
6     return base;
7 }

```

Pressure появляется, когда algorithms добавляются независимо, выбираются runtime, а central switch приходится править для каждого варианта.

```

1 class PricingStrategy {
2 public:
3     virtual ~PricingStrategy() = default;
4     virtual int price(int base) const = 0;
5 };
6 class StudentPricing final : public PricingStrategy {
7 public:
8     int price(int base) const override { return base * 80 / 100; }
9 };

```

PricingStrategy — Strategy, конкретные тарифы — ConcreteStrategy, Checkout — Context. Caller выбирает object, а стабильный Context знает только abstraction. Compiler проверяет hierarchy, pure virtual contract и override; runtime virtual dispatch выбирает реализацию по dynamic type. Templates будут в блоке 9 и здесь не являются основным механизмом.

В лаборатории automatic strategy живёт дольше Context; const& не владеет ею. Цена: extra object/type, indirection и обычно один virtual call. Allocation не обязательна. Для короткого неизменного switch Strategy хуже. В hot path стоимость измеряют; вне него design value часто важнее одного call.

8.4 Factory Method: выбор product y creator hierarchy

Constructor, helper make_x и simple factory с одним switch могут быть хороши, но это не GoF Factory Method. Здесь pressure иной: общий алгоритм Creator работает через Product interface, а ConcreteCreator решает, какой ConcreteProduct предоставить.

```

1 class ReportCreator {
2 public:
3     std::string publish(const std::string& title) const {
4         const Product& product = create_product();
5         return title + " -> " + product.format(title);
6     }
7 protected:
8     virtual const Product& create_product() const = 0;
9 };

```

Product — interface; TextProduct/HtmlProduct — ConcreteProduct; ReportCreator — Creator с factory method create_product; два derived creators — ConcreteCreator. Именно creator polymorphism, а не слово create, делает пример Factory Method.

Каждый ConcreteCreator владеет product как member и возвращает const Product&. Reference non-owning и действительна лишь пока жив creator. Heap allocation не нужна. Production factory часто возвращает owning smart pointer, но ownership tools относятся к блоку 11. Цена — две hierarchies, virtual calls, types и lifetime contract. Для одного product

constructor/helper проще.

8.5 Observer: publisher не знает всех получателей

Baseline publisher напрямую вызывает A, B и C. Новый receiver требует менять publisher. Pressure: несколько независимо добавляемых consumers реагируют на одно event.

```

1 class Observer {
2 public:
3     virtual ~Observer() = default;
4     virtual void on_completed(const std::string& job, int result) = 0;
5 };
6 void subscribe(Observer& observer) { observers_.push_back(&observer); }
7 std::vector<Observer*> observers_; // non-owning

```

Subject/Publisher хранит registry abstractions; ConcreteObserver реализует effect; event — завершение job; notify вызывает каждого receiver. Automatic observers обязаны жить дольше регистрации и notify. Subject ничего не удаляет. Уничтоженный observer оставит dangling pointer и возможное UB. В долгоживущей системе нужен unsubscribe/registration design. Smart pointers не универсальное лекарство: shared ownership тоже имеет цену и cycles.

Notify стоит $O(n)$ virtual calls и storage для n observers; появляются вопросы порядка callbacks и reentrancy. Для одного постоянного receiver прямой call честнее.

8.6 Pattern, feature, library и композиция

virtual, inheritance, pointer и reference — language features. Strategy, Factory Method и Observer — patterns, реализуемые этими features. Позже возможны templates, lambdas, std::function и smart pointers; реализация не меняет design pressure. Patterns можно сочетать, но не обязаны: Strategy выбирает policy, Observer сообщает результат, Factory Method иногда делегирует product creator. «Паттерн-комбайн» без потребности не нужен.

8.7 Эксперимент 1: patterns_lab

01_patterns_lab проверяет direct switch и две strategies с общим workflow и разными totals; общий Creator::publish через два creators и products; два events для console/statistics observers. Контрольный case оставляет стабильный двухвариантный выбор обычным conditional.

```

cd examples/08_design_solid_and_patterns/01_patterns_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/patterns_lab
ctest --test-dir build --output-on-failure

```

PASS: отдельные markers STRATEGY_CASE, FACTORY_METHOD_CASE, OBSERVER_CASE, SIMPLE_CASE и итоговый PATTERNS_LAB=PASS; exit code 0.

8.8 Эксперимент 2: course_audit_patterns

Read-only utility исследует реальные 15 pairs. AuditEngine зависит от AuditStrategy. Structural проверяет наличие pair, Strict — ещё непустую лекцию и examples. Третья, lecture-only policy, локально проверяет расширяемость: она добавлена без правки engine workflow.

Три observers получают каждый result: console summary, issue detail и statistics. Engine не знает concrete effects. Strategies и observers созданы раньше engine, поэтому его non-owning references/pointers действительны.

```
cd examples/08_design_solid_and_patterns/02_course_audit_patterns
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/course_audit_patterns --root ../../.. --policy structural
./build/course_audit_patterns --root ../../.. --policy strict
ctest --test-dir build --output-on-failure
```

Structural ожидает 15 pairs. Strict честно находит готовые 01–08 и семь будущих incomplete blocks. PASS означает 15 доставленных events и ожидаемый итог policy, а не готовность блоков 09–15.

Factory Method здесь сознательно не применён: CLI choice короток, objects локальны, общего polymorphic Creator algorithm нет. Добавление creator/product hierarchies было бы overengineering.

8.9 SOLID-аудит практики

SRP: inspection, evaluation и reporting разделены; main остаётся уместным composition root; OCP: strategy и observer добавляются к неизменному engine; CLI mapping для нового пользовательского имени всё же меняется; LSP: каждая policy выдаёт осмысленный result для любого block, каждый observer принимает любое событие без проверки concrete publisher; ISP: два interfaces узки под настоящих clients; дальше дробить незачем; DIP: high-level engine зависит от abstractions, injection выполнена reference parameter и subscribe, без framework.

8.10 Overengineering и cost model

Pattern добавляет objects, types, pointer/reference indirection, virtual dispatch, code size и compile complexity. Factory Method не требует heap; Observer notify имеет $O(n)$; Strategy runtime обычно даёт одну дополнительную indirection. Обычная реализация virtual использует vtable/vptr, но стандарт не требует такого layout. Performance зависит от implementation и hot path: неверны и «паттерны медленные», и «нулевая стоимость» без контекста.

Если два варианта не меняются, switch может быть яснее. Abstraction без variation point — unnecessary complexity. Хороший design минимален для текущих и ожидаемых требований; pattern не самоцель.

8.11 Проверка понимания

После блока нужно уметь объяснить все пять SOLID как guidelines; начать с baseline и называть pressure; отличить Factory Method от helper; назвать роли трёх patterns; доказать non-owning lifetime; отделить pattern от feature; оценить trade-offs и отказаться от pattern, когда direct code проще.

8.12 Источники для сверки

Virtual calls, abstract classes, references и lifetime сверены с C++17 draft N4659 (clauses 6, 10, 12, 15). OOP background сопоставлен с MIT 6.096 Lecture 7 и лекциями 5–7. Strategy, Factory Method и Observer употреблены в каноническом GoF смысле; cost context — с локальным C++ Performance TR, target-based сборка — с Modern CMake.

9 Лекция 9. Templates: generic code как библиотека

В блоке 1 запись `std::vector<int>` читалась как «vector элементов int», а в блоке 3 появилась `std::array<int, 4>`. Теперь разберём механизм языка, благодаря которому библиотеке не нужны вручную написанные `vector_int` и `vector_string`. Главный вопрос блока: как один исходный алгоритм или тип сделать reusable для разных типов, не стирая их различия во время выполнения?

Причинная цепочка

Дублирование по типам → function template → deduction → instantiation → requirements к типу → class template и value parameter → маленькая generic library → мост к STL.

9.1 Design pressure: алгоритм тот же, тип другой

До templates максимум двух типов пришлось бы записать дважды:

```
1 int max_int(int left, int right) {
2     return left < right ? right : left;
3 }
4
5 double max_double(double left, double right) {
6     return left < right ? right : left;
7 }
```

Имена и типы различаются, но решение одно: сравнить два значения и вернуть большее. Исправление ошибки или изменение правила придётся повторять. Overload делает единое имя вызова, но тела всё ещё дублируются. Common base и virtual function из блока 7 здесь искусственны: `int` и `double` не должны становиться объектами одной hierarchy.

Generic programming описывает алгоритм через необходимые операции, а не через один заранее выбранный concrete type. Языковой механизм templates позволяет compiler получить concrete entities для подходящих arguments.

9.2 Function template по частям

```
1 template <typename T>
2 const T& maximum(const T& left, const T& right) {
3     return left < right ? right : left;
4 }
```

`template <typename T>` вводит **template parameter** `T`: здесь это parameter-типа. В этом месте `class T` синтаксически тоже допустимо; в конспекте последовательно используем `typename`, потому что `parameter` может быть не `class`, а, например, `int`.

Всё определение — **function template declaration**. Это не одна обычная функция, способная runtime принимать значения «без типа». Template задаёт семейство возможных functions. Записи `maximum<int>` и `maximum<double>` называют разные **specializations** этого template с разными function types.

Template, specialization и instantiation

Template — исходное параметризованное объявление. Specialization — entity, полученная для конкретных template arguments или заданная отдельно. Instantiation — процесс получения specialization из template. В простом вызове compiler не обязан буквально копировать текст; это лишь удобная первая модель, а не требование к реализации или object code.

9.3 Template argument deduction

Обычно argument типа выводится из function arguments:

```
1 int a = maximum(4, 9);           // T deduced as int
2 double b = maximum(2.0, 3.5);    // T deduced as double
3 std::string c = maximum(std::string{"a"}, std::string{"b"});
```

Это **template argument deduction**: compiler сопоставляет parameter types const T& с types expressions в call и выводит T. Вызов не делает type runtime-данными; решение принимается при translation.

Template argument можно указать явно:

```
1 double value = maximum<double>(2, 3.5);
```

Здесь первый argument `int` преобразуется к явно выбранному `double`. Явная запись полезна, когда deduction невозможен или нужен осознанный target type, но очевидный argument обычно не повторяют.

Если оба parameters содержат один T, compiler должен получить согласованный результат. Вызов `maximum(2, 3.5)` пытается вывести из первого argument `int`, а из второго `double`; простое правило не выбирает «лучший общий тип», и call ill-formed. Можно привести arguments к одному смысловому типу или явно выбрать `maximum<double>`.

Это только базовая deduction. Сложные правила, auto, forwarding и class template argument deduction относятся к следующим блокам.

9.4 Instantiation: что требуется compiler

Когда program использует `maximum(4, 9)`, происходит implicit instantiation подходящей specialization `maximum<int>`. Использование со strings создаёт `maximum<std::string>`. Каждая проверяется как concrete typed function.

Стандарт задаёт правила well-formed program и observable behavior, но не обещает отдельную физическую копию machine code для каждой specialization. Compiler/linker могут inline, объединять эквивалентный code или вообще не оставить callable symbol после optimization. Обратное тоже возможно: разные specializations дают разные инструкции и занимают место.

9.5 Requirements: generic не означает «любой тип»

Implementation maximum содержит `left < right`. Значит, выбранный `T` должен предоставлять подходящую операцию `<`, а её результат должен работать как condition. Это **requirement** алгоритма.

Обязательная формулировка

Template не «принимает любой тип». Он работает с теми типами, которые удовлетворяют операциям, используемым implementation.

Для следующей функции requirements другие:

```
1  template <typename Container, typename Value>
2  std::size_t count_matching(const Container& values, const Value& wanted) {
3      std::size_t count{};
4      for (const auto& value : values) {
5          if (value == wanted) {
6              ++count;
7          }
8      }
9      return count;
10 }
```

Container должен поддерживать range-for, а его element — сравнение с Value через `==`. Функция не требует именно vector и не обещает весь interface контейнера. Range-for здесь использует уже знакомую форму; полный iterator model будет в блоке 14.

В простом C++17 interface эти requirements явно не записаны. Они проверяются при instantiation. Если вызвать maximum для NotOrdered без operator<, compiler укажет ошибку внутри template и цепочку вроде «required from here». Это compile-time error, а не runtime fallback.

9.6 Как читать template diagnostics

Длинная diagnostic часто содержит три уровня: 1) пользовательский call, который запросил instantiation; 2) concrete template arguments, например `T = NotOrdered`; 3) первая неподходящая operation внутри definition.

Формат GCC и Clang различается. Практический порядок: найти первый свой call, установленные template arguments и ближайшую первичную причину, а не исправлять все последующие сообщения. Negative tests сохраняют настоящие diagnostics в build-каталог, но PDF не копирует многоэкранный compiler dump.

В C++20 Concepts позволяют выразить часть requirements в interface и улучшить диагностику. Здесь мы не вводим их синтаксис: сначала важно увидеть сам контракт операций.

9.7 Value, const reference и mutable reference

Templates не отменяют семантику блока 4: `T value` создаёт отдельный parameter object; это уместно для маленького значения или нужной копии; `const T& value` читает существующий object без намеренной копии и подходит maximum; `T& value` означает изменение caller's object.

Выбор определяется `contract`, `lifetime` и `cost`, а не словом `template`. `maximum` возвращает `const T&`; в реальной библиотеке `caller` не должен сохранять эту `reference` дольше выбранного `argument`. Для простых вычислений часто безопаснее вернуть `T` по `value`. `Forwarding references` и `perfect forwarding` здесь намеренно не нужны.

Простой `return type T` понятен, когда `operation` действительно возвращает тот же `type`. Смешанное сложение разных `types` потребовало бы иной модели результата; не будем преждевременно вводить `decltype machinery`.

9.8 Class template: семейство concrete types

Если параметризуется не одна `operation`, а состояние вместе с `interface`, нужен `class template`:

```

1  template <typename T>
2  class Box {
3  public:
4      explicit Box(const T& value) : value_{value} {}
5      const T& value() const { return value_; }
6  private:
7      T value_;
8  };
9
10 Box<int> count{9};
11 Box<std::string> title{"templates"};
```

`Box` — `template`, а `Box<int>` и `Box<std::string>` — разные `concrete class types`. Objects `count` и `title` имеют соответствующие `specializations`. Между `Box<int>` и `Box<double>` автоматически нет `inheritance relation` и `implicit runtime substitutability`.

Methods, определённые внутри `class template`, используют тот же `T`. При внешнем `definition` параметр и `specialization` класса нужно повторить:

```

1  template <typename T>
2  const T& Box<T>::value() const {
3      return value_;
4  }
```

Для коротких `methods inline definition` обычно читается проще.

9.9 Non-type template parameter

`Template parameter` может быть `compile-time value`:

```

1  template <typename T, std::size_t N>
2  class FixedBuffer {
3  public:
4      constexpr std::size_t size() const { return N; }
5      T& operator[](std::size_t index) { return values_[index]; }
6  private:
7      std::array<T, N> values_{};
8  };
```

T — type template parameter, N — non-type template parameter типа `std::size_t`. Значение N известно при translation и входит в type: `FixedBuffer<int, 3>` и `FixedBuffer<int, 5>` — разные types. Это та же основная идея, что у знакомого `std::array<T, N>`.

В учебном `FixedBuffer` `operator[]` имеет precondition `index < N`; runtime bounds check не добавлен. Storage принадлежит object через `std::array`, поэтому отдельного dynamic allocation нет. Это иллюстрация, а не попытка заменить production `std::array`.

Class templates могут иметь default arguments, например `template <typename T = int> class Box`. Тогда `Box<>` выбирает `int`. Default полезен только при естественном частом выборе; скрывать важный design parameter ради короткой записи не следует.

9.10 Primary template, specialization и overloading

Обычное параметризованное определение называется **primary template**. Иногда один type требует действительно другой implementation:

```
1  template <typename T>
2  struct ValueLabel {
3      static const char* text() { return "value"; }
4  };
5
6  template <>
7  struct ValueLabel<bool> {
8      static const char* text() { return "logical value"; }
9  };
```

Вторая запись — explicit, или full, specialization для `bool`. Specialization повышает coupling к concrete type, поэтому её применяют редко и осмысленно, когда общий contract сохраняется. Partial specialization class templates существует, но нужна более поздним generic designs. Function templates частично специализировать нельзя.

Functions чаще расширяют overloading: рядом могут быть ordinary function и function template либо несколько templates с различными parameter lists. Overload участвует в выборе подходящей function; specialization задаёт вариант уже существующего template. Подробные правила выбора и SFINAE здесь не нужны.

9.11 Почему definitions templates часто находятся в headers

Для non-template function привычна схема: declaration в `.hpp`, body в одном `.cpp`, linker находит готовое definition. Для implicit instantiation compiler обычно должен видеть template definition и concrete arguments в точке, где specialization требуется. Одной declaration часто недостаточно, чтобы сгенерировать нужную specialization.

Поэтому небольшие template libraries обычно помещают полные definitions в headers с include guard. Это не означает «template невозможно определить в `.cpp`». Можно управлять набором через explicit instantiation, но тогда нужно заранее перечислить поддерживаемые arguments; такой способ менее generic и не относится к основной модели этой лекции.

Каждая translation unit может увидеть один и тот же header. ODR и toolchain обеспечивают корректное объединение допустимых одинаковых template entities; точные object symbols зависят от compiler/linker.

9.12 Compile-time и runtime polymorphism

В блоках 7–8 runtime Strategy использовала common abstract base и virtual call. Caller мог выбрать concrete strategy по runtime argument, а engine работал через base reference.

Generic algorithm использует иной extension point:

```

1  template <typename Blocks, typename Policy, typename Reporter>
2  AuditSummary run_audit(const Blocks& blocks,
3                        const Policy& policy,
4                        Reporter& reporter) {
5      AuditSummary summary;
6      for (const auto& block : blocks) {
7          const AuditResult result = policy.evaluate(block);
8          ++summary.checked;
9          if (result.passed) {
10             ++summary.passed;
11         }
12         reporter.on_result(result);
13     }
14     reporter.on_finish(summary);
15     return summary;
16 }

```

Concrete Policy и Reporter известны при instantiation. Им не нужна общая base class: достаточно evaluate, on_result и on_finish с подходящим смыслом. Это compile-time polymorphism.

	Templates	Virtual interface
Выбор варианта	при translation	может быть runtime
Common base	не обязателен	задаёт единый interface
Dispatch	concrete call, часто inline-able	обычно indirect virtual call
Расширение	новая instantiation	новый subtype через base
Основная цена	compile time, diagnostics, code size	interface/dispatch, lifetime design

Ни один подход не всегда лучше. Runtime Strategy нужна, когда policy выбирают из config во время выполнения, хранят разнородные strategies вместе или подключают через stable ABI. Template policy удобна, когда concrete type известен при сборке, common hierarchy не несёт смысла, а operations образуют достаточный compile-time contract.

9.13 Cost model без лозунга zero-cost

Template abstraction не обязана добавлять runtime dispatch или хранить type metadata в object. Compiler видит concrete type и может inline и специализировать code. Это возможность оптимизации, а не гарантия «быстрее».

Цена переносится и в toolchain: parsing и instantiation увеличивают compile time; длинные chains ухудшают diagnostics; specializations для разных types могут увеличить binary size. Оптимизатор может убрать или объединить часть code, поэтому code bloat измеряют для конкретной сборки. Benchmark и assembly в блоке N/A: архитектурное различие не требует выдуманного доказательства скорости.

9.14 Эксперимент 1: mini_generic_library

Практический каталог `mini_generic_library` содержит настоящий header `mini_generic.hpp`. В нём находятся: `maximum<T>` с requirement operator<; `count_matching<Container, Value>` с range-for и operator==; `FixedBuffer<T, N>` с type и non-type parameters; небольшая full specialization `ValueLabel<bool>`.

`Positive run` использует deduced `maximum<int>`, explicit `maximum<double>`, `std::string` и честный project type `Revision` с ordering по номеру. Подсчёт проверен на `std::vector<int>` и `std::vector<std::string>`; buffers — на двух T и двух N.

```
cd examples/09_templates/mini_generic_library
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/mini_generic_library
ctest --test-dir build --output-on-failure
```

```
maximum<int>=9
maximum<double>=3.5
maximum<string>=beta
maximum<Revision>=9
matching<int>=2
matching<string>=2
FixedBuffer<int,3>.size=3
FixedBuffer<string,2>.size=2
specialization<bool>=logical value
MINI_GENERIC_LIBRARY=PASS
```

CTest отдельно компилирует `negative/no_less.cpp`. Type `NotOrdered` не предоставляет <; PASS означает ненулевой exit code compiler и непустую реальную diagnostic в `build/negative/no_less.log`, а не поддельный текст ошибки.

9.15 Эксперимент 2: generic_course_audit

Второй проект read-only инспектирует настоящий набор из 15 blocks. Один `run_audit` инстанцируется для `CompactReporter` и `DetailedReporter`. Эти classes не наследуются от общего base: центральному workflow нужны только названные operations. Оба reporters должны обработать 15 blocks и согласиться, что после этой работы complete первые 9.

```
cd examples/09_templates/generic_course_audit
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
./build/generic_course_audit --root ../../..
ctest --test-dir build --output-on-failure
```

```
.....FFFFFF compact checked=15, passed=9
...
detailed checked=15, passed=9
reporters agree=true
GENERIC_COURSE_AUDIT=PASS
```

Точки означают successful blocks, F — будущие incomplete 10-15, а не сбой программы. До-

полнительный negative test передаёт BadReporter без on_result/on_finish; compiler отвергает соответствующую instantiation. В отличие от runtime Strategy блока 8, reporter нельзя заменить неизвестным subtype во время run. Здесь это уместно: набор output types известен при сборке и virtual hierarchy ничего не добавила бы к предметному смыслу.

PASS блока требует: обе CMake-конфигурации C++17, warning-clean builds, 5 CTest cases, оба positive markers, две сохранённые negative diagnostics и общую сборку PDF. Sanitizers и benchmark — N/A: практика не содержит memory-safety hypothesis или performance claim.

9.16 STL начинается с этой идеи

`std::vector<T>` — class template, поэтому element type — частью concrete vector type. `std::array<T, N>` дополнительно получает compile-time size. Generic algorithms описываются через требования к данным и операциям, а не копируются для каждого element type.

Это только мост. Мы ещё не изучаем container taxonomy, iterator categories, complexity contracts и standard algorithms: они составят блок 14. Теперь их архитектура не будет выглядеть магией — templates дают языковой фундамент для независимости container/algorithm от одного concrete element type.

9.17 Проверка понимания

Нужно уметь объяснить: 1) какое дублирование устраняет function template; 2) различия template, specialization и instantiation; 3) что выводит deduction и почему смешанные arguments могут конфликтовать; 4) почему generic не означает «для любого типа»; 5) requirements maximum, count_matching и run_audit; 6) почему `Box<int>` и `Box<std::string>` — разные types; 7) почему `N` в `FixedBuffer<T, N>` известен compile time; 8) зачем template definition обычно видима в месте instantiation; 9) когда выбрать template policy, а когда runtime Strategy; 10) compile-time, diagnostic и code-size costs; 11) как templates подводят к STL, не заменяя изучение её contracts.

9.18 Источники для сверки

Термины template parameters, deduction, implicit/explicit instantiation и explicit specialization сверены с C++17 working draft N4659, clauses [temp.param], [temp.deduct], [temp.inst] и [temp.expl.spec]. Базовая педагогическая последовательность сопоставлена с MIT 6.096 Advanced I; устаревший стиль источника заменён на C++17 и `std::`. Связь с vector/array проверена по предыдущим блокам и *Modern C++ Tutorial*; performance claims ограничены моделью ISO C++ Performance TR и не выданы за гарантии generated code.

10 Лекция 10. Ошибки API: status, exceptions и RAII

До сих пор примеры в основном шли по успешному пути. Но файл может отсутствовать, данные могут быть испорчены, а запись — запрещена операционной системой. Нужно решить, как сообщить об ошибке caller, провести её через несколько уровней и кто освободит уже полученные ресурсы.

Причинная линия

Успешный алгоритм → ошибка → проверки status на каждом уровне → отделение failure path через exception → stack unwinding → destructors локальных objects → RAII → осознанный выбор error contract.

Exceptions не «лучше» status codes. RAII также не требует exceptions: он обеспечивает cleanup при normal exit, early return и unwinding. Это две разные design decisions: как переносить failure и как владеть ресурсом.

10.1 Ошибка как часть contract API

Слово «ошибка» скрывает разные ситуации: обычный допустимый результат: поиск нашёл 0 совпадений; отсутствие значения в domain semantics: лекция 11 ещё не заполнена; ожидаемое recoverable condition: неверный ввод можно повторить; programming error: caller нарушил documented precondition; exceptional failure операции: обязательный отчёт нельзя прочитать; environment failure: file missing, permission denied, malformed input.

Это не абсолютная классификация. Missing file может быть нормальным результатом поиска optional config и failure обязательной загрузки. Поэтому способ сообщения — часть contract. Caller должен знать return type, возможные exceptions и гарантии о state после неуспеха. assert проверяет programming invariant в подходящих сборках; это не общий транспорт recoverable errors.

10.2 Baseline: явный status

В C++17 нет standard std::expected. Небольшому API достаточно простой структуры, не имитирующей целую библиотеку:

```
1 struct Status {
2     bool ok;
3     std::string message;
4 };
5
6 Status read_config(const Path& path, std::string& value);
```

Полезный result передаётся через reference, а return несёт success/failure. В нескольких слоях появляется propagation:

```
1 Status transform(const Path& input, const Path& output) {
2     std::string value;
3     Status status = read_config(input, value);
4     if (!status.ok) return status;
5     // normal algorithm
6     return {true, {}};
7 }
8
9 int command(const Path& input, const Path& output) {
10     const Status status = transform(input, output);
11     if (!status.ok) {
12         std::cerr << status.message << '\n';
13         return 2;
14     }
15 }
```



```
15     return 0;  
16 }
```

Плюсы: control flow виден в interface и source; частый ожидаемый исход удобно обрабатывать рядом. Цена: checks повторяются, пропущенная проверка теряет failure, normal value и error transport приходится совмещать. `[[nodiscard]]` помогает заметить игнорирование результата, но не проектирует result за автора.

10.3 Exceptions: синтаксис и semantics

```
1  std::string read_required(const Path& path) {  
2      std::ifstream input{path};  
3      if (!input) {  
4          throw std::runtime_error{"cannot open " + path.string()};  
5      }  
6      return read_value(input);  
7  }  
8  
9  try {  
10     build_report(path);  
11 } catch (const std::invalid_argument& error) {  
12     std::cerr << "bad data: " << error.what() << '\n';  
13 } catch (const std::exception& error) {  
14     std::cerr << "operation failed: " << error.what() << '\n';  
15 }
```

throw expression инициализирует exception object и начинает поиск matching handler. После throw normal path текущего scope не продолжается. try отмечает guarded statements, catch — handler. Более specific handler ставят раньше общего.

Если подходящего handler в текущей функции нет, поиск продолжается во внешнем динамическом call chain. Промежуточный уровень обычно не ловит failure, который не умеет исправить. Если exception достигает верхней границы без handler, runtime вызывает механизм завершения, включающий `std::terminate`; это точнее, чем обещание «просто crash» определённого вида.

Для standard hierarchy обычна форма catch (const std::exception& error). Reference избегает copy и slicing, const запрещает изменение, virtual what() даёт сообщение. Raw integers и strings технически допустимы, но это плохой production contract. Когда смысл совпадает, используют `std::runtime_error`, `std::invalid_argument`; знакомый library example `std::out_of_range` бросает, например, `vector::at`.

catch (...) совпадает с любым exception. Он нужен редко, например для boundary action и rethrow. Он не даёт object для what() и не оправдывает молчаливое проглатывание ошибки.

10.4 Rethrow

```
1  void middle() {  
2      try {  
3          low_level();  
4      } catch (const std::exception& error) {
```

```
5     std::cerr << "middle context: " << error.what() << '\n';  
6     throw;  
7 }  
8 }
```

Пустой `throw`; внутри handler повторно бросает текущее exception и сохраняет его type. `throw error`; — новый throw из expression: object копируется по static type expression, поэтому возможно slicing. Rethrow уместен, если слой добавил локальный контекст, но не способен восстановиться.

10.5 Stack unwinding и lifetime

Когда exception покидает scope, уничтожаются уже сконструированные automatic objects, lifetime которых заканчивается из-за выхода. Их destructors вызываются в порядке, обратном construction. Затем поиск handler продолжается наружу. Это называется **stack unwinding**.

```
1 void low() {  
2     Trace low{"low"};  
3     throw std::runtime_error{"deep failure"};  
4 }  
5  
6 void middle() {  
7     Trace middle{"middle"};  
8     low();  
9 }
```

Эксперимент печатает:

```
+ top  
+ middle  
+ low  
- low  
- middle  
- top  
top boundary caught: deep failure
```

Это наблюдение destructors, а не требование реализовать automatic storage как конкретные CPU stack frames. Standard задаёт lifetime и destruction; ABI и machine representation выбирает implementation.

10.6 RAII: ресурс принадлежит object lifetime

RAII — Resource Acquisition Is Initialization. Owner получает ресурс при construction (или создаётся сразу в корректном owning state), хранит его и освобождает в destructor. Scope задаёт lifetime. Cleanup срабатывает при обычном выходе, early return и unwinding.

Resource — не только heap memory. Это file handle, temporary file, mutex lock, socket/OS handle, transaction guard или logging scope. Smart pointers — лишь будущий пример RAII; для понимания механизма они не нужны.

Без owner алгоритм обрастает ветками:

```

1 Handle h = acquire();
2 if (!operation_a(h)) {
3     release(h);
4     return failure;
5 }
6 operation_b(h);           // may throw: кто вызовет release?
7 release(h);

```

С RAII основной алгоритм не отвечает за release:

```

1 ResourceOwner owner{acquire()};
2 if (!operation_a(owner)) return failure;
3 operation_b(owner);       // destructor работает и при throw

```

Standard library уже использует RAII. `std::ifstream/ofstream` закрывают файл, `std::vector` и `std::string` освобождают storage. Не нужен wrapper без новой ответственности. Учебный `TemporaryReport` добавляет policy: удалить temporary artifact, если commit не состоялся.

10.7 RAII-owner и transactional thinking

Owner из `error_model_lab` открывает `report.tmp` через `std::ofstream`. Сору запрещена через `= delete`: два owners не должны владеть одним файлом. Move semantics до блока 12 не нужны, поскольку owner остаётся local object.

При `success commit()` закрывает stream и переименовывает temporary file. При `status early return` или `exception committed_` остаётся false; destructor закрывает stream и удаляет temporary path. Release не дублируется.

Это transactional pattern: подготовить новое состояние отдельно, а опубликовать его после успеха. До `commit` новый report не виден; при failure temporary state убирается. Пример не обещает универсальную atomic replace semantics для всех OS и filesystems: это отдельный contract production utility.

10.8 Constructor failure

Если constructor бросает, полностью constructed object не появляется и его destructor не вызывается как для готового object. Уже успешно constructed base/member subobjects уничтожаются. Так owner получает сильный invariant, либо не существовать. В `TemporaryReport` stream-member уже создан к моменту проверки; при throw он будет уничтожен автоматически.

10.9 Destructor и exceptions

Destructor выполняется и при active unwinding. Если из destructor наружу выходит второй exception, пока обрабатывается первый, вызывается `std::terminate`. Destructors имеют связанные с bases/members implicit поехсепт свойства; здесь достаточно правила: обычный cleanup destructor не должен бросать.

Rollback использует filesystem overload с `std::error_code` и не выпускает cleanup failure. Production code может безопасно фиксировать его в telemetry или дать отдельную явную

close/commit, способную сообщить ошибку до destructor. Destructor не должен ложно обещать успешное сохранение.

10.10 Exception-safety guarantees

Guarantee описывает state после failure: **No-throw guarantee.**: Operation обещает не выпускать exception и выполнить свою задачу; обычный кандидат — простой cleanup destructor; **Strong guarantee.**: Либо success, либо observable state как до operation; **Basic guarantee.**: Invariants сохранены и leaks нет, но state мог измениться; **No guarantee.**: После failure полезного обещания о state нет.

В лаборатории rollback обеспечивает отсутствие нового temporary artifact и утечки stream на проверяемых failure paths. Prepare-then-commit даёт strong guarantee относительно появления нового report. Это не доказательство полной transaction atomicity при любом filesystem failure.

10.11 Boundary design

Полезная architecture: 1) low-level operation бросает rich exception об I/O failure; 2) intermediate functions не ловят его без возможности восстановиться; 3) top-level main/command handler ловит по const&; 4) boundary превращает failure в сообщение и nonzero exit code.

Catch после каждого вызова уничтожил бы отделение failure path. Но ожидаемое «block incomplete» в course audit не exception: это BlockResult, который отчёт обязан показать. Выбран hybrid: domain-negative results являются data; невозможность выполнить audit переносится exception; CLI — основная catch boundary. Status mode оставлен рядом для сравнения того же domain logic.

10.12 Exceptions или status: честное сравнение

Критерий	Status/result	Exception
Ожидаемая альтернатива	Явна в return type	Менее ясна как частая ветвь
Propagation	Повторные checks	До matching handler
Normal algorithm	Смешан с transport	Отделён от failure path
Control flow	Локально виден	Non-local, требует дисциплины
Cleanup	RAII работает	RAII и unwinding вызывают destructors
Rich context	Нужен error type	Переносит typed object
Цена	Checks на normal path	Throw path дорог; возможны tables/code size

Стоимость non-throwing path зависит от compiler и ABI. Реальный throw обычно существенно дороже return, поэтому exceptions не используют как частый loop control. Неверны обе крайности: «exceptions всегда медленные» и «бесплатны». Destructor work RAIi выполняется при окончании lifetime независимо от транспорта. Стабильный benchmark для этого design conclusion не нужен: BENCHMARKS=N/A.

10.13 Standard library использует разные модели

`vector::at` бросает `std::out_of_range`. Filesystem API часто предлагает throwing overload и overload со `std::error_code`. Streams по умолчанию имеют state flags и могут быть configured бросать по exception mask. Не все failures standard library обязательно throw: читают contract операции.

10.14 Эксперимент 1: error_model_lab

Обе версии решают одну задачу: low-level читает config, middle преобразует его и пишет report, top выдаёт user-facing result. Status version проверяет Status на трёх уровнях. Exception version бросает вниз, middle обычно не ловит, а main ловит по `const&`.

Modes детерминированно вызывают failure после acquisition, trace unwinding и handler с throw;. PASS: success создаёт final report; failure даёт ожидаемый code; temporary file удалён; trace показывает reverse destruction.

```
cd examples/10_exceptions_and_raii/error_model_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./build/error_model_lab unwind build
./build/error_model_lab rethrow build
```

10.15 Эксперимент 2: course_audit_error_boundary

Продолжение `course_audit_patterns` и `generic_course_audit` сохраняет domain model, но делает error policy явной. Utility проверяет 15 pairs. Заготовки 11-15 дают нормальные INCOMPLETE; synthetic I/O failure — operational failure.

Exception mode не размазывает catch по `line_count`, `inspect` и `audit_exception`; main переводит failure в сообщение и code 2. Status mode преобразует тот же failure в Status. Основная project policy — hybrid, потому что она не путает future content state с I/O failure.

```
cd examples/10_exceptions_and_raii/course_audit_error_boundary
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./build/course_audit_error_boundary --root ../../.. --transport exception
```

PASS: обе transport versions проверяют 15 blocks; expected incomplete не даёт failure exit; synthetic I/O failure даёт nonzero code и boundary diagnostic. Sanitizers не применялись: manual memory ownership и намеренного UB здесь нет.

10.16 Проверка понимания

После блока нужно уметь: 1) отличить domain result, precondition violation и operation failure; 2) объяснить throw, try, matching catch и поиск наружу; 3) мотивировать catch standard exception по `const&`; 4) отличить throw; от throw error;; 5) по trace восстановить destruction при unwinding; 6) определить RAII без smart pointers и назвать разные resources; 7) объяснить cleanup при return, normal exit и exception; 8) описать constructor failure и риск

throwing destructor; 9) различить no-throw, strong, basic и no guarantee; 10) выбрать status или exception по contract, frequency и boundary.

10.17 Источники для сверки

Throwing, handler matching, rethrow, termination, construction/destruction и exception safety сверены с C++17 working draft N4659, прежде всего clauses 15 и 18, lifetime — также с clause 6. RAII сопоставлен с локальными MIT 6.096 materials по memory management и блоками 6, 8, 9. Cost claims ограничены implementation-dependent формулировками C++ Performance TR; build согласован с target-based Modern CMake. Старые материалы не использованы как modern style.

11 Лекция 11. Умные указатели и владение

В предыдущих лекциях мы уже встречали raw pointers, lifetime, RAII и полиморфные interfaces. Остался центральный design-вопрос: *кто отвечает за уничтожение объекта?* Сам тип T* на этот вопрос не отвечает. Указатель может быть лишь временным способом обратиться к уже существующему object, а может случайно оказаться единственным адресом dynamically allocated resource. Во втором случае ошибка ownership быстро превращается в leak, double delete или dangling pointer.

Причинная линия

Raw pointer не кодирует владение → ownership нужно выразить в API → unique_ptr задаёт одного owner → shared_ptr задаёт совместное владение → weak_ptr наблюдает shared object, не продлевая его lifetime.

Главная цель smart pointers — не «автоматически сделать любой pointer безопасным». Они делают **ownership** явной частью типа и используют RAII: когда owning smart pointer заканчивает lifetime, он выполняет соответствующий release. При этом обычные references и raw pointers остаются полезными для *non-owning access*.

11.1 Сначала словарь: object, access, ownership

Разделим три понятия.

Object имеет lifetime. Пока lifetime не начался или уже закончился, обращаться к нему как к живому object нельзя.

Access означает возможность обратиться к object. Reference T&, const T& или raw pointer T* могут предоставлять такой доступ, ничего не говоря об обязанности уничтожить object.

Ownership — ответственность за окончание lifetime ресурса в нужный момент. Для dynamically allocated object это обычно означает, что некоторый owner обязан обеспечить ровно один корректный release.

Почему raw pointer недостаточен как ownership contract?

По declaration Widget* p caller не видит, должен ли он вызвать delete p, запрещено ли это, может ли p быть null и кто должен жить дольше кого. Эти сведения приходится хранить в комментариях и соглашениях. Smart pointer переносит важную часть этого contract в type system.

В современном интерфейсе полезна следующая исходная эвристика: `T&/const T&`: объект обязан существовать; функция его не владеет; `T*`: non-owning access, если допустимо отсутствие объекта или pointer semantics действительно удобна; `std::unique_ptr<T>`: один owner; `std::shared_ptr<T>`: несколько owners действительно должны совместно продлевать lifetime; `std::weak_ptr<T>`: наблюдение за объектом, которым владеют `shared_ptr`, без продления lifetime.

Это design guideline, а не правило синтаксиса стандарта. Legacy API может использовать raw owning pointers, но такой contract труднее читать и проверять.

11.2 Проблема naked new/delete

Учебный код легко написать так:

```
1 Product* make_product() {
2     return new Product{/*...*/};
3 }
4
5 void command() {
6     Product* product = make_product();
7     use(*product);           // а если здесь throw?
8     delete product;
9 }
```

Здесь ownership существует только в голове программиста. Early return, exception или новый путь control flow легко пропускает delete. Передача product другой функции также не отвечает на вопрос, передаётся ли ownership.

RAII-идея из лекции 10: ответственность за release нужно поместить в destructor объекта-owner. Standard library уже предоставляет такие owners.

11.3 std::unique_ptr: один владелец

`std::unique_ptr<T>` находится в `<memory>`. Он владеет одним object (или пуст) и при уничтожении owner освобождает принадлежащий ему object подходящим deleter.

```
1 #include <memory>
2
3 auto product = std::make_unique<Product>();
4 product->run();
```

`std::make_unique<T>(args...)` создаёт T и возвращает `unique_ptr<T>`. В C++14 и новее это обычный предпочтительный способ получить unique ownership: raw owning new не оказывается между несколькими выражениями user code.

Что гарантируется семантически

`unique_ptr` movable, но не copyable owner. Два независимых `unique_ptr` не могут обычным копированием начать владеть одним и тем же object. Это важная часть ownership contract.

Попытка:

```

1 auto first = std::make_unique<int>(42);
2 auto second = first; // compile error: copy запрещена

```

проверяется настоящим negative compile test в эксперименте.

11.4 Доступ через unique_ptr

Owner ведёт себя pointer-like: *owner — object; owner->member() — member access; owner.get() — raw pointer без передачи ownership; if (owner) — проверка, есть ли owned object; owner.reset() — заменить/освободить owned object; owner.release() — отдать raw pointer и отказаться от владения.

Осторожно с release()

После release() smart pointer уже не отвечает за cleanup. Caller снова получает manual ownership problem. Это низкоуровневая операция для interop и специальных contracts, а не обычный способ «достать pointer». Для временного non-owning access нужен get().

11.5 Передача unique ownership

Ownership можно передать другому owner. Для уже существующей named variable обычно нужна операция std::move:

```

1 void consume(std::unique_ptr<Job> job);
2
3 auto job = std::make_unique<Job>();
4 consume(std::move(job));
5
6 if (!job) {
7     // ownership ушло в consume
8 }

```

Здесь std::move используется только как минимальный bridge: он разрешает выбрать move operation у movable type. Полная модель lvalue/rvalue, value categories и то, что именно делает std::move, — тема следующего блока. Сейчас достаточно увидеть ownership contract: после передачи old owner больше не владеет объектом.

Функция, которая *принимает ownership*, может принимать std::unique_ptr<T> by value. Функция, которая только использует объект, обычно не должна требовать unique pointer:

```

1 void print_product(const Product& product); // borrow
2 void maybe_print(const Product* product); // nullable borrow

```

Так caller не обязан менять способ владения только ради обычного чтения.

11.6 Возврат ownership из factory

В блоке 08 Factory Method специально обходился без smart pointers: creator хранил concrete product внутри себя и возвращал non-owning reference. Теперь мы можем выразить другой contract: factory создаёт новый polymorphic object и *передаёт caller единственное владение*.


```
1 class Product {
2 public:
3     virtual ~Product() = default;
4     virtual std::string format() const = 0;
5 };
6
7 class Creator {
8 public:
9     virtual ~Creator() = default;
10    std::unique_ptr<Product> create() const {
11        return make_product();
12    }
13
14 protected:
15     virtual std::unique_ptr<Product> make_product() const = 0;
16 };
```

Concrete creator возвращает, например, `std::make_unique<TextProduct>()`. Преобразование `unique_ptr<Derived>` в совместимый `unique_ptr<Base>` поддерживает polymorphic ownership. Virtual destructor base здесь снова принципиален. В итоге unique owner хранит `Product*`, но уничтожить нужно полный dynamic object.

Почему factory return type полезен

Return type `std::unique_ptr<Product>` сразу сообщает caller: «ты получаешь новый object и теперь отвечаешь за его lifetime, но cleanup автоматический». Return `Product*` этого не выражал бы.

11.7 Rule of Zero возвращается

Если class хранит resource через `unique_ptr` или standard containers, ему часто не требуется manual destructor. Это продолжает Rule of Zero из лекции 6: resource-managing member сам выполняет cleanup.

Но есть важное следствие. Class с `unique_ptr` member обычно сам становится non-copyable, потому что unique ownership нельзя копировать автоматически. Позже move semantics позволит таким объектам передавать ownership как значение.

11.8 Массивы и unique ownership

Существует специализация `std::unique_ptr<T[]>` для dynamically allocated arrays, и `std::make_unique<T[]>(n)` умеет её создавать.

Для обычной динамической последовательности чаще удобен `std::vector<T>`. Он хранит size, поддерживает iteration и value semantics. `unique_ptr<T[]>` полезен, когда задача действительно требует низкоуровневого owning array buffer.

Smart pointer не заменяет container abstraction.

11.9 std::shared_ptr: совместное владение

Иногда одного owner недостаточно по смыслу. Представим audit session, которая должна оставаться живой, пока ею независимо пользуются console view и statistics view. Ни один

из них не подходит на роль единственного owner.

```

1 auto session = std::make_shared<AuditSession>("block-11");
2
3 ConsoleView console{session};
4 StatisticsView stats{session};
5 session.reset(); // views всё ещё совместно владеют session

```

Копирование `shared_ptr` создаёт ещё одного owner того же object. Object уничтожается, когда исчезает последний owning `shared_ptr`.

Обычная реализация

Обычно существует control block со счётчиками strong/weak references и информацией, необходимой для destruction. `shared_ptr` обращается к нему при copy/destruction. Standard определяет observable semantics, но не требует конкретного layout control block или конкретной ABI-структуры.

`std::make_shared<T>` обычно предпочтительнее отдельного `shared_ptr<T>` с ручным `new T(...)`: код проще, exception-safe, а обычная implementation может объединить allocation object и control block. Последнее — optimization possibility, а не переносимая гарантия числа allocations для всех реализаций.

11.10 use_count() не проектирует ownership

`shared_ptr::use_count()` показывает число owning references в момент наблюдения и полезен в учебных diagnostics. Но application logic не должна обычно строиться на предположении «если count равен 1, то...»: между потоками и изменениями owners это хрупкое знание, а сам count не объясняет правильную модель владения.

Сначала проектируют, почему owners несколько; затем выбирают `shared_ptr`.

11.11 Цена shared ownership

Shared ownership удобнее, но не бесплатно: control block занимает память; copy/reset owners изменяют reference count; typical thread-safe reference-count updates могут требовать atomic operations; lifetime становится менее локально очевидным; циклы strong ownership не разрываются автоматически.

Поэтому policy проекта остаётся простой: если естественный owner один, предпочитаем `unique_ptr`. `shared_ptr` появляется только при реальном shared lifetime requirement.

11.12 Цикл shared_ptr

Рассмотрим два objects, каждый из которых владеет другим:

```

1 struct Node {
2     std::shared_ptr<Node> peer;
3 };
4
5 auto a = std::make_shared<Node>();
6 auto b = std::make_shared<Node>();
7 a->peer = b;

```

```
8 b->peer = a;
```

После уничтожения local owners `a` и `b` каждый Node всё ещё имеет одного strong owner: соседний Node. Reference counts не становятся нулём, destructors не вызываются, возникает resource leak.

Это не dangling-pointer UB. Наоборот, objects остаются логически «живыми» из-за неправильной ownership graph.

11.13 std::weak_ptr: наблюдение без владения

`std::weak_ptr<T>` связан с object, которым владеют `shared_ptr`, но сам не увеличивает strong owner count.

Он нужен для non-owning edges в shared graph: parent link в tree, где parent владеет children; cache observation; subscriber, который не должен продлевать lifetime publisher; back-reference, разрывающий ownership cycle.

У `weak_ptr` нет обычного operator->: сначала нужно временно получить owning `shared_ptr` через `lock()`.

```
1 std::weak_ptr<Session> observed = session;
2
3 if (auto alive = observed.lock()) {
4     alive->print();
5 } else {
6     // object уже уничтожен
7 }
```

`lock()` атомарно относительно shared ownership semantics получает `shared_ptr`, если object ещё существует, либо пустой `shared_ptr`. `expired()` сообщает, что owning object уже исчез, но для непосредственного использования всё равно удобнее проверять `result lock()`.

11.14 Правильное направление ownership graph

Для tree естественная модель:

```
1 class Node {
2     std::vector<std::shared_ptr<Node>> children_; // owning edges
3     std::weak_ptr<Node> parent_;                // non-owning back edge
4 };
```

Это не означает, что любой tree обязан использовать `shared_ptr`. Если весь tree имеет одного owner, `unique_ptr` для children может быть лучше. Пример показывает именно принцип: ownership direction должен следовать lifetime responsibility, а обратная навигация не обязана владеть.

11.15 Raw pointer после smart pointer остаётся нормальным

Smart pointers не отменяют raw pointers. Ошибка — считать любой `T*` owning pointer без ясного contract.

```

1 void inspect(const Product& product);
2
3 auto owner = std::make_unique<TextProduct>();
4 inspect(*owner);           // borrow через reference
5
6 const Product* view = owner.get(); // non-owning pointer
7 if (view != nullptr) {
8     inspect(*view);
9 }

```

view не должен пережить owner. Smart pointer гарантирует cleanup owned object, но не может автоматически сделать безопасными все заимствованные pointers/references.

Smart pointer не лечит dangling borrow

Если сохранить `owner.get()`, затем уничтожить `owner` и после этого разыменовать raw pointer, получится прежняя lifetime error. Ownership стало явнее, но lifetime non-owning views всё ещё нужно соблюдать.

11.16 Как читать parameter types

Практическая таблица API:

Parameter	Обычно означает	Вопрос caller
<code>const T&</code>	read-only borrow	object точно существует?
<code>T&</code>	mutable borrow	mutation разрешена?
<code>const T*/T*</code>	nullable/non-owning view	возможен null?
<code>unique_ptr<T> by value</code>	transfer unique ownership	готов отдать owner?
<code>const unique_ptr<T>&</code>	обычно слишком привязанный API	почему нужен именно owner type?
<code>shared_ptr<T> by value</code>	функция становится co-owner	shared lifetime действительно нужен?
<code>const shared_ptr<T>&</code>	доступ к handle без нового owner	нужен именно handle contract?
<code>weak_ptr<T></code>	non-owning shared observation	object может исчезнуть?

Нет запрета принимать smart pointer by reference. Но если функция лишь вызывает методы T, parameter `T&/const T&` обычно слабее связывает API с конкретной ownership strategy.

11.17 Return types и ownership

Return type особенно хорошо выражает создание: T — вернуть самостоятельное value; `unique_ptr<T>` — вернуть единственного owner polymorphic/dynamic object; `shared_ptr<T>` — вернуть shareable owner, если API действительно обещает shared lifetime; `T*/T&` — обычно вернуть non-owning view на object, lifetime которого обеспечивается где-то ещё.

Возвращать `shared_ptr` «на всякий случай» плохо: caller уже не может понять, почему ownership должна быть shared, а code получает лишний control block/reference-counting contract.

11.18 Exception safety и smart pointers

Smart pointers являются RAII owners, поэтому хорошо сочетаются с exceptions.

```
1 auto item = std::make_unique<Item>();
2 prepare(*item);           // may throw
3 store(std::move(item));
```

Если prepare бросает до transfer, local unique owner уничтожится при stack unwinding и освободит Item. Manual delete branch не нужен.

make_unique/make_shared также уменьшают число мест, где raw resource временно существует без owner.

11.19 Ошибочные способы создать несколько owners

Нельзя создавать два независимых smart owners из одного raw owning pointer:

```
1 Widget* raw = new Widget;
2 std::shared_ptr<Widget> a{raw};
3 std::shared_ptr<Widget> b{raw}; // два разных control blocks: опасно
```

Это не «два shared owners», а два независимых ownership systems, каждый считает, что обязан уничтожить один object. Итог способен привести к double delete.

Правильный shared owner копируют:

```
1 auto a = std::make_shared<Widget>();
2 auto b = a; // один shared ownership group
```

А unique owner вообще не копируют.

11.20 Нужно ли всегда хранить shared_ptr в members?

Нет. Member type должен выражать responsibility конкретного class. Если class обязан продлевать lifetime collaborator, shared owner оправдан. Если collaborator гарантированно живёт снаружи дольше class, reference/raw non-owning pointer может быть точнее.

Но такой borrow contract должен быть простым и доказуемым. Smart pointers не заменяют design lifetime graph.

11.21 Custom deleter: зачем существует

Иногда resource освобождается не через обычный delete: C API может требовать fclose, OS API — специальную release function. unique_ptr и shared_ptr поддерживают custom deleters.

В этой лекции мы не строим большой wrapper вокруг C resource: RAII-механизм уже был изучен в блоке 10. Важно только увидеть общую идею: smart pointer — это policy-owning RAII handle, и способ destruction может быть частью его type/state.

11.22 Потокбезопасность: аккуратная граница обещаний

Shared ownership имеет thread-safety guarantees для операций над разными shared_ptr objects, относящимися к одному control block, но это не делает сам T thread-safe. Два threads, меняющие один T через shared owners без собственной synchronization, всё ещё могут создать data race.

Полную concurrency theory здесь не вводим.

11.23 Стоимость: что именно мы покупаем

unique_ptr обычно проектируется как очень лёгкий owner: для обычного deleter обычная implementation хранит один pointer, а destruction сводится к проверке/удалению. Но стандарт не обещает универсально sizeof(unique_ptr<T>) == sizeof(T*): custom deleter и implementation могут менять representation.

shared_ptr обычно дороже: pointer/handle state; control block; reference-count updates; дополнительная indirection; более сложный lifetime.

Это цена именно shared ownership, а не причина избегать его там, где несколько owners необходимы.

Не измеряем бессмысленный microbenchmark

Главный выбор здесь семантический: один owner или несколько. Если ownership model неверна, выигрыш в несколько наносекунд не исправляет design. Benchmark уместен только после выбора корректной модели и при доказанном hot path. Поэтому в блоке BENCHMARKS=N/A.

11.24 Эксперимент 1: ownership_factory_lab

Каталог: examples/11_smart_pointers/01_ownership_factory_lab.

Эксперимент возвращается к Factory Method из блока 08, но теперь factory создаёт новый polymorphic Product и возвращает std::unique_ptr<Product>.

Проверяются четыре идеи: 1) concrete creators возвращают unique ownership через base interface; 2) virtual destructor уничтожает полный dynamic object; 3) ownership можно передать через std::move, после чего old owner пуст; 4) helper, который лишь читает Product, принимает reference/raw non-owning view, а не smart pointer.

Отдельный copy_unique_fail.cpp действительно не компилируется: копирование unique_ptr запрещено.

```
cd examples/11_smart_pointers/01_ownership_factory_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./run_checks.sh
```

PASS означает, что все runtime cases успешны, negative compile действительно отклонён compiler, а ownership/destructor trace совпадает с contract.

11.25 Эксперимент 2: shared_weak_graph_lab

Каталог: `examples/11_smart_pointers/02_shared_weak_graph_lab`.

Здесь есть три разных lifetime graph: 1) несколько independent views совместно владеют одним `AuditSession`; 2) два strong back-links создают цикл, и после внешнего scope destructor count остаётся нулевым; 3) tree хранит children через `shared_ptr`, а parent back-link через `weak_ptr`; цикл исчезает, destructors вызываются.

Также проверяется `weak_ptr::lock`: пока strong owner существует, `lock()` успешен; после его уничтожения weak observer сообщает expiration.

```
cd examples/11_smart_pointers/02_shared_weak_graph_lab
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./build/shared_weak_graph_lab
```

Shared-cycle case намеренно демонстрирует logical leak в отдельном process. Он не используется как production design и не UB. Исправленный weak graph завершается нормальным destruction trace.

11.26 Сводка выбора

Ситуация	Инструмент	Что владеет	Главный риск
Один owner	<code>unique_ptr</code>	один handle	premature transfer
Несколько owners	<code>shared_ptr</code>	ownership group	cycles/overuse
Наблюдение shared object	<code>weak_ptr</code>	никто дополнительно	object может исчезнуть
Короткий обязательный borrow	<code>T&</code>	никто	owner должен жить дольше
Nullable borrow	<code>T*</code>	никто	null/dangling

Главный вывод

Smart pointer выбирают не по правилу «raw pointer плохой», а по вопросу ownership. Unique ownership — default для dynamically owned object. Shared ownership требует реального основания. Borrowing API обычно выражается references/raw pointers и не должен случайно продлевать lifetime.

Следующий блок объяснит move semantics и value categories. Тогда станет понятно, почему `unique_ptr` можно передавать, но нельзя копировать, и что именно означает уже использованный здесь `std::move`.

12 Лекция 12. Move-семантика и явные приведения типов

В лекции 11 возникла странная на первый взгляд операция: `unique_ptr` нельзя копировать, но его можно передать другому owner через `std::move`. Теперь разберём, что именно означает эта запись и почему C++ различает выражения не только по типу, но и по **value category**.

Вторая половина лекции посвящена похожей design-идее: если programmer намеренно ме-

няет способ интерпретации значения или типа, это намерение лучше показать явно. Поэтому современный C++ предпочитает специализированные casts `static_cast`, `dynamic_cast`, `const_cast` и `reinterpret_cast` вместо C-style cast.

Сквозная идея

Move и C++ casts не являются «магическими действиями». Они меняют то, *как expression участвует в type system*. После этого обычные constructors, overload resolution и conversion rules определяют реальную операцию.

12.1 Почему одного сору недостаточно

Представим object, который владеет большим buffer или `std::unique_ptr<Resource>`. Copy должен создать независимое состояние: для большого buffer это может означать allocation и копирование всех элементов, а `unique ownership` вообще не допускает двух owners одного resource.

Но часто старый object после передачи больше не нужен:

```
1 Packet packet = make_packet();  
2 send(/* packet больше не нужен после этого вызова */);
```

Вместо глубокого сору можно **передать** внутренний resource новому object, а старый оставить в корректном, но опустошённом состоянии. Это и есть главный смысл move semantics.

12.2 Expression и object — не одно и то же

Object существует в памяти в течение lifetime. Expression — фрагмент программы, который вычисляется и имеет type и value category.

Один и тот же object может появляться в expressions разных категорий:

```
1 Packet packet{"report"};  
2  
3 inspect(packet);           // expression packet: lvalue  
4 inspect(std::move(packet)); // expression std::move(packet): xvalue
```

Сам object packet не превращается в другой object. Меняется категория expression, через которое мы к нему обращаемся.

12.3 Карта value categories в C++17

Удобно сначала увидеть полную схему:

Категория	Интуиция	Обычный пример
lvalue	glvalue с устойчивой идентичностью	named variable
xvalue	glvalue, чей resource можно переиспользовать	<code>std::move(x)</code>
prvalue	вычисляет значение для инициализации/result	<code>Packet{"x"}</code>
glvalue	lvalue или xvalue; expression идентифицирует entity	<code>x</code> , <code>std::move(x)</code>
rvalue	prvalue или xvalue	temporary, <code>std::move(x)</code>

Эта схема точнее старой фразы «lvalue стоит слева от присваивания». Например, `const lvalue` может вообще не быть допустимой левой частью `assignment`.

Минимальная точная модель

В C++17 lvalue и xvalue относятся к glvalue; prvalue и xvalue относятся к rvalue. Xvalue одновременно glvalue и rvalue.

12.4 Lvalue

Named variable expression обычно lvalue:

```
1 std::string name{"block-12"};
2 std::string& ref = name;
```

Lvalue удобно понимать как expression, который идентифицирует уже существующий object/function и обычно позволяет продолжать обращаться к той же сущности.

Но наличие адреса — плохое универсальное определение lvalue: язык задаёт категории семантически, а не тестом «можно ли вызвать operator &».

12.5 Prvalue

Temporary/value-producing expression часто prvalue:

```
1 std::string{"temporary"}
2 7 + 5
3 make_value()
```

В C++17 prvalue тесно связан с инициализацией результата. Не нужно представлять, что compiler обязан сначала создать отдельный временный object, а затем обязательно скопировать его в destination: C++17 во многих случаях прямо строит result object на нужном месте.

12.6 Xvalue

Xvalue — glvalue, обозначающий object, resource которого разрешено переиспользовать. Самый знакомый источник xvalue:

```
1 std::move(packet)
```

Именно к `xvalue` обычно привязывается `rvalue reference`, позволяя выбрать `move overload`.

12.7 Rvalue references

Синтаксис:

```
1 Packet&& ref = Packet{"temporary"};
```

`Packet&&` — `rvalue reference`. Она может привязаться к подходящему `rvalue`. Такой `parameter` позволяет `overload-у` сказать: «`caller` предоставляет `object`, состояние которого можно забрать».

Обычная пара:

```
1 void process(const Packet& packet); // borrow/read
2 void process(Packet&& packet);      // можно переиспользовать resource
```

`Overload resolution` выбирает подходящую функцию из `type` и `value category argument`.

12.8 Named rvalue reference — lvalue-expression

Это один из самых важных моментов:

```
1 void relay(Packet&& packet) {
2     process(packet);          // packet --- named expression => lvalue
3     process(std::move(packet)); // xvalue => rvalue overload
4 }
```

У `variable packet` declared `type` содержит `&&`, но `expression` с её именем — `lvalue`. Поэтому `move` не «распространяется автоматически» через `named variables`.

12.9 Что на самом деле делает `std::move`

Несмотря на имя, `std::move` сам по себе не переносит `bytes` и не обнуляет `object`. Концептуально он выполняет `cast` к подходящей `rvalue-reference` категории:

```
1 // Упрощённая идея, не реализация для копирования:
2 static_cast<T&&>(object)
```

Реальная `library template` учитывает `reference removal`. Результат `std::move(x)` — `xvalue`. После этого `overload resolution` может выбрать `move constructor` или `move assignment operator`.

Почему после `std::move` иногда всё равно происходит `copy`?

`std::move` только меняет `category expression`. Если подходящей `move operation` нет, но существует `copy operation`, способная принять такой `argument`, `compiler` может выбрать `copy`. Поэтому фраза «`std::move` перемещает» технически неточна.

12.10 Move constructor

Обычная форма:

```
1 class Packet {
2 public:
3     Packet(Packet&& other) noexcept
4         : data_{std::move(other.data_)} {}
5 };
```

Новый Packet создаётся из rvalue другого Packet. Для unique-owned resource move constructor обычно передаёт ownership member-у нового object.

Если member — unique_ptr, его собственная move operation делает source smart pointer пустым.

12.11 Move assignment

Move assignment работает с *уже существующим* destination:

```
1 Packet& operator=(Packet&& other) noexcept {
2     if (&this != &other) {
3         data_ = std::move(other.data_);
4     }
5     return *this;
6 }
```

Как и в copy semantics из блока 06:

```
1 Packet b = std::move(a); // construction
2 b = std::move(c);        // assignment
```

— это разные операции.

Self-move нужно учитывать в contract конкретного type. Не следует строить общую программу на предположении, что после `x = std::move(x)` состояние обязательно сохраняется прежним.

12.12 Moved-from object

После move source object продолжает существовать, пока не закончился его lifetime. Но его состояние определяется contract типа.

Для многих standard-library types после move гарантируется состояние *valid but unspecified*: invariants сохранены, destructor и операции, не требующие конкретного старого значения, допустимы, но прежнее содержимое предполагать нельзя.

Не универсальное правило «после move всегда пусто»

unique_ptr после move действительно становится empty по своему contract. Для произвольного std::string нельзя переносимо строить логику на том, что empty() обязательно true после move.

Для собственного Packet мы вправе определить более сильный contract: например, moved-from packet становится пустым и может быть заново присвоен.

12.13 Rule of Five

В блоке 06 был Rule of Three. После появления move operations design-guideline расширяется до **Rule of Five**: если class вручную определяет resource semantics и вынужден писать один из destructor; copy constructor; copy assignment; move constructor; move assignment. нужно осознанно рассмотреть все пять.

Это не требование синтаксиса C++.

12.14 Rule of Zero по-прежнему лучше

Современный class часто не должен писать ни одну из пяти operations вручную. Если members сами корректно управляют lifetime, compiler-generated special members могут быть правильным решением.

Например, class с std::string и std::vector обычно получает правильную copy/move semantics из своих members.

Class с unique_ptr автоматически не становится copyable, но при подходящих условиях может получить move operations. Не добавляйте ручной move constructor только ради демонстрации синтаксиса в production code.

12.15 Почему noexcept важен для move

Move operation resource-owning type часто может просто передать handles и не требует операций, способных бросить exception. Тогда её можно объявить noexcept.

Standard containers при перераспределении storage должны соблюдать свои exception-safety contracts. Для copyable type они могут предпочесть copy вместо potentially-throwing move, если это помогает сохранить guarantee.

Не превращаем это в миф

Нельзя утверждать «vector всегда вызывает move при reallocation». Выбор зависит от properties element type и требований операции. Поэтому noexcept move важен не только как документация, но и как информация, которую library algorithm может использовать.

12.16 Copy elision и RVO

Move semantics не означает, что каждый return object проходит через move constructor.

В C++17:

```
1 Packet make_packet() {  
2     return Packet{"audit"};  
3 }  
4  
5 Packet packet = make_packet();
```

в соответствующем prvalue case result строится непосредственно как destination; отдельный copy/move constructor для промежуточного temporary не требуется.

NRVO для named local:

```
1 Packet make_packet() {
```

```

2   Packet result{"audit"};
3   return result;
4 }

```

— разрешённая optimization, но не тем же самым обязательным случаем.

Не писать `return std::move(local)` автоматически

Принудительный `std::move` named `local` в `return` может помешать NRVO. Обычный `return local`; чаще выражает намерение правильное: language rules уже умеют использовать copy elision/move там, где это предусмотрено.

12.17 Move-only types как API contract

`unique_ptr` — классический move-only type. Собственный object также может быть move-only:

```

1 class Packet {
2 public:
3     Packet(const Packet&) = delete;
4     Packet& operator=(const Packet&) = delete;
5
6     Packet(Packet&&) noexcept = default;
7     Packet& operator=(Packet&&) noexcept = default;
8 };

```

Это сообщает: два независимых copies не имеют смысла, зато ownership/state можно передать.

В эксперименте `move_ownership_pipeline` этот contract проверяется реальными compile-failure cases.

12.18 Forwarding reference

После templates из блока 09 можно понять ещё один случай T&. Если T — deduced template parameter в форме:

```

1 template <typename T>
2 void relay(T&& value);

```

то это **forwarding reference**. Она способна принять и lvalue, и rvalue. При lvalue-вызове deduction и reference-collapsing rules сохраняют информацию, что source был lvalue.

Не каждый && — forwarding reference: `Packet&&` без deduction — обычная rvalue reference, как и `const T&&` в template.

12.19 Reference collapsing

Для forwarding достаточно запомнить таблицу:

Комбинация	Результат
T& &	T&
T& &&	T&
T&& &	T&
T&& &&	T&&

Практический смысл: наличие lvalue-reference на любом этапе «побеждает», поэтому forwarding template может сохранить исходную category.

12.20 std::forward

Named parameter value внутри relay всё равно lvalue-expression. Простая передача потеряет rvalue category:

```
1 template <typename T>
2 void bad_relay(T&& value) {
3     process(value); // всегда named expression => lvalue
4 }
```

std::forward<T> восстанавливает category, которую template получил от caller:

```
1 template <typename T>
2 void relay(T&& value) {
3     process(std::forward<T>(value));
4 }
```

Это основа **perfect forwarding**: intermediate generic layer не должен без причины превращать rvalue в lvalue или наоборот.

Move и forward решают разные задачи

std::move(x) безусловно выражает «дальше x можно рассматривать как источник для move». std::forward<T>(x) условно сохраняет category, deduced от caller. Не заменяйте один инструмент другим механически.

12.21 Move как cast: мост ко второй половине лекции

Теперь видно, почему тема casts естественно следует за move. std::move не выполняет resource transfer сам: его роль близка к явному преобразованию value category. Реальный transfer делает выбранная special member function.

В C++ есть и другие случаи, когда programmer явно говорит compiler: «эту conversion я намеренно разрешаю». Для разных смыслов язык предоставляет разные casts.

12.22 Implicit conversion или явный cast?

Implicit conversion хороша, когда преобразование естественно и contract очевиден:

```
1 double value = 42; // int -> double
```

Если conversion может терять информацию или её смысл важен для читателя, явность полезна:

```
1 double value = 7.9;
2 int truncated = static_cast<int>(value);
```

Cast не делает conversion автоматически безопасной. Он лишь явно фиксирует намерение применить конкретные conversion rules.

12.23 static_cast

`static_cast<T>(expression)` выполняет набор conversions, которые compiler может проверить на основе static types.

Обычные применения: числовое преобразование, которое хотим отметить явно; conversion enum к integer и обратно при понимании диапазонов; некоторые conversions внутри class hierarchy, если correctness уже доказана другой логикой; получение обратно typed pointer из подходящего void*.

```
1 double ratio = 7.9;
2 int whole = static_cast<int>(ratio);
3
4 Severity severity = Severity::warning;
5 int code = static_cast<int>(severity);
```

Явный cast не проверяет диапазон за вас

Например, floating-to-integer conversion имеет собственные ограничения representability. `static_cast<int>(x)` не означает «безопасно зажать x в диапазон int». Range policy нужно проектировать отдельно.

12.24 Static downcast и его предел

Если Derived наследуется от Base, compiler может разрешить определённый `static_cast<Derived*>` из base pointer. Но такой cast не проверяет dynamic type object во время выполнения.

Поэтому использовать static downcast можно только когда program logic уже гарантирует concrete type. Для runtime-проверяемого polymorphic downcast нужен dynamic_cast.

12.25 dynamic_cast

`dynamic_cast` работает с polymorphic hierarchy и использует runtime type information для проверяемого downcast/cross-cast.

Pointer form:

```
1 Report* base = find_report();
2
3 if (auto* lecture = dynamic_cast<LectureReport*>(base)) {
4     lecture->print_lines();
5 }
```

Если object не подходит target type, pointer result равен nullptr.

Reference form:

```
1 try {
2     LectureReport& lecture = dynamic_cast<LectureReport&>(report);
3 } catch (const std::bad_cast&) {
4     // dynamic type не подходит
5 }
```

При неудаче reference cast бросает std::bad_cast.

12.26 Когда base должен быть polymorphic

Для runtime downcast source type должен удовлетворять требованиям dynamic_cast; обычный случай — polymorphic base, имеющий хотя бы одну virtual function.

Эксперимент содержит real compile-failure, где programmer пытается выполнить dynamic downcast из non-polymorphic base.

12.27 Нужен ли cast для обычного upcast?

Нет. Derived pointer/reference обычно неявно преобразуется к доступному base pointer/reference:

```
1 LectureReport lecture;
2 Report& report = lecture; // нормальный upcast
```

Добавлять dynamic_cast<Report&>(lecture) только ради «явности» не нужно: cast должен решать проблему, а не украшать код.

12.28 const_cast

const_cast меняет cv-qualification там, где language rules это разрешают. Главный учебный случай — различить: 1) view const, но underlying object на самом деле mutable; 2) underlying object изначально создан как const.

Первый случай:

```
1 int counter = 5;
2 const int* view = &counter;
3
4 int* mutable_view = const_cast<int*>(view);
5 *mutable_view = 7; // underlying object counter не const
```

Второй:

```
1 const int limit = 10;
2 int* p = const_cast<int*>(&limit);
3 // *p = 11; // Undefined behavior: настоящий object const
```


Cast не отменяет constness самого object

`const_cast` может дать type без `const`, но если original object действительно `const`, попытка изменить его через такой access приводит к undefined behavior.

Практически `const_cast` часто появляется при interop со старым API. В нормальном собственном interface лучше сразу правильно описать const contract.

12.29 reinterpret_cast

`reinterpret_cast` предназначен для низкоуровневых conversions, где programmer явно работает с representation/pointer mapping.

Пример эксперимента делает только контролируемый pointer-integer round trip:

```
1 int value = 42;
2 int* pointer = &value;
3
4 std::uintptr_t bits =
5     reinterpret_cast<std::uintptr_t>(pointer);
6 int* restored =
7     reinterpret_cast<int*>(bits);
```

На implementation, предоставляющей `std::uintptr_t`, этот integer type способен хранить преобразованное object pointer representation. Mapping в integer implementation-defined; превращать число в переносимый «адрес в формате стандарта» нельзя.

reinterpret не создаёт object другого типа

Получить pointer другого type и сразу разыменовать его — совсем другой вопрос: вступают в силу lifetime и aliasing rules. `reinterpret_cast` не делает такой доступ автоматически корректным.

12.30 C-style cast скрывает намерение

Запись:

```
1 Target value = (Target)source;
```

может использовать несколько разных C++ conversion mechanisms. Читателю сложнее понять, что именно programmer хотел разрешить, а поиск опасных low-level conversions становится труднее.

Например, C-style pointer cast способен одновременно убрать `const` qualification в коде, который не прошёл бы как обычная implicit conversion.

Поэтому production-style курса: сначала попытаться обойтись без cast; если cast действительно нужен — выбрать самый узкий named C++ cast; C-style casts оставлять для чтения legacy/C-oriented code, а не как основной стиль C++.

12.31 Четыре casts рядом

Cast	Главный смысл	Проверка/риск
<code>static_cast</code>	compile-time known conversion	не доказывает runtime dynamic type
<code>dynamic_cast</code>	checked polymorphic cast	runtime check; pointer null/reference exception
<code>const_cast</code>	изменить cv-qualification	mutation настоящего const object может быть UB
<code>reinterpret_cast</code>	low-level reinterpretation/mapping	lifetime, aliasing, portability

12.32 Cast не должен маскировать design problem

Если code постоянно делает `dynamic_cast` по длинной цепочке concrete types, возможно, common virtual interface спроектирован слишком слабо.

Если code постоянно использует `const_cast`, возможно, constness API описана неверно.

Если application logic построена на `reinterpret_cast`, нужно проверить, не скрывается ли за этим неподходящее representation или нарушение ownership.

Cast — точечный инструмент, не замена хорошему type design.

12.33 Эксперимент 1: move_ownership_pipeline

Каталог: `examples/12_move_semantics_and_casts/01_move_ownership_pipeline`.

Move-only Packet владеет payload через `unique_ptr`. Lab проверяет: 1) copy construction и copy assignment запрещены; 2) move construction передаёт resource и оставляет source empty по contract именно этого Packet; 3) move assignment делает то же для уже существующего destination; 4) named variable выбирает lvalue overload, temporary и `std::move` — rvalue overload; 5) named rvalue-reference variable снова lvalue-expression; 6) forwarding function с `std::forward` сохраняет category; 7) prvalue return в C++17 не требует промежуточного move в проверяемом case.

```
cd examples/12_move_semantics_and_casts/01_move_ownership_pipeline
./run_checks.sh
```

Два negative compile cases подтверждают отдельно: copy move-only Packet запрещён, а const `unique_ptr` нельзя передать обычным move construction, потому что его resource нельзя забрать из const owner.

12.34 Эксперимент 2: casts_lab

Каталог: `examples/12_move_semantics_and_casts/02_casts_lab`.

Positive executable проверяет: явную numeric conversion через `static_cast`; успешный и неуспешный pointer `dynamic_cast`; exception от reference `dynamic_cast`; безопасную mutation через `const_cast`, когда original object действительно non-const; pointer-integer round trip через `reinterpret_cast` на текущей implementation.

Два negative compile cases проверяют const access и попытку runtime downcast из non-polymorphic base. Ещё один compile-only case показывает, что C-style cast способен убрать const в source code — программа его только компилирует и *не выполняет опасную запись*.

```
cd examples/12_move_semantics_and_casts/02_casts_lab
./run_checks.sh
```

12.35 Стоимость

Move часто превращает глубокое копирование resource в перенос нескольких handles/pointers, но это свойство конкретного type. Для small trivially copied value move может не давать выигрыша вообще.

static_cast и const_cast как языковые conversions сами по себе не означают обязательный runtime dispatch. dynamic_cast для down/cross cast требует runtime type check и может иметь ненулевую стоимость. reinterpret_cast тоже не «дорогая функция», но его семантическая цена — portability и риск нарушить низкоуровневые правила.

Сначала корректная semantics

Главный вопрос блока не «что быстрее: copy или move?» и не «какой cast быстрее?». Сначала нужен корректный ownership/type contract. Измерение имеет смысл только после того, как обе сравниваемые реализации решают одну задачу корректно. Поэтому отдельный benchmark в блоке не нужен.

12.36 Итоговая схема API

Передача аргумента: const T& — borrow read-only object; T& — borrow mutable object; T by value — функция получает своё value; caller может copy или move в него; T&& — конкретный API принимает rvalue и может переиспользовать состояние; T&& с deduced template parameter — forwarding reference, если выполнены соответствующие условия.

Явная conversion: сначала проверить, нельзя ли написать interface без cast; static_cast — обычная compile-time conversion; dynamic_cast — runtime-checked polymorphic conversion; const_cast — узкая работа с cv-qualification; reinterpret_cast — низкоуровневая boundary.

Главный вывод

std::move не перемещает object: он разрешает рассматривать expression как источник для move. Реальное изменение состояния выполняет выбранная operation типа. Точно так же named C++ casts не «делают всё безопасным», а выражают конкретное намерение conversion и позволяют отдельно рассуждать о его preconditions.

В следующем блоке мы разберём type deduction, auto, decltype, structured bindings, CTAD, lambdas, std::function и std::optional. Они используют уже построенную здесь модель expressions, types и value categories.

13 Лекция 13. Вывод типов, лямбды и вызываемые объекты

В предыдущих блоках мы всё чаще встречали длинные типы: std::unique_ptr<Product>, iterator types, template arguments и callable objects. Современный C++ позволяет не дубли-

ровать тип там, где compiler уже может его вывести. Но `auto` не означает «динамический тип», а `lambda` не «анонимная функция» в смысле обычного `function pointer`.

В этой лекции мы строим прикладную линию:

`type deduction → lambda → callable API → std::function → std::optional.`

Итоговый design-вопрос звучит так: как написать API, который принимает поведение и возвращает отсутствие результата без `magic values`, оставаясь понятным по типам?

Сквозная идея

Вывод типа убирает повторение, но не отменяет `type system`. `Lambda` создаёт `object` уникального `closure type`. `std::function` стирает конкретный `callable type` ради единого `runtime-интерфейса`. `std::optional<T>` выражает «либо `T`, либо отсутствует значение».

13.1 `auto`: compiler выводит тип из `initializer`

Простейший пример:

```
1 auto count = 42;
2 auto ratio = 0.5;
3 auto name = std::string{"C++"};
```

Здесь `types` соответственно выводятся из `initializer`. После `deduction variable` имеет обычный статический C++ `type`. Никакой `runtime-перемены типа` не появляется.

Зачем использовать `auto`, если тип и так известен?

Главная польза не в экономии символов, а в устранении дублирования. Если `expression` уже точно задаёт тип, повторять его слева иногда только создаёт место для рассинхронизации.

Например:

```
1 std::vector<std::string> names{"Ada", "Bjarne"};
2 auto it = names.begin();
```

Тип `iterator` может быть длинным и зависит от `container`. `auto` сохраняет важную информацию: `it` имеет именно тот `type`, который возвращает `begin()`.

13.2 Что `auto` делает с `references` и `const`

Обычный `auto deduction` во многом похож на `template type deduction`.

```
1 const int value = 10;
2 const int& ref = value;
3
4 auto a = ref;           // int
5 const auto b = ref;     // const int
6 const auto& c = ref;    // const int&
```

Top-level `const/reference` у `initializer` не обязательно сохраняются простым `auto`. Если `semantics` требует `reference`, нужно написать `auto&` или `const auto&` явно.

Auto не должен скрывать ownership или lifetime

Если функция возвращает reference, запись `auto x = f();` может создать копию вместо alias. Если нужен borrow, пишите `auto&` или `const auto&` и проверяйте lifetime.

13.3 auto&, const auto&, auto&&

Практическая таблица:

Форма	Идея	Обычный смысл
<code>auto x</code>	новое value	copy/move result
<code>auto& x</code>	mutable borrow	изменить существующий object
<code>const auto& x</code>	read-only borrow	избежать copy
<code>auto&& x</code>	deduction с value category	generic/forwarding context

После блока 12 последняя строка уже не выглядит магией: `auto&&` следует deduction и reference-collapsing rules.

13.4 decltype: type expression без вычисления

`decltype(expression)` позволяет получить type по правилам языка, не выполняя expression ради side effects.

```

1 int value = 7;
2 decltype(value) copy = 9; // int
3 decltype((value)) alias = value; // int&

```

Разница двойных скобок принципиальна. Для unparenthesized id-expression `decltype(name)` обычно даёт declared type entity. Для общего expression результат зависит от value category: lvalue expression $\rightarrow T\&$; xvalue expression $\rightarrow T\&\&$; prvalue expression $\rightarrow T$.

Скобки вокруг имени меняют результат

`decltype(x)` и `decltype((x))` могут быть разными types. Вторая запись рассматривает `(x)` как обычное expression и учитывает его value category.

13.5 decltype(auto)

Return type `decltype(auto)` применяет правила `decltype` к return expression:

```

1 decltype(auto) at(std::vector<int>& values, std::size_t i) {
2     return (values[i]);
3 }

```

Скобки здесь намеренно сохраняют reference. Это мощный, но опасный инструмент: если вернуть reference на object с закончившимся lifetime, deduction не спасёт.

Для простого API явный return type часто читабельнее.

13.6 Trailing return type

Иногда return type зависит от parameters:

```
1 template <typename A, typename B>
2 auto add(const A& a, const B& b) -> decltype(a + b) {
3     return a + b;
4 }
```

В C++14+ многие функции могут просто использовать deduced auto return type, но trailing form остаётся полезной, когда return type нужно явно выразить через parameter expressions.

13.7 Structured bindings

C++17 позволяет разложить tuple-like object или подходящую structure:

```
1 std::map<std::string, int> scores{
2     {"templates", 9},
3     {"move", 10}
4 };
5
6 for (const auto& [topic, score] : scores) {
7     std::cout << topic << ": " << score << '\n';
8 }
```

Имена topic и score связываются с компонентами элемента. Важно выбрать форму binding: auto [a,b] и auto& [a,b] имеют разные copy/reference semantics.

Не забывайте & в range-for

Если нужно изменять members элемента container, простое auto [key, value] может работать с отдельным binding object/copy, а auto& [key, value] выражает работу с самим элементом там, где это разрешено типом.

13.8 CTAD: class template argument deduction

До C++17 template arguments class часто писали явно:

```
1 std::pair<std::string, int> item{"templates", 9};
```

В C++17:

```
1 std::pair item{std::string{"templates"}, 9};
```

Compiler использует deduction guides и constructors, чтобы вывести template arguments. Это называется **CTAD**.

CTAD не делает class template «нетипизированным»

После deduction объект всё равно имеет конкретный specialization, например std::pair<std::string, int>. Deduction происходит на этапе компиляции.

Для user-defined class templates compiler может использовать implicit deduction guides, а programmer при необходимости может объявлять свои. В этом блоке достаточно понимать сам механизм; сложные guides оставим библиотечному design.

13.9 Callable object: общий термин

В C++ «что-то можно вызвать» не означает только обычную функцию. Callable может быть: function; function pointer; class object с operator(); lambda closure object; wrapper вроде std::function.

Templates особенно хорошо работают с callables, потому что конкретный type известен compiler.

13.10 Function object

До lambda поведение часто упаковывали в class:

```
1 struct Threshold {
2     int limit;
3
4     bool operator()(int value) const {
5         return value >= limit;
6     }
7 };
```

Это обычный object со state и member function operator(). Lambda делает такой локальный pattern намного компактнее.

13.11 Минимальная lambda

```
1 auto is_even = [](int value) {
2     return value % 2 == 0;
3 };
4
5 bool result = is_even(42);
```

Синтаксис: 1) [] — capture list; 2) (int value) — parameters; 3) optional specifiers; 4) body в фигурных скобках.

13.12 Что создаёт compiler

Каждая lambda expression создаёт prvalue уникального unnamed **closure type**. Концептуально можно представить:

```
1 class SomeUniqueClosure {
2 public:
3     bool operator()(int value) const {
4         return value % 2 == 0;
5     }
6 };
```

Это не буквальный required layout и не имя, которым можно воспользоваться. Модель нужна, чтобы понять: lambda variable — object, его можно копировать/перемещать согласно свойствам closure type, а вызов похож на вызов operator().

13.13 Capture by value

```
1 int threshold = 10;
2
3 auto enough = [threshold](int value) {
4     return value >= threshold;
5 };
```

Closure хранит захваченное значение. Дальнейшее изменение внешнего `threshold` не обязано менять сохранённую `copy`.

Value capture особенно удобен, когда callback должен пережить локальную `variable`, а захваченный object сам безопасно копируется.

13.14 Capture by reference

```
1 int accepted = 0;
2
3 auto count = [&accepted](int value) {
4     if (value > 0) {
5         ++accepted;
6     }
7 };
```

Closure хранит возможность обратиться к внешнему object, но не продлевает его `lifetime`.

Reference capture — borrowing contract

Если lambda сохраняется дольше захваченной `local variable`, последующий вызов может обратиться к object после окончания `lifetime`. Это тот же класс проблемы, что `dangling reference/pointer`.

Поэтому callback registry, который хранит lambda, требует особенно внимательно читать capture list.

13.15 Default capture

[=] задаёт default capture by value, [&] — by reference.

Коротко, но большие default captures могут скрывать реальные зависимости. В callback, который живёт долго, explicit capture list часто лучше:

```
1 auto callback = [prefix, &counter](const std::string& value) {
2     ++counter;
3     std::cout << prefix << value << '\n';
4 };
```

По коду видно, что именно closure хранит/заимствует.

13.16 mutable lambda

По умолчанию call operator non-mutable lambda, которая захватывает by value, const относительно сохранённого state.

```
1 auto next = [counter = 0]() mutable {  
2     return ++counter;  
3 };
```

mutable разрешает изменять сопу внутри closure. Внешний object, если он вообще существовал отдельно, этим не изменяется.

13.17 Init-capture

C++14 добавил возможность создать capture прямо в списке:

```
1 auto callback =  
2     [name = std::string{"audit"}]() {  
3     return name.size();  
4 };
```

Это удобно для rename, conversion и move-capture:

```
1 auto owner = std::make_unique<Job>();  
2  
3 auto task =  
4     [job = std::move(owner)]() {  
5     job->run();  
6 };
```

После такого capture closure может стать move-only, потому что содержит unique_ptr. Это важно для std::function в C++17: он требует CopyConstructible target.

13.18 Generic lambda

C++14 разрешает auto в lambda parameters:

```
1 auto greater = [](const auto& a, const auto& b) {  
2     return a > b;  
3 };
```

Концептуально call operator closure — function template. Один closure object может работать с несколькими types, если body корректен для каждого instantiation.

Это естественное продолжение templates из блока 9.

13.19 Lambda return type

Часто return type выводится:

```
1 auto square = [](int x) {
```

```
2     return x * x;  
3 };
```

Если нужна явность:

```
1 auto parse = [](const std::string& text) -> std::optional<int> {  
2     // ...  
3 };
```

Return expressions должны удовлетворять правилам deduction. Если branches возвращают несовместимые types, compiler сообщит ошибку.

13.20 Lambdas и algorithms

Следующий блок подробно разберёт STL algorithms. Уже сейчас видно главное применение:

```
1 auto it = std::find_if(  
2     blocks.begin(),  
3     blocks.end(),  
4     [](const CourseBlock& block) {  
5         return block.ready;  
6     }  
7 );
```

Algorithm отвечает за traversal, lambda — только за policy/condition. Это пример разделения механизма и поведения без class hierarchy.

13.21 Когда template callable лучше

Generic function:

```
1 template <typename Predicate>  
2 bool any_match(const std::vector<CourseBlock>& blocks,  
3               Predicate predicate);
```

получает concrete callable type в каждом instantiation. Compiler может inline call и оптимизировать через границу callback.

Плюсы: нет обязательного type-erasure wrapper; callable может быть move-only; concrete type доступен compiler.

Минусы: implementation обычно должна быть видна в header; разные callable types создают разные instantiations; heterogeneous callbacks нельзя просто хранить в одном `std::vector<Predicate>`.

13.22 std::function: единый runtime callable type

`std::function<R(Args...)>` из `<functional>` может хранить разные callable objects с совместимой сигнатурой:

```
1 std::function<void(const Event&)> callback;
```

```
2
3 callback = free_function;
4 callback = SomeFunctor{};
5 callback = [prefix](const Event& event) {
6     std::cout << prefix << event.name;
7 };
```

Внешний type один и тот же, хотя внутренний callable type различается. Это **type erasure**.

13.23 Зачем нужна type erasure

Представим registry:

```
1 using Callback = std::function<void(const Event&)>;
2
3 std::vector<Callback> callbacks;
```

Теперь callbacks можно выбирать во время выполнения и хранить в одном container. Template parameter одного compile-time call site этого сам по себе не даёт.

Именно runtime heterogeneity — основное основание для `std::function`, а не желание «завернуть любую lambda».

13.24 Стоимость std::function

Стандарт задаёт semantics, но не требует конкретный layout или конкретную small-buffer optimization.

Практически wrapper может вносить: indirect call; дополнительное состояние для type erasure; иногда dynamic allocation для target; невозможность некоторых optimizations по сравнению с полностью известным concrete callable type.

Не говорить «std::function всегда выделяет память»

Реализации часто используют small-object optimization, но размер буфера и факт allocation для конкретного closure не являются универсальным обещанием, на которое должен опираться переносимый код.

Если callback только немедленно вызывается, template parameter часто проще и дешевле. Если callback нужно хранить heterogeneously, `std::function` может быть именно тем abstraction, который нужен.

13.25 Пустой std::function

Wrapper может не содержать callable:

```
1 std::function<void()> callback;
2
3 if (callback) {
4     callback();
5 }
```

Вызов пустого `std::function` бросает `std::bad_function_call`. Обычно `registry` проверяет наличие или гарантирует `invariant`, что `slot` заполнен.

13.26 Copyability target в C++17

Для конструкции `std::function` из callable target в C++17 target должен удовлетворять требованиям `copyability`.

Поэтому:

```
1 auto owner = std::make_unique<Job>();
2
3 auto move_only =
4     [job = std::move(owner)]() {
5         job->run();
6     };
7
8 // std::function<void()> f = std::move(move_only);
9 // для move-only closure это не подходит в C++17
```

Эксперимент проверяет это настоящей `compile failure`.

13.27 std::optional<T>: отсутствие как часть типа

До C++17 функции часто кодировали «не найдено» через: magic value -1; пустую строку; nullable pointer; отдельный `bool` + output parameter.

Если функция концептуально возвращает value `T`, но value может отсутствовать, `std::optional<T>` делает это явным:

```
1 std::optional<CourseBlock> find_block(int number);
```

Result либо содержит `CourseBlock`, либо находится в `disengaged state`.

13.28 Создание optional

```
1 std::optional<int> a = 42;
2 std::optional<int> b = std::nullopt;
3 auto c = std::make_optional<std::string>("ready");
```

Для функции удобно:

```
1 std::optional<int> find_score(const std::string& topic) {
2     if (topic == "templates") {
3         return 9;
4     }
5
6     return std::nullopt;
7 }
```

13.29 Проверка optional

```
1 if (auto score = find_score("templates")) {  
2     std::cout << *score;  
3 }
```

Можно использовать: contextual bool conversion; `has_value()`; `operator*` / `operator->` после доказанного наличия; `value()`; `value_or(default)`.

`value()` у пустого `optional` бросает `std::bad_optional_access`. Разыменовывать пустой `optional` через `operator*` нельзя.

13.30 Optional не означает error details

`optional<T>` отвечает только на вопрос «есть T или нет». Он не сообщает, *почему* значения нет.

Если caller должен различать `invalid input`, `I/O failure` и `not-found`, нужны другие `error contracts`: `status object`, `exception` или более богатый `result type`.

Это связывает тему с блоком 10: `abstraction` выбирается по информации, которую обязан получить caller.

13.31 Optional и pointers

Если функция возвращает `non-owning view` на существующий `object`, `nullable pointer` может быть естественнее:

```
1 const CourseBlock* find_block_view(int number);
```

Если функция возвращает самостоятельное `value`, которое может отсутствовать:

```
1 std::optional<CourseBlock> find_block_copy(int number);
```

Выбор зависит не от моды, а от `ownership/lifetime contract`.

13.32 Optional reference отсутствует

В C++17 стандартный `std::optional<T&>` не поддерживается. Для `optional borrow` обычно используют `pointer` или `std::reference_wrapper<T>` внутри `optional`, если это действительно улучшает `design`.

Не пытайтесь превращать `optional` в универсальный контейнер «для всего».

13.33 Прикладной query API

Соберём темы вместе:

```
1 template <typename Predicate>  
2 std::optional<CourseBlock>  
3 find_first(const std::vector<CourseBlock>& blocks,  
4           Predicate predicate) {
```

```
5     for (const auto& block : blocks) {
6         if (predicate(block)) {
7             return block;
8         }
9     }
10
11     return std::nullopt;
12 }
```

Caller:

```
1  const int min_examples = 2;
2
3  auto result = find_first(
4      blocks,
5      [min_examples](const CourseBlock& block) {
6          return block.examples >= min_examples;
7      }
8  );
9
10 if (result) {
11     std::cout << result->slug;
12 }
```

Здесь: template выводит concrete predicate type; lambda хранит threshold by value; const auto& внутри query не копирует элементы; optional<CourseBlock> явно выражает отсутствие; caller не использует magic index -1.

13.34 Structured binding в прикладном API

Если функция возвращает несколько связанных значений:

```
1  auto summary = std::pair{ready_count, total_count};
2  const auto [ready, total] = summary;
```

STAD выводит template arguments pair, structured binding даёт имена двум components. Но если у пары есть domain meaning, отдельный named struct может быть читабельнее.

Современный синтаксис не отменяет моделирование

auto, STAD и structured bindings уменьшают шум. Они не должны превращать программу в набор безымянных tuple/pair, если предметная область заслуживает собственного типа.

13.35 Callback registry как runtime boundary

Registry хранит разные callbacks:

```
1  class Registry {
2  public:
3      using Callback = std::function<void(const Event&)>;
4
5      void subscribe(Callback callback) {
```

```
6     callbacks_.push_back(std::move(callback));
7 }
8
9 void publish(const Event& event) const {
10     for (const auto& callback : callbacks_) {
11         callback(event);
12     }
13 }
14
15 private:
16     std::vector<Callback> callbacks_;
17 };
```

Здесь `std::function` оправдан: `callbacks` имеют разные closure types, создаются в разных местах и должны храниться вместе после завершения `subscribe`.

13.36 Capture lifetime в registry

Опасный вариант:

```
1 Registry registry;
2
3 {
4     int local = 0;
5     registry.subscribe([&local](const Event&) {
6         ++local;
7     });
8 } // local уничтожен
9
10 // registry.publish(...); // dangling reference внутри closure
```

`std::function` корректно владеет closure object, но closure object может сам содержать `dangling reference`. Type erasure не исправляет `capture lifetime`.

Безопасные варианты: `capture independent state by value`; захватывать `shared_ptr`, если `callback` действительно должен продлевать `shared lifetime`; использовать `weak_ptr`, если `callback` только наблюдает `owner`; гарантировать внешний `lifetime reference capture` явным `design contract`.

13.37 Mutable state внутри callback

`Init-capture` позволяет `callback` иметь собственное состояние:

```
1 registry.subscribe(
2     [calls = 0](const Event&) mutable {
3         ++calls;
4         std::cout << calls << '\n';
5     }
6 );
```

Каждая `copy closure` получает собственную `copy state`. Если `wrapper/registry` копирует `callback`, это может иметь значение. Нельзя автоматически считать, что `mutable lambda` —

«один глобальный счётчик».

13.38 Lambda и this

В member function lambda может обращаться к object через capture this. Но если callback переживёт object, raw this станет dangling.

Для long-lived callback ownership model нужно проектировать явно. Например, shared-owned object иногда передаёт weak_ptr в callback и перед использованием вызывает lock().

Так блоки 11 и 13 соединяются напрямую.

13.39 Когда function pointer всё ещё достаточен

Captureless lambda может преобразовываться к совместимому function pointer. Обычный function pointer остаётся хорошим инструментом для C API или API, где state не нужен.

std::function не нужно использовать автоматически в каждом месте, где параметр можно вызвать.

13.40 Generic lambda или std::function?

Сравнение:

Свойство	Template/generic callable	std::function
Concrete type известен compiler	да	скрыт type erasure
Heterogeneous storage	не напрямую	да
Move-only callable в C++17	да, если API поддерживает move	target для std::function должен быть copyable
Inlining opportunities	обычно лучше видны compiler	indirect boundary может мешать
Stable runtime wrapper type	нет одного общего type	да

13.41 Эксперимент 1: course_query_pipeline

Каталог: examples/13_type_deduction_lambdas_and_function/01_course_query_pipeline.

Эксперимент строит маленький query API для учебных blocks: 1) auto, decltype и static_assert проверяют реальные deduced types; 2) structured bindings разбирают summary; 3) CTAD создаёт std::pair без явных template arguments; 4) generic lambda работает с несколькими numeric types; 5) predicate lambda фильтрует blocks; 6) std::optional<CourseBlock> представляет found/not-found без magic index.

Отдельный negative compile file действительно использует local variable в lambda без capture и должен быть отвергнут compiler.

```
cd examples/13_type_deduction_lambdas_and_function/01_course_query_pipeline
./run_checks.sh
```


13.42 Эксперимент 2: callback_registry_lab

Каталог: `examples/13_type_deduction_lambdas_and_function/02_callback_registry_lab`.

Registry хранит `std::function<void(const Event&)>` и принимает: free function; function object; lambda с value capture; mutable lambda с собственным state; lambda, наблюдающую shared-owned object через `weak_ptr`.

Проверяется также `empty std::function` и `std::bad_function_call`. Negative compile case показывает ограничение C++17: move-only closure с `unique_ptr` нельзя положить в `std::function`.

```
cd examples/13_type_deduction_lambdas_and_function/02_callback_registry_lab
./run_checks.sh
```

13.43 Стоимость и границы abstraction

`auto`, `decltype`, structured bindings и CTAD в основном влияют на compile-time type deduction и сами по себе не добавляют отдельный runtime object «для вывода типа».

Lambda closure — настоящий object. Его size зависит от captures и implementation details/alignment. Captureless closure не обязана иметь size zero.

Template callable часто позволяет compiler видеть concrete call path. `std::function` покупает runtime uniformity ценой type-erasure layer и возможного allocation/indirect call.

`optional<T>` обычно должен хранить состояние engaged/disengaged плюс место для T, но конкретный layout не гарантируется как простая формула `sizeof(T)+1`.

Не угадываем стоимость по синтаксису

Короткая lambda не обязательно «быстрее функции», а `std::function` не обязательно делает allocation. Если callback path важен для performance, измеряют реальную программу и конкретные target/compiler settings.

13.44 Итоговый checklist

Когда видите длинный type: если initializer однозначно задаёт type — рассмотрите `auto`; если важны reference/const semantics — напишите их явно; если нужен type expression — `decltype`; для tuple-like decomposition — structured bindings; CTAD используйте, когда deduction делает type очевиднее, а не загадочнее.

Когда нужно передать поведение: local behavior — lambda; generic immediate call — template callable; runtime heterogeneous storage — `std::function`; capture lifetime проверяется отдельно от wrapper.

Когда результат может отсутствовать: standalone value — `std::optional<T>`; non-owning optional view — часто pointer; если нужны причины ошибки — более богатый error contract.

Главный вывод

Современный C++ позволяет писать меньше типов вручную, но правильный design всё равно строится вокруг точных contracts. `auto` не отменяет type, lambda не отменяет lifetime, `std::function` не отменяет стоимость runtime abstraction, а `optional` не отменяет проектирование ошибок.

Следующий блок переходит к STL как к единой системе containers, iterators и algorithms. Там callable objects этой лекции станут обычными predicates, comparators и operations для standard algorithms.

14 Лекция 14. STL: контейнеры, итераторы и алгоритмы

До этого мы изучали отдельные части standard library: `std::vector`, `std::map`, smart pointers, optional, lambdas и templates. Теперь объединим их в главную архитектурную идею STL:

container → iterator/range → algorithm → callable policy.

В прикладной части будем строить pipeline обработки каталога учебных блоков: хранение записей, индекс, поиск, фильтрация, сортировка, преобразование, агрегация и удаление. Одновременно разберём complexity guarantees и invalidation, потому что выбор container определяется не только удобством синтаксиса.

Идея STL

Container отвечает за хранение и lifetime элементов. Iterator описывает позицию и способ пройти range. Algorithm работает с range, не обязуясь знать конкретный container. Lambda/function object задаёт переменную часть поведения.

14.1 Что такое STL в этом блоке

Исторически термин STL относился к Standard Template Library, а современная C++ standard library шире. В учебной практике под «STL-стилем» мы будем иметь в виду прежде всего: containers; iterators; algorithms; function objects/predicates; связанные utility abstractions.

Не вся standard library относится к STL, и не нужно смешивать эти понятия как строгие синонимы.

14.2 Почему контейнеров несколько

Если бы единственным критерием было «хранить набор объектов», одного vector хватило бы почти всегда. Но программы предъявляют разные требования: нужен ли contiguous storage; важен ли порядок; нужен ли быстрый lookup по key; часто ли вставляем в середину; нужны ли стабильные references/iterators; сколько allocations приемлемо; важна ли cache locality; нужен ли deterministic ordered traversal.

Выбор container — часть design и cost model.

14.3 `std::array`

`std::array<T, N>` — fixed-size container, размер известен в type.

```
1 std::array<int, 4> values{1, 2, 3, 4};
```

Плюсы: contiguous elements; size не меняется; обычный container API: `begin()`, `end()`, `size()`; хорошо работает с algorithms.

Это удобная modern abstraction над built-in array, когда размер фиксирован.

14.4 std::vector

`vector<T>` — основной general-purpose sequence container: элементы contiguous; быстрый random access; amortized constant-time `push_back`; обычно хорошая locality при sequential traversal; рост может вызвать reallocation.

```
1 std::vector<CourseBlock> blocks;  
2 blocks.push_back({14, "stl", 2});
```

Для большинства динамических последовательностей начинайте с `vector`, если нет конкретного требования, которое ему противоречит.

14.5 Size и capacity

У `vector` различаются: `size()` — число живых элементов; `capacity()` — сколько элементов можно разместить без следующего расширения storage.

```
1 std::vector<int> values;  
2 values.reserve(100);
```

`reserve(100)` не создаёт 100 элементов и не меняет `size()`. Он просит `capacity` не меньше 100.

Что гарантирует reserve

Если requested capacity больше текущей, происходит reallocation. Если нет — reallocation не происходит. После успешного `reserve` вставки не вызывают reallocation, пока `size` не превысит `capacity`.

14.6 Почему reallocation важен

При росте `vector` может получить новый storage и перенести/скопировать элементы. Старый storage освобождается.

Следствия: addresses элементов меняются; pointers/references/iterators на элементы становятся invalid при reallocation; стоимость отдельного роста может быть линейной по числу элементов; последовательность многих `push_back` всё равно имеет amortized constant complexity.

Amortized O(1) не означает каждый push_back O(1)

Иногда `push_back` требует allocation и переноса существующих элементов. Гарантия говорит о средней стоимости последовательности операций в амортизированном анализе.

14.7 std::deque

`deque<T>` — sequence с efficient insertion/removal на обоих концах.

Он не обязан хранить все элементы в одном contiguous block. Random access поддерживается, но locality/layout отличаются от `vector`.

Обычный выбор: часто нужны `push_front/pop_front`; random access всё ещё нужен; contiguous storage не требуется.

Не делайте вывод «deque = vector без reallocation»: правила invalidation и representation другие и должны читаться отдельно.

14.8 std::list

list<T> — doubly linked list.

Полезные свойства: insertion/erase по известному iterator обычно constant complexity; references/iterators к другим элементам обычно сохраняются при insertion/erase; нет random-access iterator; каждый node обычно хранится отдельно; locality при traversal обычно хуже contiguous vector.

O(1) insertion не делает list автоматически быстрее

Чтобы вставить «в середину по номеру», сначала может понадобиться линейно найти позицию. Отдельные allocations и pointer chasing также дают большие constant factors. Asymptotics — только часть стоимости.

14.9 std::forward_list

forward_list<T> — singly linked list. Он экономнее по links и поддерживает операции around a known predecessor, но interface менее удобен.

В обычном application code встречается реже. Выбирать его стоит под конкретное требование, а не потому что «односвязный список проще».

14.10 Ordered associative containers

std::map<Key, T> и std::set<Key> поддерживают ordering по comparator.

Обычные guarantees: lookup/insert/erase — logarithmic complexity; traversal идёт в порядке comparator; insertion обычно не invalidates iterators/references к существующим элементам; erase invalidates только erased elements.

map хранит key-value pairs, set — keys.

14.11 Unordered associative containers

std::unordered_map и std::unordered_set организованы вокруг hashing.

Обычный contract: average-case lookup/insert/erase — constant complexity; worst case может быть linear; iteration order не sorted; rehash влияет на iterators; quality hash/equality важна.

O(1) у unordered_map — average, не абсолютное обещание

Нельзя писать «unordered_map всегда быстрее map». Учитываются размер данных, hash cost, allocations, locality, ordering requirements и worst-case behavior.

14.12 Когда map, когда unordered_map

map подходит, если: нужен sorted traversal; нужны ordered operations вроде lower_bound; важен logarithmic worst-case contract.

unordered_map подходит, если: нужен lookup по exact key; порядок не важен; хороший hash доступен; average constant lookup соответствует задаче.

14.13 Адаптеры контейнеров

`stack`, `queue`, `priority_queue` — container adapters. Они намеренно предоставляют более узкий interface поверх underlying container.

Например, `queue` выражает FIFO semantics. Если application должен доставать произвольный элемент по iterator, возможно, `queue` выбран неверно.

14.14 Iterator: абстракция позиции

Iterator — объект, с помощью которого algorithm обращается к position в range.

```
1 auto it = blocks.begin();
2
3 if (it != blocks.end()) {
4     std::cout << it->slug;
5 }
```

Iterator не обязан быть raw pointer, хотя iterator contiguous container может быть pointer-like.

Не нужно утверждать, что iterator «это указатель внутри STL». Это abstraction с определёнными operations и guarantees.

14.15 Half-open range [first, last)

Большинство classic algorithms принимает пару iterators:

$$[first, last).$$

`first` относится к range, `last` указывает на позицию *после* последнего элемента.

Преимущества: empty range задаётся `first == last`; length естественно выражается distance; соседние ranges стыкуются без специальной обработки границы.

14.16 Iterator categories

Не все iterators поддерживают одинаковые операции.

Классическая иерархия требований: input iterator; output iterator; forward iterator; bidirectional iterator; random-access iterator; в C++20 отдельно формализован contiguous iterator.

`vector` предоставляет random-access iterators. `list` — bidirectional iterators.

Поэтому algorithm `std::sort` требует random-access iterator и не работает с `std::list::iterator`.

14.17 Почему list имеет member sort

```
1 std::list<int> values{4, 1, 3};
2 values.sort();
```

List знает свою node-based structure и может сортировать её без random-access iterator. Попытка передать `list.begin()/end()` в `std::sort` проверяется в практике как real compile failure.

14.18 Const iterator

`const_iterator` не разрешает менять элемент через `iterator`.

```
1 const std::vector<int> values{1, 2, 3};
2 auto it = values.begin(); // iterator to const element
```

Также есть `cbegin()/cend()`, если хочется явно запросить read-only iterator у non-const container.

14.19 Range-based for

```
1 for (const auto& block : blocks) {
2     std::cout << block.slug;
3 }
```

Это удобный syntax поверх `begin/end` pattern, когда алгоритм прост: последовательно пройти range.

Но если задача — «найти», «отсортировать», «преобразовать», «посчитать», `standard algorithm` часто выражает намерение лучше ручного `loop`.

14.20 std::find и std::find_if

```
1 auto it = std::find_if(
2     blocks.begin(),
3     blocks.end(),
4     [](const CourseBlock& block) {
5         return block.number == 14;
6     }
7 );
```

Result — iterator. Если ничего не найдено, он равен `end()`.

Это non-owning position в container, поэтому применяются `lifetime/invalidation rules` container.

14.21 std::count_if

```
1 const auto ready = std::count_if(
2     blocks.begin(),
3     blocks.end(),
4     [](const CourseBlock& block) {
5         return block.complete;
6     }
7 );
```

Algorithm принимает predicate из блока 13. Container при этом может быть другим, если `iterator requirements` выполнены.

14.22 std::copy и output iterators

Algorithms могут писать через output iterator.

```
1 std::vector<CourseBlock> ready;
2
3 std::copy_if(
4     blocks.begin(),
5     blocks.end(),
6     std::back_inserter(ready),
7     [](const CourseBlock& block) {
8         return block.complete;
9     }
10 );
```

std::back_inserter создаёт adapter, который превращает assignment algorithm-а в push_back container.

14.23 std::transform

```
1 std::vector<std::string> slugs;
2
3 std::transform(
4     blocks.begin(),
5     blocks.end(),
6     std::back_inserter(slugs),
7     [](const CourseBlock& block) {
8         return block.slug;
9     }
10 );
```

Mechanism traversal находится в algorithm, conversion policy — в lambda.

14.24 std::accumulate

std::accumulate находится в <numeric>:

```
1 const int examples = std::accumulate(
2     blocks.begin(),
3     blocks.end(),
4     0,
5     [](int total, const CourseBlock& block) {
6         return total + block.examples;
7     }
8 );
```

Тип initial value важен: он влияет на accumulator type.

Например, initial 0 — int; initial 0.0 — double.

14.25 std::sort

```
1 std::sort(  
2     blocks.begin(),  
3     blocks.end(),  
4     [](const CourseBlock& left, const CourseBlock& right) {  
5         return left.number < right.number;  
6     }  
7 );
```

Для `std::sort` complexity — $O(N \log N)$ comparisons в standard guarantee современной C++ library.

Comparator должен задавать корректное strict weak ordering.

14.26 Ошибка comparator

Плохо:

```
1 [](int a, int b) {  
2     return a <= b;  
3 }
```

Для равных значений comparator возвращает true в обе стороны и нарушает требования ordering.

Правильно для ascending order:

```
1 [](int a, int b) {  
2     return a < b;  
3 }
```

Algorithm contracts включают требования не только к iterator, но и к callable.

14.27 Stable sort

`std::stable_sort` сохраняет relative order equivalent elements. Обычный `std::sort` этого не обещает.

Если сначала отсортировать по secondary key, а затем stable-sort по primary key, можно сохранить нужный secondary ordering. Но зачастую яснее сразу написать comparator по tuple ключей.

14.28 lower_bound

Для sorted range:

```
1 auto it = std::lower_bound(  
2     values.begin(),  
3     values.end(),  
4     target  
5 );
```


Algorithm использует ordering и на random-access iterator выполняет logarithmic число comparisons. Но стоимость iterator movement зависит от iterator category.

Для ordered associative container часто разумнее использовать member lower_bound, потому что container может использовать внутреннюю tree structure.

14.29 all_of, any_of, none_of

Эти algorithms напрямую выражают логические queries:

```
1  const bool all_ready = std::all_of(  
2      blocks.begin(),  
3      blocks.end(),  
4      [](const CourseBlock& block) {  
5          return block.complete;  
6      }  
7  );
```

Они могут завершить обход раньше, как только result уже определён.

14.30 Алгоритм не знает container

Это основная сила STL.

```
1  template <typename Iterator, typename Predicate>  
2  std::size_t count_matching(  
3      Iterator first,  
4      Iterator last,  
5      Predicate predicate);
```

Concrete algorithm может работать с vector, list, built-in array или другим range, если iterators удовлетворяют требованиям.

Templates дают static polymorphism по iterator/callable types.

14.31 Complexity — часть API

Для container/algorithm complexity guarantee нужно читать как contract.

Примеры: vector::operator[] — constant; vector::push_back — amortized constant; map::find — logarithmic; unordered_map::find — average constant; find_if — linear в размахе range в worst case; sort — $O(N \log N)$ comparisons.

Но Big-O не описывает cache misses, allocation cost, branch behavior и object size.

14.32 Cache locality

Contiguous vector часто хорошо подходит sequential traversal: соседние элементы расположены рядом в storage.

Node-based list/map обычно требует переходов между nodes, которые могут находиться в разных местах памяти.

Это обычная performance model, а не стандартное обещание конкретного cache поведения CPU. Измерять нужно на реальном workload.

14.33 Iterator invalidation

Invalidation означает, что ранее полученный iterator/reference/pointer больше нельзя использовать в тех operations, для которых он был валиден.

Причина — container изменил storage или удалил соответствующий element.

Ошибки invalidation особенно опасны, потому что source code выглядит невинно:

```
1 auto it = values.begin();
2 values.push_back(42);
3 // *it может быть invalid, если случилась vector reallocation
```

14.34 Vector invalidation при росте

Если vector reallocate: invalid становятся все iterators; references; pointers на elements.

Если reallocation не было, insertion rules всё равно могут invalidировать positions в зависимости от места операции. Например, insertion/erase в середине сдвигают последующие elements.

Всегда читайте contract конкретной operation.

14.35 Как безопасно наблюдать reallocation

В практике мы сохраняем: data() address как число/для сравнения до изменения; старый capacity; затем вызываем push_back; после операции сравниваем старый и новый address; не разыменовываем старый pointer.

Так можно показать факт reallocation без UB.

Отдельный специально небезопасный executable выполняется только под ASan и должен дать diagnostic.

14.36 List stability

Для list insertion обычно не invalidates iterators/references к существующим elements. Erase invalidates только iterator/reference на erased element.

Это сильное отличие от vector, но за него платим node allocations, отсутствием random access и locality.

14.37 Map/set stability

Для ordered associative containers insertion/emplacement не invalidates iterators/references к существующим elements; erase invalidates erased elements.

Unordered containers имеют дополнительную ось — rehash. Rehash invalidates iterators, хотя references/pointers к elements имеют отдельные guarantees.

Не переносите правила одного container на другой.

14.38 Erase y vector

```
1 auto it = values.erase(position);
```

Result указывает на element, следующий за erased, либо end().

После erase erased element уничтожен, а элементы после него сдвигаются. Iterators/references на erased и subsequent elements invalidated.

14.39 Erase-remove idiom в C++17

Classic std::remove_if сам не уменьшает size container.

```
1 auto new_end = std::remove_if(
2     values.begin(),
3     values.end(),
4     [](int value) {
5         return value < 0;
6     }
7 );
8
9 values.erase(new_end, values.end());
```

remove_if переставляет retained elements к началу range и возвращает logical new end. Реальное удаление хвоста делает container::erase.

C++20 добавляет удобные std::erase/std::erase_if для поддерживаемых containers, но основной стандарт блока — C++17.

14.40 Почему remove_if не вызывает erase сам

Generic algorithm знает только iterator range и не обязан иметь доступ к container object или его size-management API.

Пример архитектуры STL: algorithm работает с range, container отвечает за structural mutation.

14.41 Алгоритм или member function

Иногда container предоставляет member function с похожей задачей.

Примеры: list::sort вместо std::sort; map::find использует tree structure; unordered_map::find использует hash table.

Если member может использовать внутреннюю структуру container, он часто точнее выражает задачу и даёт нужную complexity.

14.42 Index рядом с vector

Прикладной каталог удобно хранить в vector<CourseBlock> для последовательной обработки, а дополнительный lookup index строить отдельно:

```
1 std::unordered_map<int, std::size_t> index;
```

```
2
3 for (std::size_t i = 0; i < blocks.size(); ++i) {
4     index.emplace(blocks[i].number, i);
5 }
```

Теперь: vector остаётся source of truth; index хранит positions, а не copies objects; lookup по number может быть average $O(1)$.

Но structural mutation vector может сделать сохранённые indices логически неверными. Index нужно rebuild/update по contract.

14.43 Почему index по position безопаснее pointer только условно

Index `size_t` не становится dangling address после reallocation. Но если insert/erase меняет порядок элементов, старый index может указывать уже на другой object.

Иными словами: pointer invalidation — memory/lifetime problem; stale index — logic/invariant problem.

Оба случая требуют явного design.

14.44 Pipeline: filter → sort → transform → aggregate

Реальный processing pipeline можно писать по шагам:

```
1 std::vector<CourseBlock> selected;
2
3 std::copy_if(
4     blocks.begin(),
5     blocks.end(),
6     std::back_inserter(selected),
7     [](const CourseBlock& block) {
8         return block.examples >= 2;
9     }
10 );
11
12 std::sort(
13     selected.begin(),
14     selected.end(),
15     [](const CourseBlock& a, const CourseBlock& b) {
16         return a.number < b.number;
17     }
18 );
```

Дальше transform получает slugs, accumulate считает examples.

Промежуточный vector стоит памяти/copies, но делает pipeline прозрачным. Не надо раньше времени превращать код в нечитаемую цепочку abstractions.

14.45 Algorithms и mutation

Некоторые algorithms только читают: `find_if`, `count_if`, `all_of`.

Другие меняют elements через iterators: `sort`, `transform in-place`, `remove_if`.

Но structural properties container, например size, generic algorithm обычно не меняет сам. Разделяйте element mutation и container structural mutation.

14.46 Iterator lifetime и owner lifetime

Iterator — non-owning handle. Container должен продолжать существовать, а operation не должна была invalidate iterator.

```
1 std::vector<int>::iterator bad();
2
3 std::vector<int>::iterator bad() {
4     std::vector<int> local{1, 2, 3};
5     return local.begin(); // iterator переживёт container
6 }
```

Это тот же lifetime принцип, что raw pointer/reference из ранних блоков.

14.47 End iterator тоже может invalidated

Нельзя думать, что только iterators на «реальные элементы» важны. Past-the-end iterator также относится к container state и может быть invalidated операциями.

Поэтому сохранять end() до structural mutation и использовать после неё без проверки contract опасно.

14.48 Const-correct algorithms

Если algorithm только читает, передавайте const range/container там, где это естественно.

```
1 int total_examples(const std::vector<CourseBlock>& blocks) {
2     return std::accumulate(
3         blocks.cbegin(),
4         blocks.cend(),
5         0,
6         [](int total, const CourseBlock& block) {
7             return total + block.examples;
8         }
9     );
10 }
```

Это делает невозможной случайную mutation элементов через iterator.

14.49 Не копировать container без причины

Параметр:

```
1 void print(std::vector<CourseBlock> blocks);
```

создаёт своё value, что может означать copy/move container.

Если функция только читает:

```
1 void print(const std::vector<CourseBlock>& blocks);
```

Но generic algorithm APIs часто лучше принимать iterator pair или template range-like abstraction, если нужна независимость от конкретного container.

14.50 Когда конкретный container type в API уместен

Если функция использует vector-specific guarantees: contiguous data; random access; capacity/reserve.

тогда vector<T>& в API может быть честным contract.

Не нужно делать всё template-generic только ради generic programming.

14.51 Exception safety

Standard containers управляют своими resources через RAII. Но сильная/basic exception guarantee конкретной operation зависит от type properties и самой operation.

Например, vector reallocation может copy/move elements. но except move из блока 12 может влиять на выбранный library strategy.

Не утверждайте универсально «любая STL операция strong exception safe».

14.52 Allocators: только первый взгляд

Containers parameterized allocator type, например:

```
1 std::vector<int, Allocator>
```

Allocator abstraction управляет получением/освобождением storage для container. Полный allocator model сложен и будет кратко рассмотрен в блоке 15.

В обычном application code default allocator достаточен, пока profiling или специальная memory architecture не доказали другое.

14.53 Практический выбор container

Начальная эвристика: dynamic sequence — vector; fixed sequence — array; efficient both-end queue — deque; node stability/splice-specific workload — рассмотреть list; ordered key lookup — map/set; unordered exact-key lookup — unordered_map/unordered_set.

Потом проверяются: complexity, invalidation, ordering, locality, allocations и actual workload.

14.54 Эксперимент 1: course_catalog_pipeline

Каталог: examples/14_stl_containers_iterators_algorithms/01_course_catalog_pipeline.

Программа строит прикладной каталог 15 blocks: 1) source of truth — vector<CourseBlock>; 2) number index — unordered_map<int, size_t>; 3) find_if и index дают два способа query; 4) copy_if фильтрует готовые blocks; 5) sort упорядочивает выборку; 6) transform получает slugs; 7) accumulate считает число examples; 8) map строит ordered summary по status; 9) optional<CourseBlock> представляет missing query.

```
cd examples/14_stl_containers_iterators_algorithms/01_course_catalog_pipeline
./run_checks.sh
```

14.55 Эксперимент 2: invalidation_and_erase_lab

Каталог: `examples/14_stl_containers_iterators_algorithms/02_invalidation_and_erase_lab`.

Safe executable проверяет: `vector::reserve` и `growth` без `reallocation` внутри `capacity`; `forced reallocation` и изменение `data()` `address` без использования старого `pointer`; `erase-remove idiom`; возвращаемый `iterator erase`; `list iterator/reference stability` при вставке другого `element`; `unordered-map rehash observation` без использования `invalidated iterator`.

Отдельный `ASan-only executable` намеренно разыменовывает `pointer` после `forced vector reallocation` и должен дать `use-after-free diagnostic`.

Ещё один `negative compile file` подтверждает, что `std::sort` нельзя применить к `std::list iterators`.

```
cd examples/14_stl_containers_iterators_algorithms/02_invalidation_and_erase_lab
./run_checks.sh
```

14.56 Что проверяет практика, а что нет

Experiments подтверждают поведение текущей `toolchain` и конкретных `inputs`. Они не заменяют `standard guarantee`.

Например: изменение `data()` действительно показывает `reallocation` в `case`, который специально превышает `capacity`; `ASan diagnostic` показывает конкретную `lifetime ошибку`; отсутствие `sanitizer report` в `safe run` не доказывает отсутствие всех `UB`; `measured addresses` не превращаются в переносимое обещание `allocator`.

14.57 Итоговый checklist STL

Перед выбором `container` спросите: какие операции доминируют; какой `complexity contract` нужен; нужен ли `ordering`; нужна ли `pointer/reference stability`; важна ли `contiguous storage/locality`; как часто меняется структура `container`.

Перед использованием `iterator`: жив ли `owner container`; не было ли `invalidating operation`; удовлетворяет ли `iterator category` требованиям `algorithm`; не сохранили ли мы `stale end()`.

Перед написанием `manual loop`: есть ли `standard algorithm`, который точнее выражает намерение; нужен ли `predicate/lambda` из блока 13; `structural mutation` отделена от `element processing`.

Главный вывод

STL — не список удобных функций. Это система `contracts` между `storage`, `positions`, `algorithms` и `callable policies`. Хороший C++ `code` выбирает `container` по требованиям, использует `algorithms` для намерения и всегда учитывает `complexity` и `invalidation`.

Следующий, последний базовый блок соберёт современный C++ в единую картину: `constexpr`, C++20 `Concepts`, обзор `coroutines` и `modules`, `allocators`, `tooling`, дальнейший маршрут обучения и критерии выбора возможностей языка.

15 Лекция 15. Современный C++ после C++17

Основной стандарт курса — C++17. К этому моменту у нас уже есть фундамент: типы, lifetime, RAII, value semantics, smart pointers, move semantics, templates, lambdas, STL, complexity и iterator invalidation.

Финальная лекция отвечает на другой вопрос: **что реально добавил C++20 поверх этой модели и зачем это понадобилось?**

Итог

Новые стандарты не отменяют старые правила. Concept не исправляет плохой API, range не отменяет lifetime, coroutine не означает отдельный thread, а module не отменяет build graph. Современный C++ добавляет более точные инструменты поверх прежних contracts.

15.1 Стандарт и реальная toolchain

Нужно различать четыре вещи: feature входит в стандарт; compiler реализует language feature; standard library реализует library component; build system умеет корректно собрать project с этой feature.

Поэтому `-std=c++20` — необходимое, но не всегда достаточное условие. Особенно это заметно на modules.

Поддержку проверяют по документации, feature-test macros и минимальному compile probe в своей toolchain.

15.2 Feature-test macros

Пример:

```
1 #ifdef __cpp_concepts
2 static_assert(__cpp_concepts >= 201907L);
3 #endif
```

Макро сообщает о feature support implementation. Конкретные значения следует сверять с документацией/стандартными таблицами, а не запоминать навсегда.

15.3 constexpr: compile-time возможность, не приказ

```
1 constexpr int square(int value) {
2     return value * value;
3 }
4
5 static_assert(square(5) == 25);
6
7 int input = read_value();
8 int runtime_result = square(input);
```

Первый call обязан дать constant expression. Второй может быть обычным runtime call.

constexpr не равно «всегда compile time»

Keyword разрешает constant evaluation при выполнении правил языка. Он не означает, что каждый вызов обязательно исчезнет из runtime.

15.4 Как развивался constexpr

C++11 ввёл достаточно строгий вариант. C++14/17/20 постепенно разрешали всё больше обычного imperative code и library operations в constant evaluation.

Это делает compile-time programming похожее на обычный C++, но I/O и произвольные runtime resources не становятся compile-time операциями.

15.5 if constexpr: напоминание из C++17

```
1 template <typename T>
2 void print_value(const T& value) {
3     if constexpr (std::is_integral_v<T>) {
4         std::cout << "integer: " << value;
5     } else {
6         std::cout << value;
7     }
8 }
```

Discarded branch не требует той же well-formedness для текущего instantiation, что обычная runtime ветка. Это важный мост от C++17 templates к C++20 Concepts.

15.6 consteval: immediate function, C++20

```
1 consteval int make_id(int group, int number) {
2     return group * 1000 + number;
3 }
4
5 constexpr int id = make_id(12, 34);
```

Если relevant call не может быть constant expression, program ill-formed:

```
1 int x = read_value();
2 // int id = make_id(1, x); // compile error
```

consteval нужен, когда compile-time evaluation входит в контракта, а не просто возможно-стью.

15.7 constexpr: другой contract

```
1 constexpr int global_counter = 0;
2 ++global_counter; // допустимо
```

constexpr требует static initialization для объекта со static/thread storage duration, но не делает object const.

Keyword	Смысл	Runtime mutation
constexpr	constant-expression object/function capability	object нет
constexpr	immediate function	относится к calls
constinit	static initialization requirement	может быть да

15.8 Static initialization order

constinit полезен, когда global/static object должен гарантированно получить static initialization. Если initializer внезапно требует dynamic initialization, получаем diagnostic.

Но keyword не делает global mutable state хорошим design автоматически.

15.9 Почему появились Concepts

Старый generic API:

```
1 template <typename T>
2 auto twice(const T& value) {
3     return value + value;
4 }
```

Из signature не видно requirement на operator+. Ошибка может проявиться глубоко в instantiation.

До C++20 применяли documentation, static_assert, type traits и SFINAE/enable_if. Concepts позволяют сделать requirements частью interface.

15.10 Named Concept

```
1 template <typename T>
2 concept Numeric =
3     std::integral<T> || std::floating_point<T>;
```

Concept — compile-time predicate над template arguments, участвующий в constraint system.

15.11 Constrained template

```
1 template <Numeric T>
2 T twice(T value) {
3     return value + value;
4 }
```

Или:

```
1 template <typename T>
2 requires Numeric<T>
3 T twice(T value) {
4     return value + value;
5 }
```

Неподходящий type rejected на уровне constraints.

15.12 Abbreviated function template

```
1 auto twice(Numeric auto value) {  
2     return value + value;  
3 }
```

Это всё ещё compile-time generic function, а не dynamic typing.

15.13 Requires-expression

```
1 template <typename T>  
2 concept HasSize = requires(const T& value) {  
3     { value.size() } -> std::convertible_to<std::size_t>;  
4 };
```

Expression не выполняется runtime: compiler проверяет, корректна ли требуемая операция и удовлетворяет ли result constraint.

15.14 Виды requirements

Simple requirement:

```
1 requires(T value) {  
2     ++value;  
3     value++;  
4 }
```

Type requirement:

```
1 requires {  
2     typename T::value_type;  
3 }
```

Compound requirement:

```
1 requires(const T& value) {  
2     { value.size() } -> std::convertible_to<std::size_t>;  
3 }
```

Nested requirement:

```
1 requires {  
2     requires std::integral<T>;  
3     requires sizeof(T) <= sizeof(int);  
4 }
```

15.15 Concepts не доказывают semantics

Compiler может проверить наличие $a < b$, но не способен из одного `requires-expression` доказать, что `comparison` задаёт корректный `strict weak ordering` для всех `values`.

Semantic requirements STL из блока 14 остаются частью контракта programmer.

15.16 Concepts и overload resolution

```
1 template <typename T>
2 void describe(const T&) {
3     std::cout << "generic";
4 }
5
6 template <std::integral T>
7 void describe(const T&) {
8     std::cout << "integral";
9 }
```

Для `integral argument constrained overload` может быть выбран как более подходящий. Это `compile-time selection`, не `virtual dispatch`.

15.17 Concept или `static_assert`

`static_assert` хорош для `invariants` и локальных `compile-time` условий.

Concept хорош, когда `requirement` относится к `template interface` и должен участвовать в `overload resolution`.

Они дополняют друг друга.

15.18 Concept и runtime polymorphism

Concept описывает `compile-time structural requirements`. Virtual base class из лекции 7 — `runtime nominal polymorphism`.

Если нужно хранить `heterogeneous objects` в runtime, Concept сам по себе не заменяет `virtual/type-erasure abstraction`.

15.19 Стоимость Concepts

Сам `constraint` не добавляет `runtime branch`. Проверка происходит `compile time`.

Но `templates/constraints` имеют `compile-time cost`: `instantiations`, `diagnostics`, `build time` и возможный рост `code size` для разных `specializations`.

15.20 Ranges: следующий слой над STL

C++17:

```
1 std::sort(values.begin(), values.end());
```

C++20:

```
1 std::ranges::sort(values);
```

Range interface уменьшает повторение begin/end и использует Concepts для формализации требований.

15.21 Ranges algorithms

```
1 auto it = std::ranges::find(values, 42);
```

Algorithms по-прежнему имеют complexity и iterator/range requirements. Новый interface не отменяет фундамент блока 14.

15.22 Projection

```
1 std::ranges::sort(  
2     blocks,  
3     std::less{},  
4     &CourseBlock::number  
5 );
```

Projection отдельно говорит, какое поле считать sort key. Это часто проще comparator lambda.

15.23 Views

```
1 auto selected =  
2     values  
3     | std::views::filter([](int x) {  
4         return x % 2 == 0;  
5     })  
6     | std::views::transform([](int x) {  
7         return x * x;  
8     });
```

Views обычно описывают lazy processing, не создавая промежуточный container на каждом шаге.

View не означает ownership

View часто заимствует source. Source должен жить достаточно долго и не быть invalidated. Ranges не отменяют lifetime.

15.24 Lazy evaluation

Transformation обычно выполняется при iteration. Это может убрать лишние copies/allocations и позволить early termination.

Но многэтажный pipeline не всегда читабельнее простого loop. Modern syntax нужно выбирать по ясности и требованиям.

15.25 Borrowed ranges и dangling

Ranges library умеет выражать некоторые случаи, где iterator в уничтоженный temporary небезопасен, например через `std::ranges::dangling`.

Это полезная защита, но не полный lifetime verifier.

15.26 std::span, C++20

`span` — non-owning view на contiguous sequence:

```
1 int sum(std::span<const int> values) {  
2     return std::accumulate(values.begin(), values.end(), 0);  
3 }
```

Caller может передать compatible array или vector без копирования элементов.

Span — borrow

`std::span` не владеет underlying storage и не продлевает lifetime. Reallocation vector или уничтожение owner может сделать span dangling.

15.27 Static extent

```
1 std::span<const int, 4> values;
```

Размер может быть частью type, если fixed extent действительно часть contract.

15.28 Designated initializers, C++20

```
1 struct Config {  
2     int retries;  
3     bool verbose;  
4 };  
5  
6 Config config{  
7     .retries = 3,  
8     .verbose = true  
9 };
```

Имена members повышают читаемость aggregate initialization.

В C++20 designators должны идти в declaration order. Это отличается от некоторых C patterns.

15.29 Defaulted operator<=>

```
1 struct Version {  
2     int major;  
3     int minor;  
4 }
```

```
5 auto operator<=>(const Version&) const = default;  
6 };
```

Для подходящего value type compiler может синтезировать comparison по members.

Не каждому domain type нужен total ordering: semantics важнее удобного syntax.

15.30 std::erase_if

C++17 erase-remove:

```
1 values.erase(  
2     std::remove_if(values.begin(), values.end(), predicate),  
3     values.end()  
4 );
```

C++20:

```
1 std::erase_if(values, predicate);
```

Новый API короче, но понимание C++17 idiom объясняет устройство старого кода и разделение algorithm/container mutation.

15.31 std::string::starts_with

```
1 if (name.starts_with("lecture_")) {  
2     // ...  
3 }
```

Новые стандарты часто добавляют маленькие library utilities. Перед написанием домашнего helper проверьте standard library нужной версии.

15.32 std::source_location

C++20 даёт typed source-location API для logging/diagnostics:

```
1 void log(  
2     std::string_view text,  
3     std::source_location where =  
4     std::source_location::current());
```

Это современная альтернатива многим macro-based wrappers, но logging policy и стоимость всё равно проектируются отдельно.

15.33 std::jthread

std::jthread связывает thread lifetime с RAI: destructor join-ит joinable thread. C++20 также добавляет cooperative stop tokens.

Но concurrency требует отдельного изучения memory model, data races, atomics и synchronization. Одной секцией это не закрыть.

15.34 Coroutines: что это

C++20 добавляет: `co_await`; `co_yield`; `co_return`.

Coroutine — function, execution которой может suspend/resume.

Coroutine не равно thread

Coroutine сама по себе не обещает background execution, parallelism или async I/O. Scheduler/executor/event loop — отдельный слой.

15.35 Как compiler видит coroutine

Compiler концептуально превращает function в state machine и создаёт coroutine frame для состояния, которое должно пережить suspension.

Frame может содержать parameters, locals, bookkeeping и promise object. Конкретный layout/allocation — implementation detail semantics.

15.36 Promise type

Return type coroutine через coroutine protocol определяет `promise_type`. Promise сообщает compiler, как создать return object, initial/final suspend, обработать yield/return и exception.

Поэтому low-level coroutine example быстро становится разбором protocol, а не самой идеи suspend/resume.

15.37 co_await

Awaiter protocol концептуально отвечает на три вопроса: готов ли result, нужно ли suspend и что вернуть после resume.

В application code обычно работают с готовыми task/awaitable types framework, а не пишут protocol вручную в каждом месте.

15.38 co_yield

Generator-like coroutine выдаёт values по одному, сохраняя state между возобновлениями. C++20 задаёт language mechanism, но не универсальный `std::generator` container-like abstraction.

15.39 Lifetime coroutine

Coroutine frame — resource. Нужно знать: кто владеет handle/frame; когда вызывается destroy; какие references переживают suspension; живы ли owners borrowed objects.

Современный syntax снова приводит к ownership/lifetime.

15.40 Modules: зачем

Header model основан на textual inclusion и несёт repeated parsing, macro leakage и сложные include dependencies.

C++20 modules вводят language-level import/export model:

```
1 export module math;
2
3 export int add(int a, int b) {
4     return a + b;
5 }
```

Consumer:

```
1 import math;
```

15.41 Module не равен precompiled header

PCН оптимизирует traditional header model. Module имеет собственную language semantics интерфейса/import.

Они не являются одним механизмом.

15.42 Почему modules зависят от tooling

Compiler/build system должен построить dependency graph module units и intermediate module artifacts в корректном порядке.

Поэтому module workflow изучают вместе с конкретными версиями compiler и CMake. В этом курсе modules остаются обзорной темой, а не фиктивной «одной командой».

15.43 Headers продолжают жить

Большой ecosystem остаётся header-based. Реальный project может смешивать modules, classic libraries, header-only dependencies и C APIs.

Migration — engineering decision.

15.44 Allocators

Containers имеют allocator parameter. Allocator отвечает за storage acquisition/release policy, но не заменяет ownership model объектов.

```
1 std::vector<int, MyAllocator<int>> values;
```

Полный allocator model включает traits и propagation rules; это отдельная advanced тема.

15.45 std::pmr

Polymorphic memory resources появились уже в C++17. Они позволяют выбирать memory resource runtime и полезны для arena/monotonic allocation workloads.

Custom memory strategy имеет смысл после измерения реальной allocation problem, а не «потому что быстрее».

15.46 Compile-time cost

Современные templates, Concepts и ranges могут увеличивать compiler work: instantiations, constraint checking, сложные view types, diagnostics.

Это compile-time cost, не обязательно runtime cost.

15.47 Runtime cost нельзя угадывать по modern syntax

View может убрать промежуточный container, а может усложнить optimization. `std::function` может ввести indirect call. Coroutine может иметь frame.

Performance проверяют profiling/benchmark/assembly, а не возрастом стандарта.

15.48 Эксперимент 1: `constexpr` and `concepts` lab

Каталог: `examples/15_modern_cpp_and_next_steps/01_constexpr_and_concepts_lab`.

Проверяется: `constexpr` compile-time и runtime calls; `constexpr`; mutable `constexpr`; standard/custom Concepts; `requires-expression`; constrained overload; real compile rejection для неподходящего type; real compile rejection runtime argument у `constexpr`.

```
cd examples/15_modern_cpp_and_next_steps/01_constexpr_and_concepts_lab
./run_checks.sh
```

15.49 Эксперимент 2: `cpp20_feature_tour`

Каталог: `examples/15_modern_cpp_and_next_steps/02_cpp20_feature_tour`.

Проверяется: `std::ranges::sort` с projection; filter/transform views; `std::span<const int>`; designated initializers; defaulted operator`<=>`; `std::erase_if`; `std::string::starts_with`; real compile failure неправильного порядка designators.

```
cd examples/15_modern_cpp_and_next_steps/02_cpp20_feature_tour
./run_checks.sh
```

15.50 Почему `coroutines/modules` не основные experiments

Concepts/ranges/constexpr хорошо проверяются обычным portable C++20 target. Modules сильнее зависят от build workflow. Coroutine protocol требует дополнительной library abstraction и легко уводит финальную лекцию в устройство `promise_type` вместо общей картины.

Поэтому их mechanics и границы разобраны честно, но без притворства, что тема «освоена» одним игрушечным executable.

15.51 Что появилось дальше: C++23

После C++20 развитие продолжается. Например, C++23 включает: `std::expected`; новые ranges facilities; `std::mdspan`; дальнейшие constexpr/library improvements.

Каждую такую feature нужно явно маркировать версией стандарта.

15.52 `std::expected`: следующий error contract

В блоке 10 были `status/exceptions`, в блоке 13 — optional «есть/нет».

`expected<T,E>` выражает value `T` или error `E`. Это следующий логичный шаг, когда caller обязан узнать причину failure.

15.53 Как читать новую feature

Рабочий алгоритм: 1) определить проблему старого подхода; 2) узнать `standard/version`; 3) прочитать `semantics`; 4) проверить `ownership/lifetime`; 5) понять `compile/runtime cost`; 6) сделать минимальный `compile experiment`; 7) только потом использовать в `production`.

15.54 Что должно остаться после курса

Нужно уметь объяснить: `declaration/definition` и `compile/link/runtime errors`; `object lifetime`, `ownership`, `pointer/reference/const contracts`; `RAII`, `copy/move` и `value categories`; `runtime` и `compile-time polymorphism`; `templates` и `Concepts`; `exception boundaries`; `lambdas/callables`; `STL container/iterator/algorithm contracts`; `complexity` и `invalidation`; зачем `ranges/Concepts` появились после C++17.

15.55 Дальше: concurrency

Следующая самостоятельная область: `threads`, `mutexes`, `condition variables`, `atomics`, `happens-before`, `memory ordering`, `lock-free structures`, `schedulers/executors`.

Её нельзя безопасно изучить только через «запустили два `thread`».

15.56 Дальше: performance engineering

`Profiling`, `hardware counters`, `cache behavior`, `branch prediction`, `vectorization`, `assembly`, `allocation profiling` и `realistic benchmarks`.

Сначала измерение, затем `optimization`.

15.57 Дальше: library/API design

`ABI/API boundaries`, `pImpl`, `type erasure`, `customization points`, `error models`, `ownership across library boundaries`, `allocator-aware design` и `versioning`.

15.58 Дальше: advanced templates

`Variadic templates`, `fold expressions`, `advanced traits`, `specialization`, `constexpr algorithms` и `policy-based design`.

`Concepts` помогают выразить `requirements`, но не отменяют эти механизмы.

15.59 Дальше: tooling

`Warnings`, `tests`, `sanitizers`, `debugger`, `static analysis`, `formatter`, `CMake`, `CI` и `profiling` остаются частью профессионального C++, а не внешним дополнением к языку.

15.60 Что значит знать современный C++

Не помнить все keywords, а уметь: выбрать простейший корректный abstraction; понимать owner/lifetime; понимать value/reference semantics; читать diagnostics; знать complexity contract; различать compile-time и runtime mechanisms; проверять support стандарта; измерять performance claims.

15.61 Совместимость проекта

Если основной project обещает C++17, C++20 experiments отделяют явно: отдельный CMake project/target; cxx_std_20; README с required standard; отсутствие C++20 API в C++17 targets.

15.62 Финальный checklist новой feature

Перед использованием спросите: 1) какую проблему решает; 2) в каком стандарте появилась; 3) поддерживает ли toolchain; 4) какие compile-time requirements; 5) какие lifetime/runtime consequences; 6) есть ли более простое решение; 7) улучшает ли новый вариант contract; 8) как его тестировать.

Финальный вывод курса

C++ остаётся языком явных trade-offs. Новые стандарты дают больше способов выразить намерение compile time и точнее ограничить generic code, но фундамент остаётся тем же: lifetime, ownership, types, contracts, algorithms, complexity и tooling. Если основы понятны, новые стандарты перестают быть списком магических keywords.

На этом базовая линия из 15 блоков завершена. Дальше выбирайте отдельную ветку — concurrency, performance, library design, networking, graphics, embedded C++ или другую предметную область — и строить её тем же способом: theory, runnable experiments, diagnostics и проверяемые guarantees.